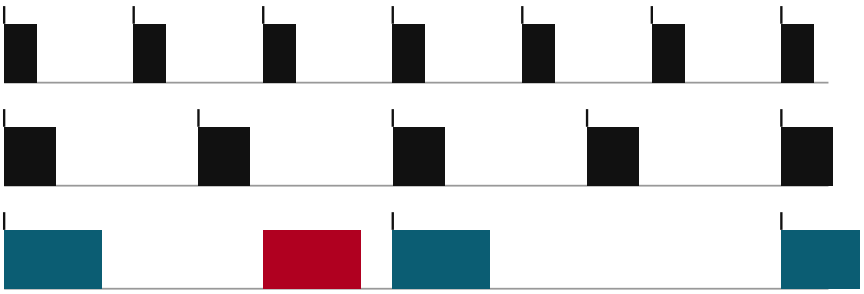
SWE EXCELLENCE · REAL-TIME FUNDAMENTALS

ON TIME

Timing, scheduling, messaging, synchronization, reliability and observability — for software engineers who build on top of an RTOS.



1 **Timing**
2 **Scheduling**
3 **Messaging**
4 **Synchronization**
5 **Reliability**
6 **Observability**

Reference implementation: FreeRTOS on QEMU. Every [RAN] result was executed; every claim traces to research/.

Contents

Read this first 3

PART 1 · TIME

1 What “real-time” means 7
2 The simplest program: a loop 11
3 Measuring time 15
4 Interrupts: reacting on time 19
5 Many jobs, one CPU: the cyclic executive 23

PART 2 · ORDER

6 Tasks and the kernel 26
7 Priorities and preemption 30
8 Utilization and the rate-monotonic bound 33
9 Deadlines and response-time analysis 36
10 EDF and the other policies 40
11 Overload: when the work does not fit 44

PART 3 · SHARE

12 Shared state and races 48
13 Priority inversion and deadlock 52
14 Memory: stacks, heaps and budgets 57

PART 4 · FLOW

15 Queues and backpressure 61
16 Publish/subscribe and quality of service 65
17 Rates and end-to-end latency 69
18 Lifecycles: components and messages 73

PART 5 · TRUST

19 Reliability: watchdogs, containment and recovery 77
20 Observability: proving it meets its requirements 82
21 Beyond one CPU: buses, clocks and other computers 87

PART 6 · PUT IT TOGETHER

22 Reading an unfamiliar framework 91
23 The implicit requirements 95
24 A worked system: taking over a pipeline 100

APPENDICES

A C survival kit for application engineers 104
B Formulas and numbers to keep 106
C FreeRTOS quick map 108
D Glossary 111
E Evidence ledger and sources 113
F Where to go next: ranked reading 115

Read this first

You are about to write application and middleware code on top of a real-time operating system, inside a framework someone else designed. You will not invent the scheduler. You must still know what it promises, what it quietly assumes, and what happens when your code breaks the assumption. This book is the shortest honest route to that knowledge.

The core idea

A real-time system is correct only if it is correct **and on time**. Everything in this book is a way of answering one question with evidence:

> *Will this component do the right thing, at the right moment, every time — including on the worst day?*

You will be handed requirements that never mention most of the things that decide the answer: how often a component runs, what it waits on, how big its queue is, what happens when the queue is full, who else touches its data, what restarts it when it hangs. Those are **implicit requirements**. Missing one is how a correct-looking change becomes an incident. Chapter 23 collects them in one list; every earlier chapter earns one or two.

The six questions

Every chapter belongs to one of six questions. Learn to ask them about any component, in any framework, in any order.

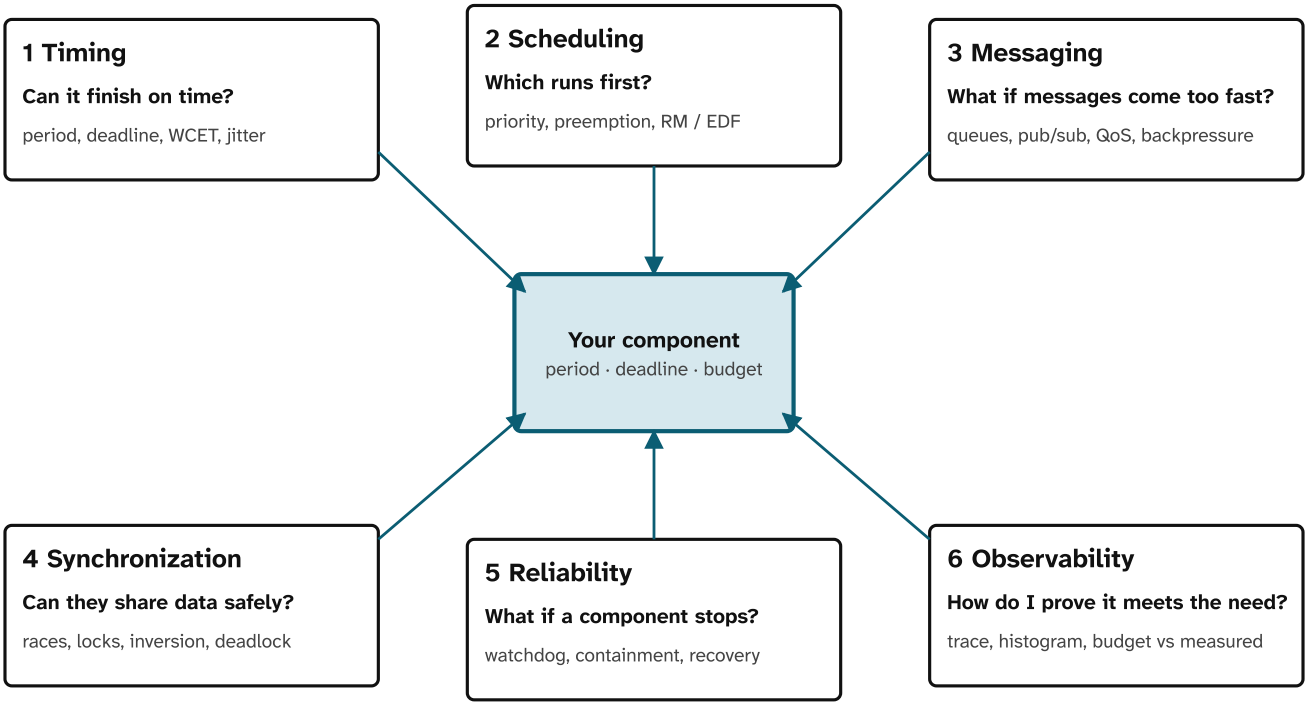


Figure 1. The six questions. Each one has a chapter group, a handbook checklist, and at least one lab.

#	Question	You can answer it when you can state...	Chapters
1	**Timing**: can my component finish on time?	period, deadline, execution-time budget, jitter limit	1–5, 17
2	**Scheduling**: which component runs first?	policy, priorities, preemption rules, worst-case response time	6–11
3	**Messaging**: what if messages arrive too fast?	queue depth, overflow policy, QoS, age limit	15–16, 18
4	**Synchronization**: can components share data safely?	owner, lock or copy rule, blocking time, lock order	12–14
5	**Reliability**: what if a component stops?	detector, reaction, containment, recovery	19
6	**Observability**: how do I prove the system meets its requirements?	one number per requirement, with a histogram, not an average	20

How the book is organised

Part	Theme	Starts from	Ends at
1 · Time	What “on time” means and how a program finds out what time it is	a loop that prints	a cyclic executive

Part	Theme	Starts from	Ends at
2 · Order	Who runs when	the first kernel and its tasks	rate-monotonic analysis, EDF, overload
3 · Share	What two pieces of code can safely do to the same data	a race	priority inversion, memory
4 · Flow	How data moves between components	a queue	publish/subscribe, rates, lifecycles
5 · Trust	Failing safely, and proving things	the watchdog	measurement, multi-CPU systems
6 · Put it together	Reading a framework, implicit requirements, a worked system	an unfamiliar codebase	a pipeline you debug to its requirements

The chapters follow history on purpose. Each mechanism appears when the previous one fails: a loop cannot keep time, so we add a timer; a timer cannot interleave jobs, so we add interrupts; interrupts cannot share fairly, so we add a scheduler; a scheduler lets jobs corrupt each other's data, so we add locks; locks invert priorities, so we add protocols. If you know *why* each piece exists you can recognise it in a framework that renamed it.

Three documents, one method

Document	Use it to	Style
ON TIME (this book)	learn the discipline, in order	read it, with a pen
Field Handbook	act on the first day with a new device or framework	checklists, questions, forms
Lab Workbook	prove each idea to yourself on a real RTOS	run, measure, compare with prediction

Marks you will see

Mark	Meaning
RAN	I ran this and kept the output (`labs/transcripts/`). Runs are deterministic: you will see the same numbers.
MANUAL	Checked against documentation but **not executed** here (for example, a VS Code configuration). Verify before relying on it.
SEC	Taken from a secondary source (a news report, an expert's slides). The source and its weakness are named.
EVIDENCE	A measured or published result, with its size and its limits.
CASE FILE	A real incident and what it teaches.

Mark	Meaning
RULE / TRAP / TRY IT / CHECK	A thing to do always, a common mistake, a small experiment, a self-test.

What this book does not do

- It does not teach assembly or hardware design. Chapter 14 and Appendix A give just enough C and memory to read an RTOS application.
- It does not replace your framework's documentation. Chapter 22 teaches you to read it quickly.
- It does not promise numbers from real boards. The labs run on an emulator with a deterministic clock; they prove *relationships* (model against measurement), not any chip's speed.
- Where evidence is thin or a source could not be checked, it says so. Appendix E lists every source and its status; `research/09-claims-we-could-not-verify.md` lists everything we could not confirm.

A study plan

Week	Read	Do	You should be able to
1	Part 1 (ch. 1–5)	Labs H1, H2, R0, R1	explain why a loop drifts, how a tick works, and what a frame overrun does
2	Part 2 (ch. 6–9)	Labs H3, R2, R8	compute a response time by hand and compare it with a measurement
3	Part 2 (ch. 10–11), Part 3	Labs R3, R6, R7, R9	name an inversion from a timeline and find it with gdb
4	Part 4 (ch. 15–18)	Labs H4, R4	size a queue from a burst and say what happens when it is full
5	Part 5	Labs H5, R5	design a watchdog that catches a stuck task and report latency honestly
6	Part 6	Lab R10 and the Handbook	take over an unfamiliar pipeline and prove it meets its requirements

Six weeks at an hour a day. If you have one afternoon, read chapters 1, 7, 9, 13, 15, 19, 20 and 23, and keep the Handbook open.

What “real-time” means

A real-time system is one whose correctness depends on *when* a result appears, not only on *what* it is. That single sentence changes how you read requirements, how you test, and what you are allowed to assume.

Correct and on time

Most software is judged by its output. A report that takes two seconds instead of one is slower, not wrong. A real-time component has a **deadline**: a result that arrives after it is a *failure*, even if every bit is right. The definition is usually credited to Stankovic’s 1988 paper, “Misconceptions About Real-Time Computing”; its best-known point is that real-time computing is not the same as fast computing.

It is not. Speed is an average-case property. Real-time is a **worst-case** property measured against a stated deadline. A controller that answers in 1 ms on average and 60 ms once an hour has a fast average and a missed deadline.

Class	If the deadline is missed	Example you will meet
Hard	the result is wrong or the system is unsafe	a motor-control loop, a safety interlock, a bus transaction cycle
Firm	the result is worthless but nothing breaks; the miss is counted	a video frame, a periodic sensor sample used in a filter
Soft	the result is still useful, only less valuable	a display refresh, a log flush

The class is a **requirement**, and it is rarely written down. You usually have to ask.

The timing contract of a component

Every component that does periodic or event-driven work has a contract. Whether or not it is written, it exists.

Term	Meaning	Typical symbol
Trigger	what starts a job: a timer (periodic), an event (sporadic), another component’s output	—

Term	Meaning	Typical symbol
Period	time between releases for a periodic component; **minimum inter-arrival** for a sporadic one	T
Deadline	latest acceptable completion, relative to release (often equal to T)	D
Budget (WCET)	the execution time you promise the job never exceeds	C
Response time	completion time minus release time, *including* waiting for the CPU	R
Jitter	variation in release or completion time between jobs	J
Miss policy	what happens when R > D: skip, abort, continue late, reset	—

The whole subject is one inequality: **R ≤ D**. Chapters 6–11 teach how to compute and bound R; chapters 3 and 20 teach how to measure it; chapters 12–19 explain what makes R larger than you expected.

> **RULE** **Rule.** A component without a stated period, deadline, budget and miss policy is not finished. When the spec is silent, write down what you assume and get it confirmed.

What the theory assumes (and what breaks it)

The first complete result in this field is Liu and Layland's 1973 paper. It proves what is schedulable under five assumptions. They double as a checklist of what real systems violate.

Assumption (Liu & Layland, 1973)	Where real code breaks it
(A1) hard-deadline requests are **periodic** with a constant interval	events arrive in bursts; timers drift (ch. 3)
(A2) the deadline is the **next request** of the same task	work backs up; the next release arrives before the last job finished (ch. 11)
(A3) tasks are **independent**	locks, shared buffers, queues (ch. 12–15)
(A4) run-time per task is **constant** (interpreted as the maximum)	caches, data-dependent branches, a rare slow path (ch. 9, 20)
(A5) non-periodic work is special: initialization and recovery	logging, retries, error handling at the worst moment

Later chapters remove assumptions one at a time: blocking (A3) in chapter 13, varying execution time (A4) in chapters 9 and 20.

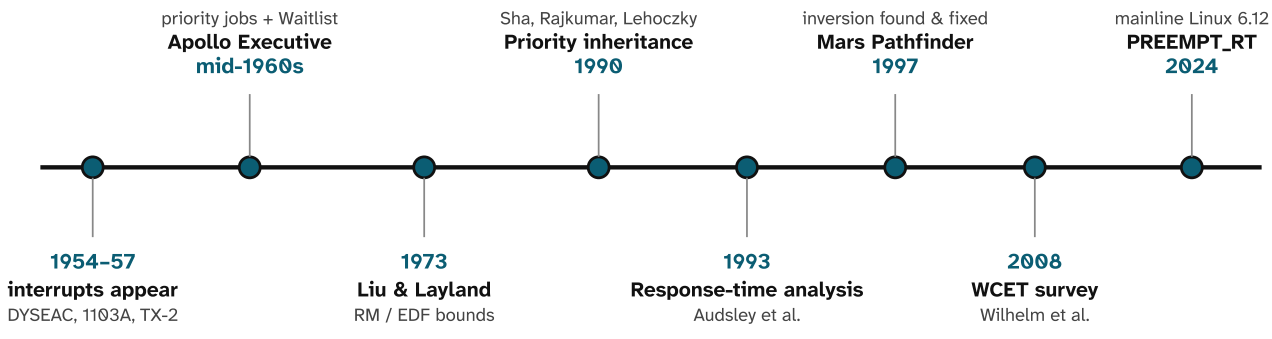
Why not simply be fast?

Three reasons the answer is “no”.

1. **You cannot test the worst case by running the average case.** The worst case needs a rare combination: a slow path, a burst, a preemption at the wrong instant. Pathfinder’s flight-software lead wrote that the failure “would only manifest itself when ASI/MET data was being collected and intermediate tasks were heavily loaded”, and that pre-launch testing used the “best case” data rates.
2. **Speed does not remove the ordering problem.** A 10× faster CPU runs a priority inversion 10× faster. It still blocks the high-priority task for as long as it takes the medium-priority work to finish.
3. **Fast hardware is not the constraint you were given.** You are handed a processor, a framework and a rate. Your job is to show your component fits.

CASE FILE **Apollo 11, 1969: the machine that shed work.** During the powered descent the computer raised program alarms 1201 and 1202. The guidance software’s authors later explained the cause: an interface to the rendezvous radar, left in the wrong configuration, “stole approximately 13% of the computer’s duty cycle” from a margin that had shrunk to about 10%. The software kept flying. The reason is a design decision made years earlier: a **priority-driven executive** whose jobs ran highest-priority first, with a fixed pool of eight job slots and a restart mechanism that kept critical jobs and dropped the rest. The authors contrast it with a “boxcar” executive, which “breaks down completely as soon as any function takes longer than the time it is allocated.” (Eyles 2004.) You will meet this design twice more: as the cyclic executive in chapter 5 and as overload handling in chapter 11.

From the loop to the RTOS: why each piece exists



Dates: interrupts per Smotherman; others per the papers and reports in research/01. Product release years are omitted (not verified).

Figure 2. Milestones used in this book. Interrupt dates follow Smotherman’s history; the rest are from the papers and reports cited in research/01.

The sequence is the plot of Parts 1 to 3:

1. **A loop** polls inputs and does work. Its period is whatever the code takes (ch. 2).
2. **A hardware timer** lets the program learn what time it is and wait precisely (ch. 3).
3. **Interrupts** let events announce themselves instead of being polled (ch. 4).
4. **A cyclic executive** divides time into fixed slots so many jobs share one CPU predictably (ch. 5).
5. **A preemptive scheduler** lets an urgent job interrupt a long one without a hand-built table (ch. 6–7).
6. **Analysis** tells you whether the result meets deadlines (ch. 8–11).
7. **Synchronization** protects data the jobs share, and **protocols** stop the protection from defeating the priorities (ch. 12–13).

Check yourself

CHECK

1. State, for any component you own, its trigger, period or minimum inter-arrival, deadline, budget and miss policy. Which did you have to guess?
2. Why is “the average response time is 2 ms” not an answer to “does it meet its deadline?”
3. Which of Liu and Layland’s five assumptions does a queue between two components violate, and what does that mean for analysis?

Lab link. Lab H3 (scheduler workbench) lets you test the assumptions by breaking them.

The simplest program: a loop

Almost every embedded program starts as a loop that reads, computes and writes. It has no scheduler and no clock. Understanding exactly what that loop does and does not guarantee explains why everything else in this book exists.

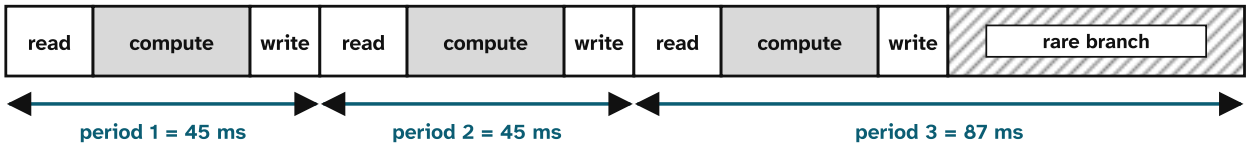
The super-loop

```
int main(void) {
    init_hardware();
    for (;;) {                      /* "super-loop" / "main loop" */
        read_sensors();
        compute();
        write_outputs();
    }
}
```

This is correct, tiny, easy to reason about, and used in production systems of every size. The loop's **cycle** is one pass. Everything about its timing follows from one fact:

> **The period of the loop is the sum of the time of everything in it.**

One program, no scheduler: the period is whatever the code takes.



A deadline of 'every 45 ms' is met twice and missed once. Nothing in the code says so.

Fixes seen later: a hardware timer sets the period (ch. 3), work is split into slots (ch. 5), or a scheduler preempts (ch. 6–7).

Scale: 4 px = 1 ms.

Figure 3. A loop's period is whatever the code takes. The third iteration takes a rare branch and is almost twice as long.

What the loop cannot do

Wish	Why the loop cannot promise it
“Run every 10 ms”	nothing measures 10 ms; the period is the sum of the code’s durations, and it changes whenever the code or the data changes
“React within 1 ms to a button”	the button is checked once per cycle: the reaction time is *up to a full period*
“Do two things at once”	there is one flow of control; a long step delays every step after it
“Add a feature without disturbing the rest”	the new step lengthens the period for *all* steps

A loop that does nothing slow is predictable. One slow step, taken rarely, is the usual way a loop fails after launch. The same shape appears in the Pathfinder lead’s account of what *testing* missed: the rare, heavily-loaded path.

Waiting: busy-wait versus a time check

The first fix people reach for is a delay.

```
for (;;) {
    do_work();                      /* takes 2 ms, sometimes 5 ms */
    delay_ms(10);                   /* wait 10 ms */
}
```

The period is now **work time + 10 ms**, not 10 ms. The error is not random: it is the work time, every cycle, so it **accumulates**. RAN Lab H1 on this machine: a 2 ms job followed by a “10 ms” relative sleep, 300 iterations, ended 1,239 ms late (4,239 ms against an ideal 3,000 ms). Sleeping until an *absolute* next instant ended 0.2 ms late.

```
next = now();
for (;;) {
    do_work();
    next += PERIOD;                 /* the schedule advances by the period, not by "now
+ period" */
    wait_until(next);               /* absolute instant: lateness does not accumulate
*/
}
```

> RULE **Rule.** Periodic work waits for *the next scheduled instant*, never for *a duration after it finished*. Every RTOS has both calls; the relative one is for “wait a bit”, the absolute one is for periods (ch. 3, ch. 6).

A busy-wait (spinning on the clock) is exact but wastes the CPU and blocks everything else. Sleeping lets the CPU idle or do other work, but a general-purpose OS wakes you *late*: on the Mac used for Lab H1 the median wake-up was 1.6 ms late. A microcontroller with a hardware timer wakes you within microseconds. That is the first reason to run on a real-time system.

Polling and the shape of events

A loop that checks inputs is **polling**. It is simple, but its worst-case reaction time equals the loop period, and its cost is paid even when nothing happens. Interrupts (ch. 4) invert this: the event announces itself.

A loop can still react well if every step is **short and never blocks**. The usual style is a **state machine per activity** that advances one small step each pass and always returns:

```
switch (state) {
case WAIT_START:  if (button_pressed()) { t0 = now(); state = DEBOUNCE; } break;
case DEBOUNCE:    if (elapsed(t0) >= 20) state = (button_pressed() ? ARMED :
WAIT_START); break;
case ARMED:       start_motor(); state = RUN; break;
/* ... */
}
```

This is **cooperative multitasking** by hand: each state machine is a “task” that yields by returning. It is the cheapest concurrency there is, and the root of the “run-to-completion” idea in chapter 6. Its single rule is also its weakness: **one step that takes too long, or blocks, stalls everything.**

Foreground and background

Add interrupts and the loop becomes the **background**; interrupt handlers are the **foreground**. The foreground reacts fast; the background does the long work. Data passes between them through variables the foreground writes and the background reads. That is already shared state, and the first place real bugs live (ch. 4, ch. 12).

Where the loop is the right answer

A super-loop is reasonable when all of these hold: few activities, similar rates, hard deadlines easy to meet with large margin, no long steps, and a reviewer who can read the whole program in one sitting. It stops being reasonable when a new feature adds a long or blocking step, or when two activities run at very different rates (a 1 kHz control loop and a once-per-second log line cannot share one pass without waste).

TRAP **Trap: the invisible rare branch.** The loop in the figure meets "every 45 ms" twice and misses once. Nothing in the source says so; review sees three short steps. Rare branches (error paths, calibration, flash writes, a long print) are exactly where periods are lost. Ask of every loop: *what is the longest single thing it can do?*

Try it

TRY IT Run Lab H1: `./h1_loop sleep_rel 10000 2000 300` and `./h1_loop sleep_abs 10000 2000 300` in `labs/host/`. Then add noise: `./h1_loop sleep_abs 10000 2000 300 6000` (every 17th iteration takes 6 ms longer). Predict first: which of the two versions loses time on the noisy iteration, and which only loses *that one* iteration?

Check yourself

CHECK

1. A loop calls `delay(100)` after 12 ms of work. What is its period? After 1,000 iterations, how far behind an ideal 100 ms schedule is it?
2. A button is polled once per loop pass of 20 ms. What is the worst-case reaction time? What is the average?
3. Name one step in a loop you know whose duration depends on data, and say what it does to every other step.

PART 1 · TIME · CHAPTER 3

Measuring time

Before you can meet a deadline you must be able to read a clock, and trust it. Clocks have a resolution, an accuracy, a width, and a type. Four of the incidents in this book are clock bugs.

The stack of clocks

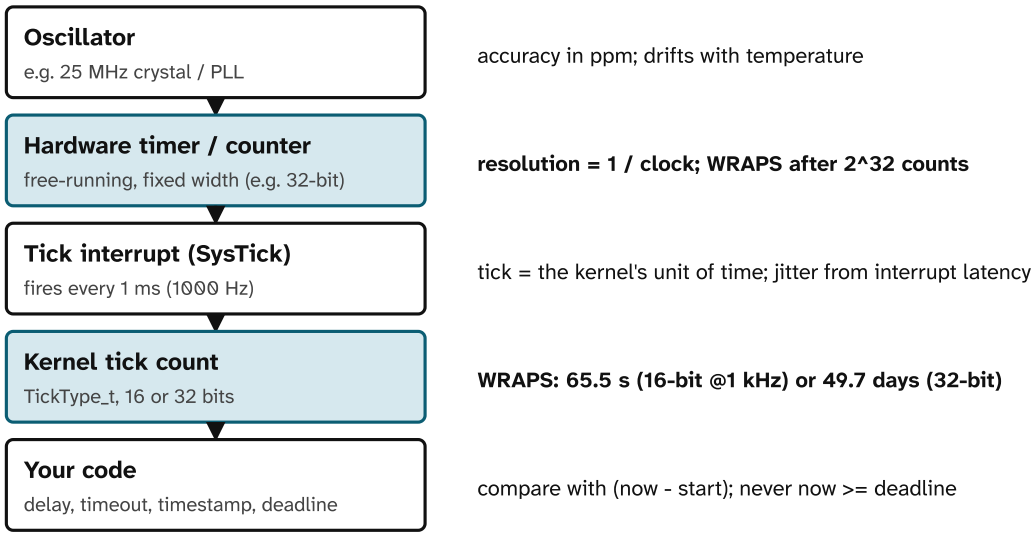


Figure 4. From oscillator to your code. Two layers wrap around; two layers add error.

Layer	Question to ask	Number to know
Oscillator	how accurate is it?	**ppm.** A 50 ppm crystal gains or loses 4.3 s per day ($50 \times 10^{-6} \times 86{,}400$)
Hardware counter	how fast does it count, how wide is it?	at 25 MHz a 32-bit counter wraps in 171.8 s ($2^{32} / 25 \times 10^{6}$); a 1 MHz microsecond counter in 71.6 minutes
Tick interrupt	what is one tick?	1 kHz → 1 ms. Tick *resolution* is 1 tick; a delay is accurate to ±1 tick
Kernel tick count	how wide is the type?	32-bit at 1 kHz wraps in 49.7 days; 16-bit in 65.5 s
Your code	how do you compare two times?	subtract first, then compare (below)

Resolution is the smallest step a clock can show. **Accuracy** is how close it stays to true time. A clock can have microsecond resolution and be wrong by seconds per day.

Every counter wraps: compare differences, not values

An unsigned counter that reaches its maximum goes to zero. Code that compares two *values* breaks at the wrap; code that compares a *difference* does not, provided the interval is less than half the range.

```
if (now >= deadline) { ... }                /* WRONG across the wrap */
if ((int32_t)(now - deadline) >= 0) { ... }  /* right, for intervals < 2^31 ticks */
```

RAN Lab R7 builds the kernel with a **16-bit tick** so it wraps after 65.5 s, jumps the clock to 6 ticks before the wrap, and asks for a 100-tick timeout. The naive comparison reported "expired" immediately (0 loop iterations, 2 µs). The safe comparison ran the full 100 ticks (99,651 µs). The kernel's own delays were unaffected: a periodic task woke every 20 ms through the wrap (ticks 65,521 → 5 → 25 …).

CASE FILE **Boeing 787, 2015: the 248-day counter.** A US airworthiness directive (mirrored by EASA) states that a 787 "powered continuously for 248 days can lose all alternating current (AC) electrical power due to the generator control units (GCUs) simultaneously going into failsafe mode", "caused by a software counter internal to the GCUs that will overflow after 248 days of continuous power." If the four main units were powered up together they would all fail together. The directive does not give the counter's width or unit. The number is *consistent with* a signed 32-bit count of 10 ms ticks: $2^{31} \times 0.01$ s = 248.55 days. The lesson is independent of the details: **every counter wraps, and identical units powered together wrap together.**

A clock that is almost right gets worse every hour

CASE FILE **Patriot, Dhahran, 1991: 0.3433 seconds.** A battery that had run for over 100 hours failed to track an incoming missile. The US Government Accountability Office found that the system kept time as an integer count of tenths of a second and converted it to seconds by multiplying by 1/10 in a 24-bit register. 1/10 has no exact binary form; the stored value was chopped. The error per tick is tiny, the number of ticks grows with uptime, so the clock error grows **linearly**: 0.0034 s after 1 hour, 0.0687 s after 20, 0.1648 s after 48, **0.3433 s after 100** — a range-gate shift GAO tabulates as 687 m.

RAN Lab H1c reproduces GAO's table to four decimals by chopping 1/10 to 23 binary places (the 24-bit register's first fractional bit is zero for 0.1, an explanation that is ours; GAO says only "24 bits"). The question to take from it is general: **what is each time value multiplied by, and how long can the system stay up?**

Wall clock versus monotonic clock

Clock	Can jump?	Use for
Wall / real-time (calendar time)	yes: NTP steps, leap seconds, manual set	timestamps for humans and logs
Monotonic	never goes backwards; may not count while suspended	durations, timeouts, periods, deadlines

CASE FILE **Cloudflare, 1 January 2017.** A leap second made a duration computed from two wall-clock readings **negative**. A component had assumed "the difference between two times would always be, at worst, zero"; the negative value reached weighted-selection code that panicked. At peak about 0.2% of DNS queries were affected. (Cloudflare's own post-mortem.) Not an embedded system, but the bug is a clock-type bug, and embedded systems that mix a calendar time from a radio or GPS with a tick count can do the same.

Delays in an RTOS: three behaviours to know

Call (FreeRTOS names)	What it means	Use it for
`vTaskDelay(n)`	block for *at least n−1 ticks plus the rest of the current tick* after this call	"wait a bit"
`xTaskDelayUntil(&last, n)` (`vTaskDelayUntil`)	wake at `last + n`, then advance `last`	**periods**
software timer	run a callback at an instant	periodic work that is short and does not block

RAN Lab R1, 1 kHz tick, 3 ms of work per cycle:

Method	Mean period	Drift after 99 iterations
work + `vTaskDelay(10 ms)`	12,999 µs	**+296,934 µs**
work + `vTaskDelayUntil(10 ms)`	9,998 µs	**−153 µs**
`vTaskDelay(1)` called at 40 different phases in a tick	between **24 µs and 999 µs** (one tick = 1,000 µs)	—

The last row is the tick's granularity: a one-tick delay can return almost immediately.

TRAP **Trap: a "next wake" time in the future.** `xTaskDelayUntil(&last, n)` assumes `last` is the previous wake time, not a future one. Initialising `last` to a tick that has not happened yet makes the kernel think the counter wrapped, and the call **does not block at all**. In our own first draft of Lab R3 a task did exactly this, spun at priority 6, and starved every lower-priority task for the whole run. Initialise `last = xTaskGetTickCount()` and delay once for an offset.

Latency, jitter and what to measure

Term	Definition	Where it comes from
Interrupt latency	event → first instruction of the handler	other interrupts, interrupts masked by critical sections
Scheduling latency	task becomes ready → task starts	higher-priority tasks, the kernel's own work, a lock
Response time	release → completion	everything above plus the task's own execution
Jitter	variation of any of the above from job to job	rare slow paths, preemption, interference

You measure them with a **second clock that does not depend on the thing measured**: a hardware timer you read (`now_us()` in the labs), a cycle counter (on Cortex-M, `DWT->CYCCNT`, enabled via `DEMCR.TRCENA` and `DWT_CTRL.CYCCNTENA`; not tested in our emulator), or a GPIO pin you toggle and watch on a logic analyser (zero overhead, because writing a pin takes one instruction). Printing is *not* a measurement: formatting and sending a line takes hundreds of microseconds and can change what you are measuring (Lab R9 measures the cost of just stopping in a debugger: about 1 ms per stop).

> **RULE** **Rule.** Report time with its **clock, resolution and wrap period**. "Latency 450" is not a measurement; "450 µs, 25 MHz timer, 32-bit, wraps at 172 s" is.

Check yourself

CHECK

1. A 32-bit tick at 1 kHz wraps in 49.7 days. A device runs for 60 days. Which of these are affected: a `vTaskDelay`, a hand-written `if (now > t_end)`, a subtraction `now - t_start` with a 5-second interval?
2. A component multiplies a tick count by a constant and adds it to a calendar time once per second. List two ways the sum can drift or jump.
3. Why does `vTaskDelay(1)` sometimes return in microseconds?

Lab links. H1 (loop timing, tick wrap, Patriot arithmetic), R1 (delay behaviour), R7 (tick wrap on a real kernel).

Interrupts: reacting on time

Polling asks “has anything happened?” on a schedule. An interrupt makes the hardware say “it happened” and run your handler now. Interrupts are how a processor meets a deadline measured in microseconds — and the first place a program gets concurrent.

A short history

Smotherman’s survey of the field dates early forms of the mechanism to the mid-1950s: the NBS DYSEAC (1954) with an I/O interrupt, the UNIVAC 1103A (1956, often cited as first), the Lincoln Lab TX-2 and IBM Stretch (1957) with vectored interrupts that jump straight to the right handler. Smotherman is explicit that attribution is contested (for example, the 1953 1103 manual does not describe interrupts), so this book says only “by the mid-1950s”. The Apollo guidance computer, a decade later, shows the mature pattern: **hardware interrupts** for the few things that cannot wait (the digital autopilot, uplink, keyboard), a **timed-task list** (Waitlist) for short code that must run at a given instant, and **priority jobs** for everything else.

What happens on an interrupt

1. The peripheral asserts a request. If that interrupt is enabled and not masked by a higher-priority one, the CPU finishes the current instruction.
2. The hardware saves a small set of registers and jumps through the **vector table** to the handler (the ISR).
3. The ISR runs. A higher-priority interrupt can preempt it.
4. The ISR returns; the hardware restores the registers and the interrupted code resumes — or, if the kernel asked for a task switch, a different task resumes (ch. 6).

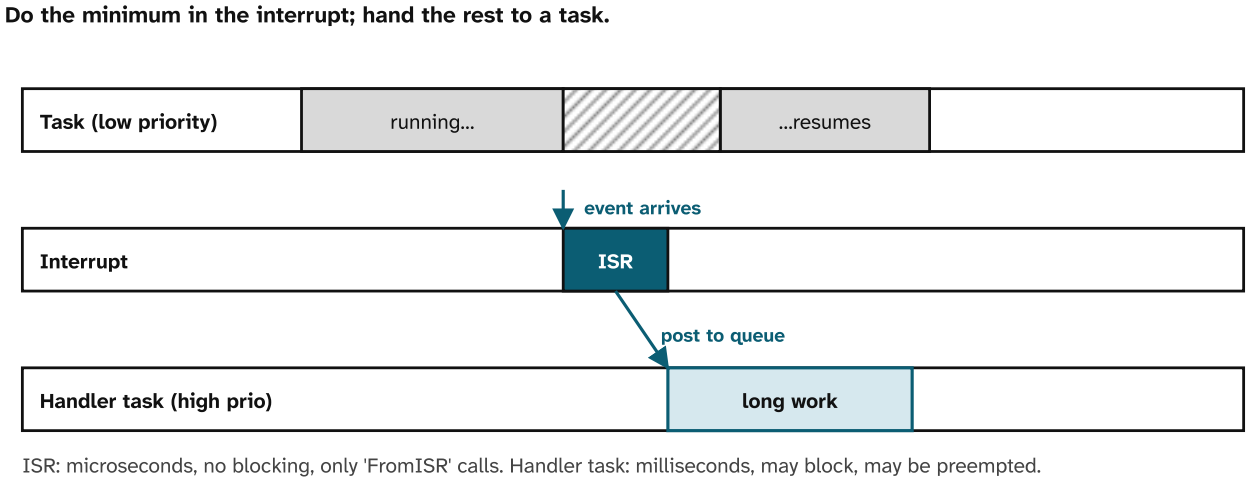


Figure 5. The standard split: the interrupt handler does the minimum and posts an event; a task does the long work.

ISR rules that matter

Rule	Why
Short: microseconds to tens of microseconds	while an ISR runs, equal and lower-priority interrupts and *every task* wait
Never block (no waiting for a lock, queue space, or time)	there is no task to put to sleep; the system would stop
Use only the kernel's "FromISR" functions	normal calls assume they run in a task; the ISR versions never block and report whether a task switch is needed
Handle the interrupt source (clear the flag)	otherwise it fires again immediately
Touch shared data with care	the ISR can run between any two instructions of the code it interrupted
Do not print, allocate or take long locks	each is long, variable or unsafe

On Cortex-M the kernel only masks interrupts **at or below** a configured priority (`configMAX_SYSCALL_INTERRUPT_PRIORITY`). An interrupt more urgent than that line is never delayed by the kernel but may **not** call kernel functions. A numerically larger priority value is *less* urgent — a frequent source of confusion.

The deferred-work pattern

RAN Lab R4 uses the pattern end to end: a timer interrupt every millisecond stamps a message with the hardware time and posts it with `xQueueSendFromISR`; a task consumes it. The ISR's whole job is 5 lines:

```
static void producer_isr(void) {
    BaseType_t woken = pdFALSE;
    msg_t m = { seq++, now_us() };                    /* stamp at the source */
    if (xQueueSendToBackFromISR(q, &m, &woken) != pdTRUE)
        dropped++;                                    /* cannot block: drop and
COUNT it */
    portYIELD_FROM_ISR(woken);                        /* switch now if the consumer
outranks the interrupted task */
}
```

The two lines to remember: **stamp the data where it is born** (the ISR), and **count what you cannot deliver**. Chapters 15 and 17 return to both.

Interrupts are part of the schedule

Interrupt handlers run at a priority above every task, so their execution time delays *every* task. An analysis that lists tasks and forgets the interrupts is optimistic by exactly their interference.

RAN Lab R8: three periodic tasks (C/T = 1/4, 2/6, 3/12 ms) plus a timer interrupt every 2 ms costing 250 µs (12.5% of the CPU).

Task	Deadline	Measured worst response	Analysis without the interrupt	Analysis with it as a top-priority task
t1	4,000 µs	1,260	1,000	1,250
t2	6,000 µs	3,534	3,000	3,500
t3	12,000 µs	11,619	10,000	11,500

Leaving the interrupt out understates t3 by 14%. Including it matches the measurement to 1%. (Method: chapter 9.)

> **RULE** **Rule.** In any timing budget, list every interrupt with its rate and worst-case handler time, and charge it to every task below it.

Masking interrupts is not free

To protect a short sequence from an interrupt, code can **disable interrupts** (a critical section). While interrupts are off, nothing preempts the code — and nothing else runs, including the interrupt you are waiting for. A critical section adds its length to the worst-case latency of every interrupt. Two consequences:

- **Keep critical sections to a handful of instructions.** Copying a 16-byte struct is fine; formatting a log line is not.

- **A masked interrupt that fires twice is delivered once.** The hardware remembers *that* it fired, not *how many times*. A 1 kHz interrupt delayed by 2.5 ms delivers one handler call, not three: samples are silently lost. Lab R10 counts these as `isr_late`.

TRAP **Trap: `volatile` is not synchronization.** A flag set by an ISR and polled by a task must be `volatile` so the compiler re-reads it. That is all `volatile` does. It does not make a read-modify-write atomic, does not order writes to *other* variables, and has been miscompiled: Eide and Regehr tested thirteen production C compilers and found volatile-access bugs in each (EMSOFT 2008; a simple source rewrite worked around 96% of the failing cases; 2008-era compilers). Chapter 12 gives the real tools.

Where the debugger and the interrupts disagree

Stopping the CPU at a breakpoint does not stop the world on real hardware: timers keep counting, peripherals keep receiving. Many microcontrollers pause the watchdog while halted, others do not. In the emulator used in this book the virtual clock is frozen during a stop but still moved forward by about 1 ms per stop in our measurement (Lab R9). Consequence: **do not use breakpoints to investigate timing**; use tracing, counters and watchpoints (chapter 20).

Check yourself

CHECK

1. A 1 kHz interrupt takes 80 µs. A critical section elsewhere disables interrupts for 1.6 ms. What happens to the interrupt, and what should the code count to notice?
2. Why can an ISR not call `xSemaphoreTake` with a timeout, and what does it call instead of `xSemaphoreGive`?
3. What does “numerically higher priority value” mean on your target? Check before you assign an interrupt priority.

Lab links. R4 (ISR to queue), R8 (interrupts in the analysis).

PART 1 · TIME · CHAPTER 5

Many jobs, one CPU: the cyclic executive

Before preemptive schedulers, engineers shared one CPU by cutting time into fixed slots and deciding, on paper, what runs in each. The design is still alive inside modern frameworks, and it teaches the vocabulary of rates, frames and hyperperiods that every later chapter uses.

Frames and cycles

A **cyclic executive** (also *timeline scheduling*, *table-driven*) works like this. Time is divided into equal **minor cycles** (frames). A hardware timer starts each frame. In each frame the executive calls a fixed list of functions in a fixed order. The whole pattern repeats every **major cycle**, which equals the **hyperperiod**: the least common multiple of all the periods.

Buttazzo's description: "the time line is divided into slots of fixed length (minor cycle) and tasks are statically allocated in each slot based on their rates and execution requirements. The schedule is then constructed up to the least common multiple of all the periods (called the hyperperiod or the major cycle) and stored in a table."

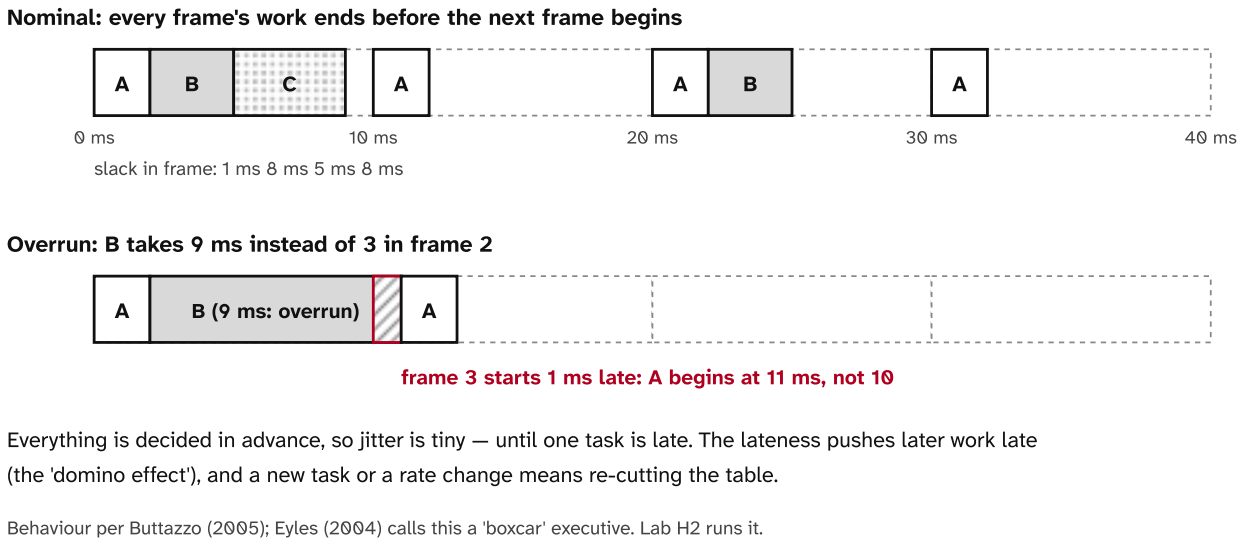


Figure 6. A 40 ms major cycle of four 10 ms frames (top). One 6 ms overrun makes the next frame start late (bottom). Lab H2 prints this table.

```
for (;;) {
    wait_for_frame_timer();           /* hardware timer: every 10 ms */
    task_A();                         /* every frame */
    if (frame % 2 == 0) task_B();     /* every 20 ms */
    if (frame % 4 == 0) task_C();     /* every 40 ms */
    frame++;
}
```

Design arithmetic

Step	Rule	Example (rates 100, 50, 25 Hz)
Hyperperiod	LCM of the periods	LCM(10, 20, 40) = 40 ms
Frame	divides the hyperperiod; every period is a whole number of frames; at least as long as the longest job (if jobs are not split)	10 ms
Slack per frame	frame − Σ budgets of the jobs in it	frame 0: 10 − (2+3+4) = 1 ms
Table size	hyperperiod / frame frames	4 frames

Non-harmonic rates inflate the hyperperiod: periods of 10, 20, 40 and 50 ms give 200 ms; 7, 11 and 13 ms give 1,001 ms. A long hyperperiod means a long table and a long time before a rare alignment is seen in testing. **Choose harmonic rates when you can** (chapter 17).

Why it was popular, and what it costs

Buttazzo summarises both sides.

For	Against
“straightforward to implement and does not introduce significant runtime overhead (since scheduling decisions are taken off-line)”	“fragile during overload situations, since a task exceeding its predicted execution time could generate (if not aborted) a domino effect on the subsequent tasks, causing their execution to exceed the minor cycle boundary”
“tasks always execute in their preallocated slots, so the experienced jitter is very small”	“a creation of a new task, or a little change in a task rate, might modify the values of the minor and major cycles, thus requiring a complete redesign of the scheduling table”

The Apollo software authors had the same view in 1960s words: a “boxcar” executive “is painful to implement because computations must be broken up arbitrarily”, and “it breaks down completely as soon as any function takes longer than the time it is allocated.” Locke’s 1992 comparison of one hard real-time application built both ways concluded that, with rate-monotonic priorities, the fixed-priority design “generally dominates” the cyclic executive (abstract-level source).

RAN Lab H2, 10 ms frames, B overruns by 6 ms in major cycle 2:

Policy	What happens
nominal	0 of 16 frames start late
overrun, no policy	B finishes at 91 ms, C at 95 ms; frames 9 and 11 start late (2 of 16)
overrun, **skip what no longer fits**	C is skipped in frame 8; frame 9 still starts 1 ms late

The choice of policy — continue late, skip, abort — is a **requirement** the table does not contain.

Its modern descendants

The idea survived because the **rate group** is a natural unit of design: “everything that runs at 10 Hz”.

- **F´ (JPL):** `Svc::ActiveRateGroup` “is an active component that drives a set of components connected to `Svc::Sched` output ports. It contains an asynchronous input `Svc::Cycle` port. This port sends a message which wakes the component task. The task invokes each output port in order … It tracks execution time and detects overruns.” Its requirement ARG-004 is a warning event “when a rate group cycle is started before previous is completed”. A rate group is a **cyclic executive running on a thread**, with overrun detection added.
- **cFS (NASA)** and other flight frameworks drive applications from a **scheduler table** that sends wake-up messages in numbered slots of a frame. The precise table layout is configuration-specific; the question to ask is the same.
- **AUTOSAR/OSEK** have *schedule tables* and alarms that activate tasks at fixed offsets.

When you meet “rate group”, “minor frame”, “slot”, “cycle” or “tick table” in a framework, translate: *which frame am I in, what else is in it, and what is its slack?*

When a cyclic executive is the right tool

It is a good choice when the task set is small and fixed, jitter must be minimal, certification wants a statically analysable schedule, and every job’s execution time is tightly bounded. It is a poor choice when jobs are long and variable, rates are unrelated, or the system will grow. Its great virtue for a reader of someone else’s code: **the table is the documentation**.

> **TRAP** **Trap: a frame with no slack.** A frame that is 100% budgeted can only be correct if every job takes *exactly* its budget. Budgets are claims; measured values include interrupts and kernel overhead. Chapter 9’s lab shows a “schedulable” set missing 80% of deadlines because its jobs finished on the deadline on paper.

Check yourself

CHECK

1. Rates 100 Hz, 40 Hz and 25 Hz: the hyperperiod, a frame length that works, and the number of frames in the table.
2. A job in a 5 ms frame occasionally takes 7 ms. Describe three policies and the consequence of each for the *next* frame.
3. In your framework, find the object that decides what runs at each rate. Does it detect overruns? What does it do then?

Lab link. H2 (cyclic executive with overrun and skip policies).

PART 2 · ORDER · CHAPTER 6

Tasks and the kernel

A kernel makes one CPU look like many. It does that with a small trick: keep a separate stack and a few saved registers for each activity, and switch between them fast enough, and at the right moments, that each looks continuous. Knowing the trick tells you what a task costs, what it can never do, and how it differs from an event handler.

Words that mean nearly the same thing

Term	What it adds	Where you meet it
Process	its own address space (memory protection)	Linux, ROS 2 nodes in separate processes
Thread	an execution context sharing a process's memory	Linux, Zephyr (*thread*), POSIX
Task	the same, in small-RTOS vocabulary; usually no memory protection	FreeRTOS, OSEK/AUTOSAR
Job	one release of a task: one pass through its work	scheduling theory
Component / app / node	a unit of design that may own one or more tasks, queues and state	F´ (component), cFS (app), ROS 2 (node)

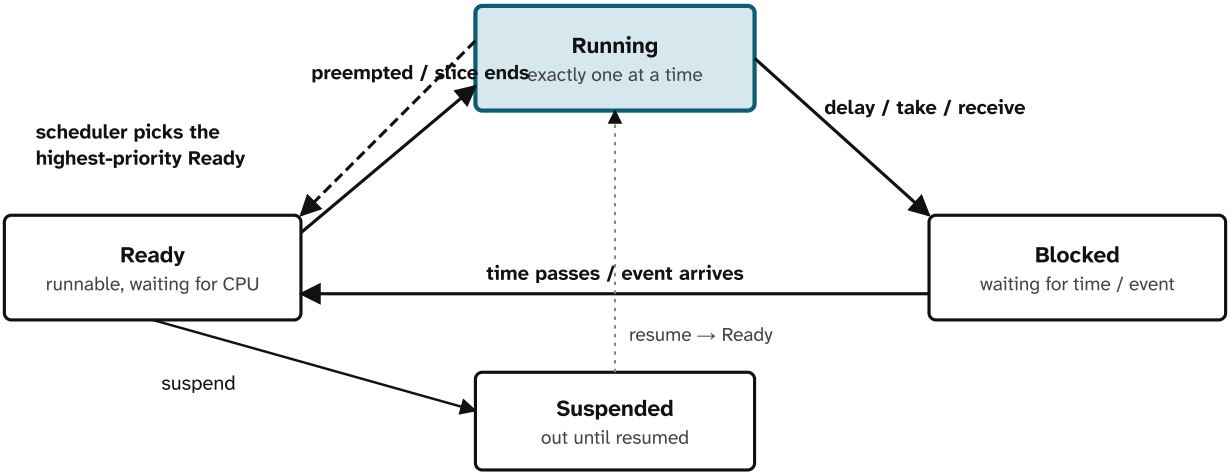
This book says *task* for the unit the scheduler sees and *job* for one release of it.

Anatomy of a task

A task is four things: a **function** that never returns (or deletes itself), a **stack** (its local variables and saved context), a **control block** (state, priority, the saved stack pointer), and the **kernel lists** it moves between.

```
void vSensorTask(void *arg) {
    TickType_t last = xTaskGetTickCount();
    for (;;) {                                  /* a task body is a loop that BLOCKS
somewhere */
        read_and_filter();
        xTaskDelayUntil(&last, pdMS_TO_TICKS(10));
    }
}
xTaskCreate(vSensorTask, "sensor", 256 /* WORDS, not bytes */, NULL, 3 /* priority */,
NULL);
```

The stack size is in **words** (4 bytes on Cortex-M). The FreeRTOS book is explicit: "the value specifies the number of words the stack can hold, not the number of bytes." RAN In Lab R6 the manager task, created with 256 words, had 145 words free at its worst point; the idle task, 118.

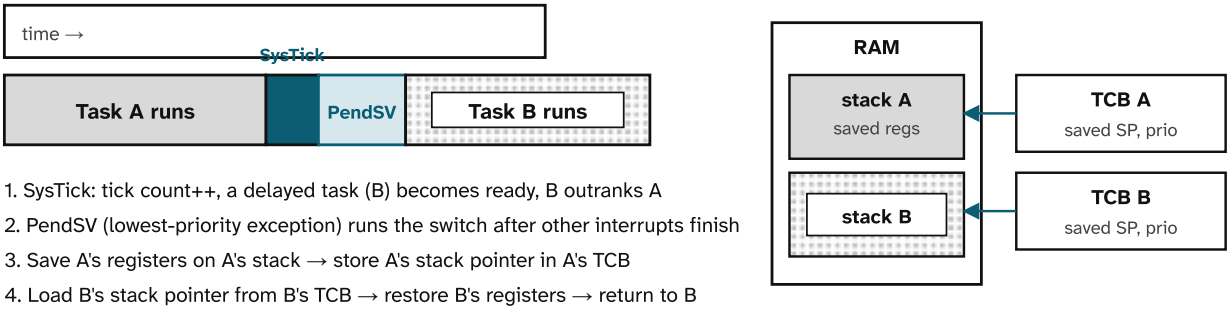


Only the highest-priority Ready task runs. A task that never blocks never lets lower-priority tasks run.

Figure 7. The four states. Only Running consumes CPU. A task that never blocks never lets a lower-priority task run.

How one CPU runs many tasks

One CPU, many stacks: the kernel swaps which stack the CPU is using.



1. SysTick: tick count++, a delayed task (B) becomes ready, B outranks A
2. PendSV (lowest-priority exception) runs the switch after other interrupts finish
3. Save A's registers on A's stack → store A's stack pointer in A's TCB
4. Load B's stack pointer from B's TCB → restore B's registers → return to B

Cost: tens of microseconds of CPU on an MCU, every switch. Frequent preemption is not free (R2 measured 4–24 µs per wake-up).

Model: FreeRTOS on Cortex-M (SysTick + PendSV); see Memfault 'RTOS context switching on Cortex-M'.

Figure 8. The kernel stores the running task's registers on its stack, saves the stack pointer in the task's control block, loads the other task's stack pointer, and returns "into" it.

On a Cortex-M the *tick interrupt* (SysTick) counts time and may make a task ready; a low-priority exception (PendSV) then performs the switch after all other interrupts have finished (Memfault's walk-through of the same mechanism is in `research/`). A switch costs tens of microseconds of CPU on a microcontroller. RAN In Lab R2, a zero-length periodic job measured **4 to 24 µs** per wake-up in steady state, and the *first* job of each task cost about 230 µs more because it includes task creation.

Nothing here is "at the same time". Two tasks printing every 100 ms in Lab R0 look simultaneous: both lines carry the same millisecond. In microseconds they are 39 µs apart (A at 4,331 µs, B at 4,370 µs), because A ran first and then blocked.

Cooperative, preemptive, and run-to-completion

Style	Who decides when to switch	Strength	Weakness
Cooperative	the task, when it blocks or yields	no preemption races; tiny kernel	one long step delays everyone (Zephyr: a cooperative thread "remains the current thread until it performs an action that makes it unready")
Preemptive	the kernel, whenever a higher-priority task is ready	short response for urgent work; analysable	any code can be interrupted between any two instructions: shared data needs protection (ch. 12)
Run-to-completion / event-driven	the handler, by returning	no stacks per activity; deterministic	a handler must never block; control flow becomes a state machine

The FreeRTOS book calls "fixed priority preemptive scheduling with time slicing" the algorithm "used by most small RTOS applications"; the same kernel can be configured cooperative (`configUSE_PREEMPTION` 0).

Stackless alternatives. Dunkels et al. (SenSys 2006) built *protothreads*: blocking-style code that compiles to a state machine, with a memory overhead of "only two bytes per protothread" and, in their evaluation, "the number of lines of code … reduced by one third" with execution overhead "on the order of a few processor cycles". The price: no local variables survive a block. **Active objects** (Samek; a vendor's viewpoint) give each object a private queue and thread, process events to completion, and block in one place only. Lee's critique of threads ("wildly nondeterministic … the job of the programmer becomes one of pruning that nondeterminism") is why many frameworks hide threads behind queues: you will see this in F´ (active components) and ROS 2 (callbacks).

What a kernel adds to your design

Kernel object	Use	Cost to remember
Task	an activity with its own wait points	one stack (permanent RAM) + control block
Software timer	a short callback at an instant	runs in the *timer service task*, at that task's priority
Queue / notification / event group	hand data or a signal between tasks and interrupts	RAM for items; behaviour when full (ch. 15)
Mutex / semaphore	protect data / signal	blocking time and inversion (ch. 12–13)
Idle task	runs when nothing else can; reclaims memory of deleted tasks	if starved, deleted tasks leak

Tasks in other frameworks

Concept	FreeRTOS	Zephyr	F´	cFS	ROS 2
unit the scheduler sees	task	thread	**active** component (queue + thread)	app's task	executor thread running callbacks
runs without its own thread	—	workqueue item	**passive** component (called synchronously)	library call	callback inside the executor
how it waits	block on kernel object	block / poll	dequeue next message	pend on a software-bus pipe	wait set / executor spin

Passive, queued and active components are F´'s three kinds ("no queue and no thread", "a queue but no thread", "both"). When you read an unfamiliar system, *count the threads* first: it is the quickest map of where preemption can happen.

> **TRAP** **Trap: a task that never blocks.** A `for (;;)` that polls without waiting runs forever at its priority; every lower-priority task, and the idle task, starves. In Lab R3's first draft a task initialised its wake time to a *future* tick (chapter 3's trap), never blocked, and starved the whole system at priority 6.

> **TRAP** **Trap: deleting tasks and the idle task.** A task that deletes itself leaves its stack and control block for the idle task to free. If something at priority 0 (or the idle task itself) is starved, memory leaks. The FreeRTOS book warns: "it is essential that the Idle task is not starved of processing time."

Check yourself

CHECK

1. List the threads in a system you work on. For each, what does it block on?
2. A task has a 128-word stack and calls a function with a 600-byte local array. What happens, and when will you find out? (Chapter 14.)
3. Why can a cooperative scheduler be easier to reason about than a preemptive one, and what is the price?

Lab links. R0 (two tasks), R1 (delays), R2 (measured overhead).

Priorities and preemption

With a fixed-priority preemptive scheduler the rule is one line: always run the highest-priority ready task. Every interesting behaviour — starvation, inversion, jitter, missed deadlines — is a consequence of that line plus the edge cases around it.

The rule and a picture

At any instant the CPU runs the **highest-priority Ready task**. If a higher-priority task becomes ready, it preempts the running task *immediately*, not at the next tick: the FreeRTOS book is explicit that "task preemption can occur at any time, not just in the RTOS tick interrupt." Tasks of **equal** priority share the CPU in turn (round robin, one tick per slice); the book adds that this "does not guarantee time is shared equally between tasks of equal priority, only that Ready state tasks of equal priority enter the Running state in turn."

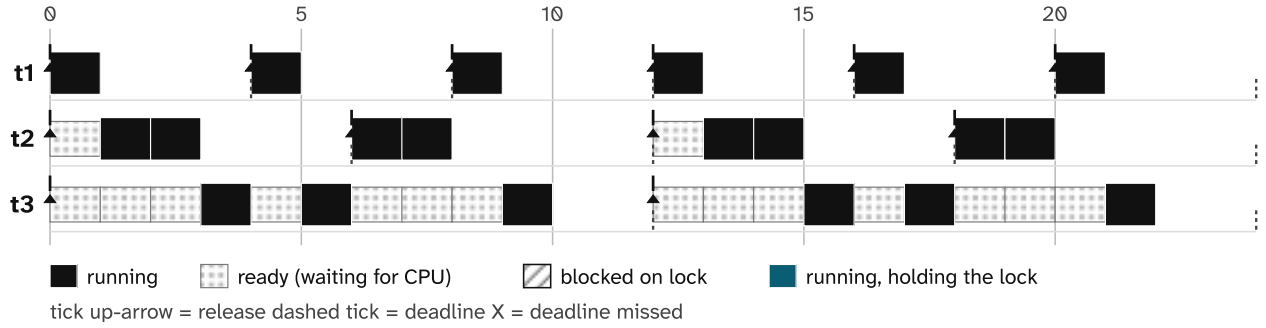


Figure 9. Three periodic tasks scheduled rate-monotonically: t1 (1 ms every 4), t2 (2 ms every 6), t3 (3 ms every 12). Dark = running; dotted = ready but not running. t3 is preempted five times and still finishes at 10 ms.

Read the figure the way you will read a trace: for each row, find the *release* (arrow), the *start* (first dark), the *finish*, and the *dotted* region between. The dotted time of a task is the interference it suffers; response time is release to finish.

Choosing priorities

Assign by	Rule	When to use
Rate (rate-monotonic)	shorter period → higher priority	periodic tasks with deadline = period; provably optimal among fixed-priority assignments for that model (Liu & Layland)
Deadline (deadline-monotonic)	shorter relative deadline → higher priority	deadline shorter than period
Importance	most critical → highest	when deadlines are loose and overload handling matters (ch. 11)

The common mistake is to rank by *importance*: “the safety task is highest”. Priority is a **scheduling** tool. A rarely-run, critical, slow task at the top can make a fast, periodic task miss its deadline while the critical task hardly ever runs. Rank by timing first; handle criticality with budgets, isolation and monitoring.

> **CASE FILE** **Mars Pathfinder’s priority list (1997).** From the flight-software lead’s own account: `bc_sched` (sets up each 1553 bus cycle) was “the highest priority task in the system (except for the vxWorks `tExec` task)”; the entry-and-landing task second; `bc_dist` (collects the bus results) third; “all of the tasks which perform other spacecraft functions are lower. Science functions, such as imaging, image compression, and the ASI/MET task are still lower.” The bus cycle ran at 8 Hz (0.125 s). The design ranks tasks by how soon a miss is fatal. It was sound until a lock in a library call let the lowest-priority task block the third-highest (chapter 13).

The edge cases

Situation	What happens	Consequence
A task never blocks	all lower priorities starve, including idle	the system “works” with part of it dead
Two tasks share a priority	time-sliced each tick	response depends on the other task’s length; their *sum* must fit
Equal priority, time slicing off	one runs until it blocks	cooperative behaviour inside one level
A task changes priorities at run time	order changes at once	analysis must cover every value it takes
A lock holder is boosted (inheritance)	its priority is temporarily higher	its priority is no longer its configured one (ch. 13)
The software-timer task runs callbacks	at the timer task’s priority, which is often the **highest**	a long timer callback preempts everything; keep callbacks tiny
An interrupt fires	above every task	interference with all of them (ch. 4)
Idle shares priority 0 with application tasks	they time-slice with idle	idle’s own work (hooks) steals time
Tick granularity	wake-ups happen on tick boundaries	a 1 ms tick gives ±1 ms of release jitter

RAN Lab R8 and R3 show these on a real kernel: a trace of each context switch becomes a timeline you can read (below).

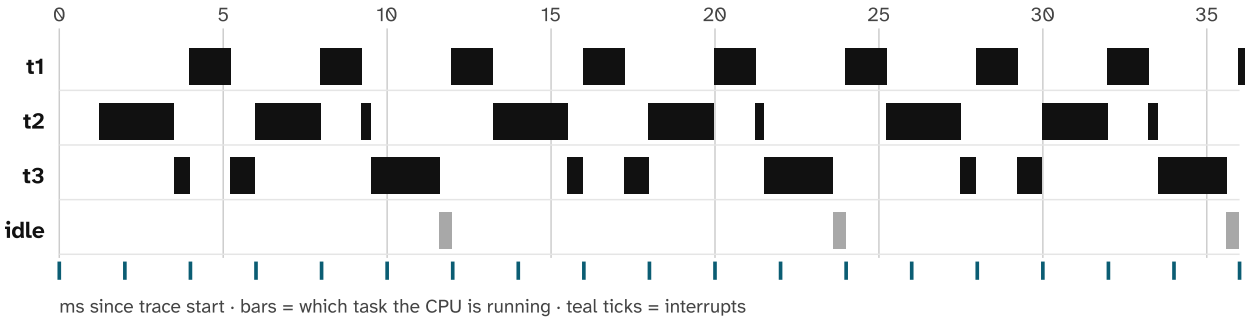


Figure 10. A trace of three periodic tasks and a 2 ms interrupt (teal ticks) on the emulated Cortex-M3, 36 ms. Same rule, real kernel: t1 runs first every 4 ms; t3 gets only the gaps.

Preemption is not free and not harmless

- **Cost.** Each switch is tens of microseconds of CPU, plus cache and pipeline effects on bigger cores.
- **Hazard.** Preemption can happen between any two instructions of lower-priority code. Anything two tasks (or a task and an interrupt) both touch is exposed (chapter 12).
- **Benefit.** The highest-priority work has a short, analysable response time, independent of what lower-priority work is doing — *provided it does not need anything a lower-priority task holds*.

> **RULE** **Rule.** Priorities encode timing relationships. Change one only after re-running the analysis (chapter 9) or at least the lab trace. A priority that "feels right" is a guess.

Check yourself

CHECK

1. A framework gives you 8 priority levels, with 0 for idle and 7 for the timer task. Four of your components run at 100, 50, 20 and 10 Hz. Assign priorities and defend each.
2. Two equal-priority tasks each need 6 ms of work every 10 ms. Is that a problem? What if one of them is a hard-deadline task?
3. Which of your components run code in a timer callback? At what priority?

Lab links. R2, R3, R8 (traces and measurements); H3 (workbench).

Utilization and the rate-monotonic bound

Utilization is the fraction of the CPU a component needs. It is the cheapest schedulability check there is, and Liu and Layland's 1973 result tells you exactly how much it can prove — and how much it cannot.

Utilization

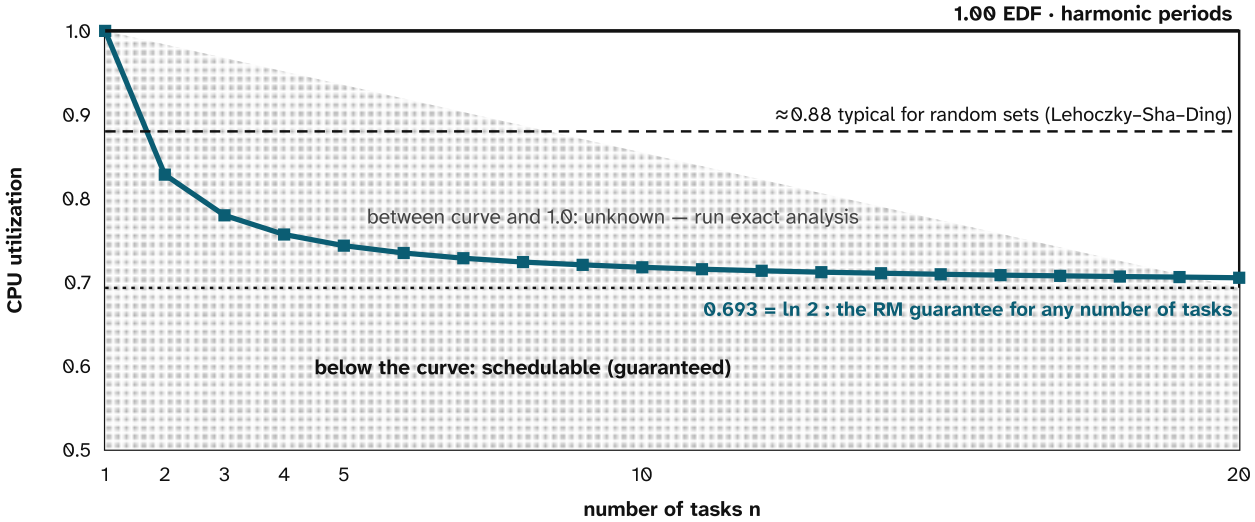
For a periodic task with execution budget C and period T, **utilization** is U = C / T. For a set, **U = Σ C_i / T_i**. Always include interrupts (their rate and handler time) and, if you can estimate it, kernel overhead.

If…	Then…
U > 1	**no** scheduler can meet every deadline: the CPU is overloaded (ch. 11)
U ≤ 1	EDF can schedule it (ch. 10) if deadlines equal periods; fixed priority *might*
U ≤ the bound below	fixed priority with rate-monotonic priorities **will** schedule it

The Liu–Layland bound

For *n* periodic tasks, independent, with deadlines equal to periods and constant run-times, rate-monotonic priorities meet every deadline whenever

U ≤ n(2^(1/n) – 1)



Curve = n(2^(1/n) – 1): 1.000, 0.828, 0.780, 0.757, 0.743 … 0.718 at n = 10, → 0.693. (Liu & Layland 1973, Theorem 5.)

Figure 11. The guarantee falls with the number of tasks and settles at ln 2 ≈ 0.693. Above the curve is not "failing": it is "unknown — run exact analysis".

n	1	2	3	4	5	10	∞
bound	1.000	0.828	0.780	0.757	0.743	0.718	0.693

Three facts to hold at once:

1. **It is sufficient, not necessary.** Above the bound a set may still be schedulable; you need the exact test (chapter 9). RAN Lab H3: C = (1, 2, 3) ms, T = (4, 6, 12) ms has U = 0.833, above 0.780 — and is schedulable (t3 finishes at 10 ms of its 12).
2. **The average case is better.** Lehoczky, Sha and Ding (1989) report that for randomly generated sets with uniformly distributed parameters the *average breakdown utilization* is about 88% (abstract-level source; a later analysis argues the large-set threshold is nearer 0.80, so treat 88% as an observation about a particular random distribution, not a rule).
3. **Harmonic periods reach 1.0.** If every period divides the next longer one (100, 50, 25 Hz), RM can schedule up to U = 1. This is one reason designers pick rates that are multiples of each other (chapter 17).

A utilization check as a first review

```
python3 labs/host/rtcalc.py sense:1:10 filter:3:20 control:4:40 log:10:100
tasks: 4   utilization U = 0.450   hyperperiod = 200
Liu-Layland bound for n=4: 0.757   -> U <= bound: schedulable under RM (sufficient test)
```

RAN The calculator in the labs prints this line, a hyperbolic-bound check (a tighter sufficient test, Bini et al.; not read here), whether the periods are harmonic, and then runs the exact analysis of chapter 9.

> CASE FILE **Toyota's electronic throttle control, as testified (2013).** In litigation over unintended acceleration, a software expert for the plaintiffs, Professor Phil Koopman, presented slides on the throttle controller. Among the claims he reported from NASA's review and trial testimony: the operating system had "multiple priority levels; each with round-robin tasking"; the main CPU had "less than 20% idle time at 5000 RPM ... in other words, more than 80% loaded"; the Liu–Layland bound for four tasks is 75.68% and "would miss deadlines even if RMA had been used." The same slides note that NASA's analysis found "no deadline misses seen" and that worst-case execution time was "difficult due to busy-wait loops, indirect recursion". SEC These are an **expert's** slides in a case Toyota contested; we use them for *the check*, not the verdict: *compare U with the bound, and ask what the interrupts and background load add.* A load above the bound is a reason to run the exact analysis, not proof of a miss.

What utilization does not capture

Missing from U	Why it matters	Where it is handled
deadlines shorter than periods	a task with D < T can miss at lower U	deadline-monotonic priorities and RTA (ch. 9)
blocking on shared resources	a high-priority task waits for a low one	blocking term B (ch. 9, 13)

Missing from U	Why it matters	Where it is handled
release jitter	releases are not exactly periodic	jitter term J (ch. 9)
interrupts and the tick	consume CPU above every task	add them as top-priority tasks (ch. 4, 9)
overhead of context switches	adds to every job	measure it (R2: 4–24 µs per wake-up)
execution time that varies	U uses the *budget*	the budget must be an upper bound (ch. 9, 20)

TRAP **Trap: "we're at 70%, we're fine."** A single-number load says nothing about the *lowest-priority* task's deadline, about bursts, or about blocking. It also hides the opposite error: a set at 95% with harmonic periods and no blocking may be fine. Use utilization to *triage*, then compute.

Check yourself

CHECK

1. Tasks (C, T) in ms: (2, 10), (3, 20), (8, 40), plus a 100 µs interrupt every 1 ms. Compute U with and without the interrupt and compare with the bound for the right *n*.
2. Which tasks in your system have harmonic periods? What would you change to make more of them harmonic?
3. A colleague says the bound is 69%, so 75% load is unsafe. What do you ask them?

Lab link. H3 demo 1 and the `rtcalc.py` workbench.

Deadlines and response-time analysis

Utilization tells you if a set is probably fine. Response-time analysis tells you exactly how long each task can take, in the worst case, and compares it with its deadline. It needs four numbers per task, and it gives a yes or a no.

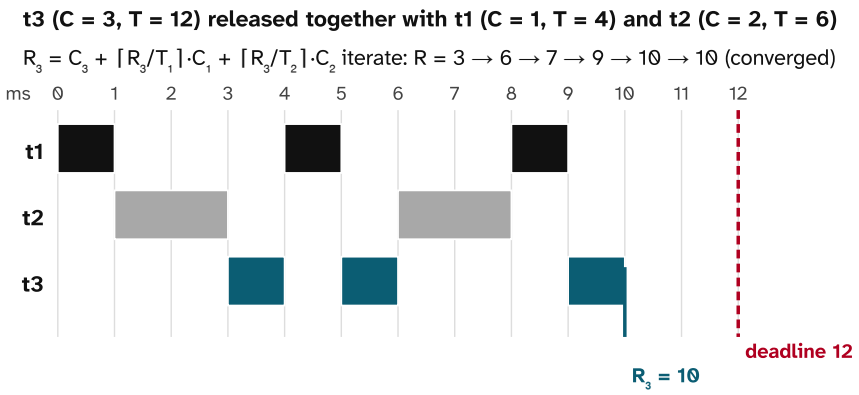
The equation

Audsley, Burns, Richardson, Tindell and Wellings (1993) give the analysis for tasks scheduled by fixed priority, preemptively:

$R_i = C_i + B_i + \Sigma$ over higher-priority j of $\lceil (R_i + J_j) / T_j \rceil \cdot C_j$

Symbol	Meaning
C_i	worst-case execution time (budget) of task *i*
B_i	worst-case **blocking** by lower-priority tasks (chapter 13)
J_j	release jitter of higher-priority task *j*
T_j	period (or minimum inter-arrival) of *j*
$\lceil x \rceil$	round up: the number of releases of *j* that land inside R_i
R_i	worst-case response time; the task is schedulable if $R_i \leq D_i$

You solve it by iteration: start with R = C + B, compute the right-hand side, feed it back, and stop when it stops changing (or exceeds the deadline). The authors note that the method “is not based on the notion of processor utilisation” and gives “sufficient and necessary” tests for the model, and that on an avionics case study its predictions were “in close agreement with the behaviour exhibited during simulation studies.”



Dark: t1. Mid-gray: t2. Teal: t3 — it only runs in the gaps (ms 3, 5, 9). Slack = 12 − 10 = 2 ms.

Utilization here is 0.833 — above the 0.780 bound — yet the exact test proves it schedulable. (Lab H3, demo 1.)

Figure 12. Worst-case response of t3 (C = 3, T = 12) with t1 (1/4) and t2 (2/6) above it: 3 → 6 → 7 → 9 → 10 → 10. t3 only runs in the gaps, at ms 3, 5 and 9.

By hand, in five steps

1. List every task *and every interrupt* with period T and budget C. Interrupts are tasks with the highest priority.
2. Sort by priority (rate-monotonic: shortest period first).
3. For the lowest-priority task, start R = C, compute the sum of interference from every higher-priority task, repeat.
4. Add blocking B (zero if tasks share nothing; chapter 13 explains how to bound it) and jitter if releases are not exact.
5. Compare R with D. Slack = D − R. Record it.

RAN `python3 labs/host/rtcalc.py a:1:4 b:2:6 c:3:12` prints each R, the slack and a Gantt chart. The simulator in `rtsim.py` agrees with the analysis on every set in the labs (for example t3: R = 10 by both).

Predict, then measure: the numbers from a real kernel

RAN Lab R2 runs the set on FreeRTOS on an emulated Cortex-M3 and records every job's response time from the release tick to completion (job 0 excluded: it includes task creation).

Task	C (µs)	T (ms)	Analysis	Measured max	Overhead
t1	1,000	4	1,000	**1,006**	6 µs
t2	2,000	6	3,000	**3,024**	24 µs
t3	3,000	12	10,000	**10,087**	87 µs

The model predicted the measurement within 1%, and the difference is the kernel's own time (ticks and switches). That is the good news. Here is the lesson in the second set.

Task	C (µs)	T (ms)	Analysis (R)	Measured max	Jobs missing the deadline
t1	2,000	5	2,000	2,008	0 of 489
t2	4,000	7	8,000 (> 7: miss)	8,284	**279 of 349 (80%)**

U = 0.971. The zero-overhead simulator predicts only **1 in 5** t2 jobs misses (R = 8 once per hyperperiod). On the real kernel **4 in 5** missed. Per-job responses show why: the simulator's pattern is R = 7, 6, 7, 6, 8 ms, so three of every five jobs finish at *exactly* the deadline. The target's was 7.08, 8.14, 7.17, 6.20, 8.04 ms: the 7 ms jobs finished 80–170 µs late.

> **RULE** **Rule.** A job that finishes at its deadline on paper misses it on target. Add a **margin** for the tick, switches and measurement error to C (or require R ≤ 0.9 D), and treat zero slack as failure.

The four inputs, and how wrong each can be

Input	Source	Danger
T	spec, or the sensor's datasheet	actual rate drifts (sensors have their own clocks) or arrives in bursts; use the *minimum* inter-arrival
D	requirement	often unstated; the default "D = T" may be wrong
C	analysis or measurement	the observed maximum is a **lower bound** on the true worst case
B	lock durations of lower-priority tasks	requires knowing every critical section (ch. 13)

On **C**: Wilhelm et al.'s survey of the worst-case execution time problem is clear that a bound must be "safe, i.e., … never underestimate" and "tight", that "the assumption that only local worst cases have to [be] considered to safely determine upper bounds on global execution times is unsafe", and that one cannot assume you have found "a worst initial execution state, to safely start measurement or analysis." Measurement shows what *did* happen; static analysis tools bound what *can*. On a small core without caches the gap is modest; on a core with caches and speculation it can be large. In this book, a measured maximum is called **observed maximum (high-water mark)**, never WCET, and a budget is **claimed** until someone defends it.

Release jitter, deadlines and priorities

If a task is released late by up to J (for example because a tick-aligned wake-up follows an interrupt), it can be pushed into a collision with its next release: add J to R. If the deadline D is shorter than T, assign priorities by **deadline** rather than period (deadline-monotonic); the analysis is the same.

Why have both utilization and RTA?

	Utilization bound	Response-time analysis
Cost	one sum	an iteration per task
Answer	yes (guaranteed) or “unknown”	yes or no, with the number
Handles blocking, jitter, D < T	no	yes
Tells you *which* task fails and by how much	no	yes

TRAP **Trap: analysing the tasks you wrote.** The set includes every interrupt, the tick, the timer service task, and any task a framework created for you. List them from the *running system* (chapter 22), not from your component’s source.

Check yourself

CHECK

1. Tasks in ms (C, T): (1, 5), (2, 8), (3, 20). Compute R for each by hand. Is the third schedulable? What is its slack?
2. Add a 200 µs interrupt every 1 ms. Recompute.
3. A task’s measured maximum is 4.1 ms over a day. Its budget in the analysis is 4.0 ms. What is wrong, and what do you do?

Lab links. H3 (workbench), R2 (predict then measure), R8 (with interrupts).

EDF and the other policies

Fixed priorities are not the only way to decide who runs. The earliest-deadline-first rule is optimal on one CPU and appears in the Linux kernel and in Zephyr. You may never configure it, but you will read documentation, papers and framework code that assume you know what it promises and what it does not.

The rule

Earliest Deadline First (EDF): at every scheduling decision run the ready job whose **absolute deadline** is nearest. Priorities are not stored; they are *computed* from the deadlines of the jobs currently in the system, so a task's priority changes from job to job.

Liu and Layland proved it optimal for their model: "if a set of tasks can be scheduled by some priority assignment, it can also be scheduled by this method. In other words, the least upper bound on the processor utilization factor is uniformly 100 percent." A set with deadlines equal to periods is schedulable under EDF **iff U ≤ 1**.

RAN Lab H3 demo 2: C = (2, 4) ms, T = (5, 7) ms, U = 0.971. Rate-monotonic misses (t2 finishes at 8 ms, deadline 7); EDF meets every deadline over the hyperperiod.

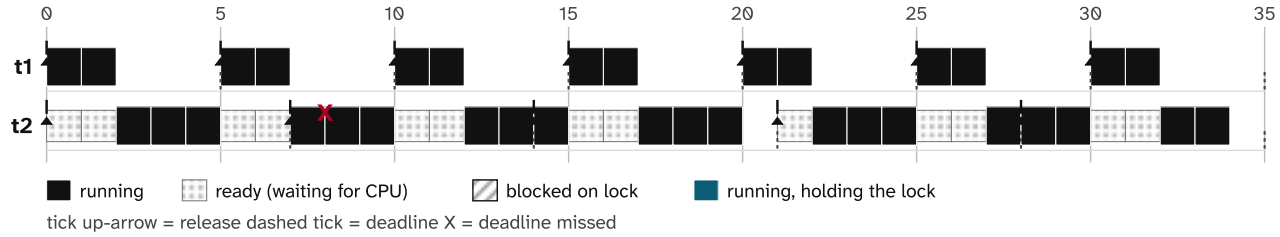


Figure 13. Fixed priority (rate-monotonic): the second task misses once per 35 ms.

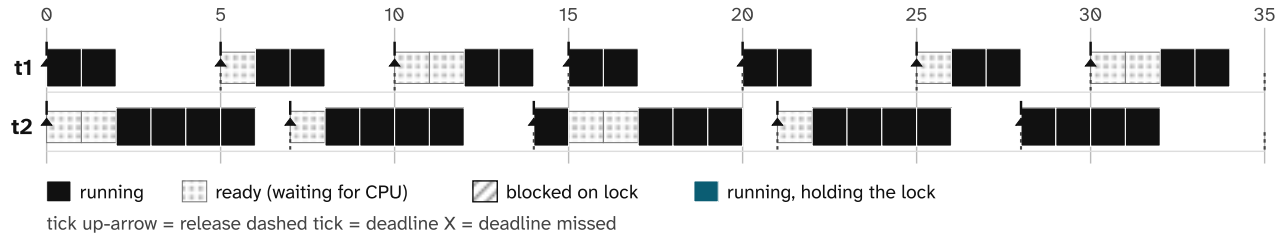


Figure 14. EDF on the same set: no miss. The two tasks swap places depending on whose deadline is nearer.

Where EDF exists in real systems

System	What it provides	Source
Linux	`SCHED_DEADLINE`: "basically an implementation of the Earliest Deadline First (EDF) scheduling algorithm, augmented with a mechanism (called Constant Bandwidth Server, CBS)". Each task declares `runtime`, `deadline` and `period`; the CBS gives "bandwidth isolation" so one task cannot steal another's reserved time	kernel documentation
Zephyr	optional `CONFIG_SCHED_DEADLINE` (off by default): among threads of **equal static priority**, the nearer deadline wins; `k_thread_deadline_set()` takes a cycle-count delta; the scheduler does **not** enforce that the deadline is met	Kconfig and kernel docs
FreeRTOS	none: fixed priority (with time slicing among equals)	FreeRTOS kernel book
OSEK / AUTOSAR OS	fixed priority with priority-ceiling resources	secondary sources

Read the Zephyr entry twice: EDF there is a **tie-breaker inside a priority level**, not a replacement for priorities. Many "EDF" features in embedded systems are like that.

The evidence: EDF versus fixed priority

Buttazzo's "Rate Monotonic vs. EDF: Judgment Day" (2005) opens: "many misconceptions still exist about the properties of these two scheduling methods", including that RM "is easier to analyze than EDF, it introduces less runtime overhead, it is more predictable in overload conditions, and causes less jitter". "Since the above statements are either wrong, or not precise ...". The paper is an advocate's, but it argues with counterexamples you can run.

Folk claim	What the paper shows	What we ran
EDF needs more preemptions and overhead	counting context switches, EDF often preempts *less* than RM (simulation curves, and two-task examples)	— (see the paper's figures)

Folk claim	What the paper shows	What we ran
RM is predictable in overload: the long-period tasks fail first	"this property does not hold under RM (neither under EDF)": with C = (2, 3, 1, 1), T = (5, 9, 20, 30) and a 1.5-unit overrun in the first two jobs of τ1, the task that misses is τ2, **not** τ4	**RAN** H3 demo 3a reproduces it (times doubled): t2 misses
In permanent overload RM and EDF both degrade gracefully	under RM, "a permanent overload may cause a complete blocking of the lower priority tasks"; under EDF the average period of every task scales by U	**RAN** H3 demo 3b, U = 1.25 over 120 ticks: FP completes **15, 10, 0** jobs; EDF completes **12, 8, 4**
What RM *does* guarantee	"an overrun in task τi cannot cause tasks with higher priority to miss their deadlines" (under EDF "any other task could miss")	consistent with the H3 runs

So which should you use?

Almost always you do not choose; the framework did. Choose with these facts:

You want...	Prefer
the simplest analysis, wide tool support, hardware priority levels	**fixed priority** (rate/deadline-monotonic)
to use the last 10–30% of the CPU with unrelated rates	**EDF** or a reservation server (CBS)
isolation between components from different teams	**reservation-based** (budgets enforced), e.g. SCHED_DEADLINE
minimal jitter, certified static table	**cyclic executive** (ch. 5) or **time-triggered** design
overruns that hurt only the culprit	fixed priority (damage flows *down*) or budget enforcement

TRAP **Trap: "EDF is optimal, so it cannot miss."** Optimal means *if any algorithm can schedule the set, EDF can*. If U > 1, nothing can. EDF in overload also has no priority order to fall back on: a late job's deadline is in the past, so it keeps outranking everyone unless the system aborts it (Zephyr's own comment on the API: a deadline in the past "still ranks above others").

Other policies you will see named

Name	One line
Round robin / time slicing	equal-priority tasks take turns each tick; fair-ish, not predictable
Cooperative	switch only when a task blocks or yields
Time-triggered	everything starts at pre-planned instants (cyclic executive across a network)
Reservation / server (CBS, sporadic server)	each component gets a budget per period; excess is delayed
Mixed criticality	different assurance levels share hardware with enforced isolation (named here; not covered)

Check yourself

CHECK

1. A set has U = 0.93 with unrelated periods. RM analysis says t3 misses. Name two different fixes that do not involve a faster CPU.
2. Your framework offers a “deadline” parameter on tasks. Find out: does the scheduler *use* it (ordering), *monitor* it (detect), or *enforce* it (abort)?
3. Why does the cost of a context switch matter more to a comparison of RM and EDF than the cost of choosing the next task?

Lab link. H3 demos 2 and 3 (RM versus EDF; overload).

Overload: when the work does not fit

Every analysis in the last four chapters assumed the budgets hold. Sooner or later one does not: a slow path, a burst, a fault. What your system does then is a design decision, and the best time to make it is before it happens.

Two kinds of overload

Kind	Cause	Example	Shape
Transient	a job runs longer than budget, or events arrive in a burst	a recalibration step, a retry storm	passes; the system must recover its schedule
Permanent	demand exceeds capacity for as long as the condition lasts	a new feature, a faster sensor, a misconfigured peripheral	unfinished work accumulates until something gives

What fixed priority does

Under fixed priority an overrun in task *i* cannot make any **higher**-priority task late; it can make any **lower** one late. In permanent overload the lowest-priority tasks starve completely. RAN Lab H3 demo 3b, three tasks with U = 1.25, 120 ticks: tasks a, b, c (priority order) completed **15, 10 and 0** jobs. Task c never ran.

Buttazzo's counterexample shows the "which task fails first?" question has a less obvious answer than "the longest period". RAN Tasks C = (2, 3, 1, 1) ms, T = (5, 9, 20, 30) ms are schedulable (U = 0.817); give τ1's first two jobs 1.5 ms extra and **τ2**, not τ4, misses its deadline.

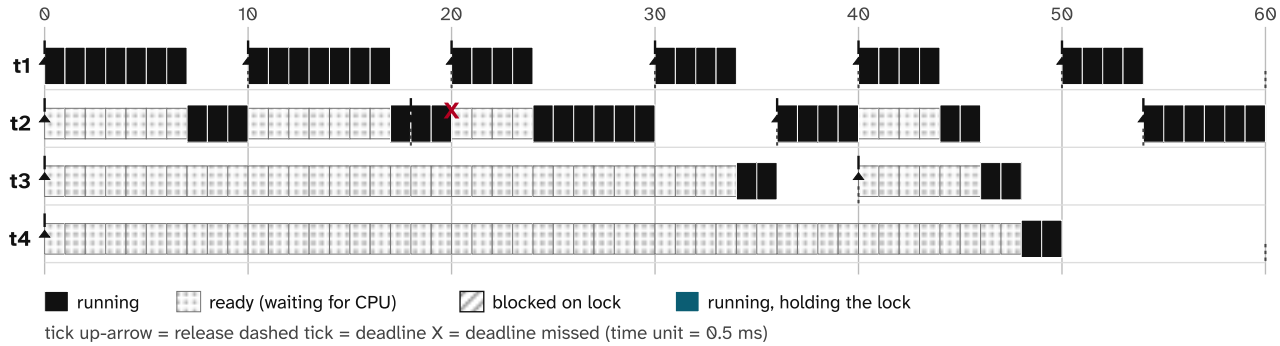


Figure 15. Transient overload under rate-monotonic scheduling. The overrun in t1 delays t2 past its deadline (X); t3 and t4 are pushed but still make theirs. Time unit 0.5 ms.

What EDF does

In permanent overload EDF does not starve a task; every task's effective period stretches by the same factor U. RAN H3 demo 3b: EDF completed **12, 8 and 4** jobs. A system whose tasks all run slower is sometimes better than one where one task never runs, and sometimes much worse; it depends on what the tasks are. A control loop that runs at half rate may be unstable; a log task at half rate is fine.

Case file: the Apollo executive

CASE FILE **Apollo 11, 1969: overload by accumulation.** (All numbers from Eyles 2004.) The guidance software read the accelerometers every 2 seconds in a task called `READACCS`, which scheduled a large job, `SERVICER`, at the **lowest** priority of the descent jobs. Resources were fixed: **eight "core sets"** (register save areas) and **five vector-accumulator areas**; at most eight jobs could be queued. In normal operation the number of jobs started equalled the number finished. Then a radar interface problem consumed **about 13%** of the computer's time. The duty-cycle margin had been "over 15%" before the landing radar locked, about 13% after, and "10% or less" once a monitor display was added. With a 10% margin and a 13% drain, the computer "simply did not have enough CPU time".

What failed: `SERVICER`, last in line, "had not yet reached its conclusion when the next READACCS, running punctually, scheduled SERVICER again." The unfinished job had not released its core set, so each new request took another. "After a short span of such operation the Executive exhausted its supply of core sets and/or VAC areas", and called `BAILOUT` with alarm **1202** (no core sets) or **1201** (no VAC areas).

What did not fail: the system. A software restart kept only the most recent copy of each restart-protected job, "flushed the uncompleted SERVICER 'stubs' that had accumulated", and terminated functions "that had not been restart protected because they were not deemed critical" (the monitor display). "Here a system of restart protection that was primarily motivated by the possibility of hardware glitches synergistically provided a means to shed computational load in response to a software logjam." In a later phase, with less than 10% margin even without the radar, "the software restart flushed the Executive queue but could not shed load" and the alarms kept coming.

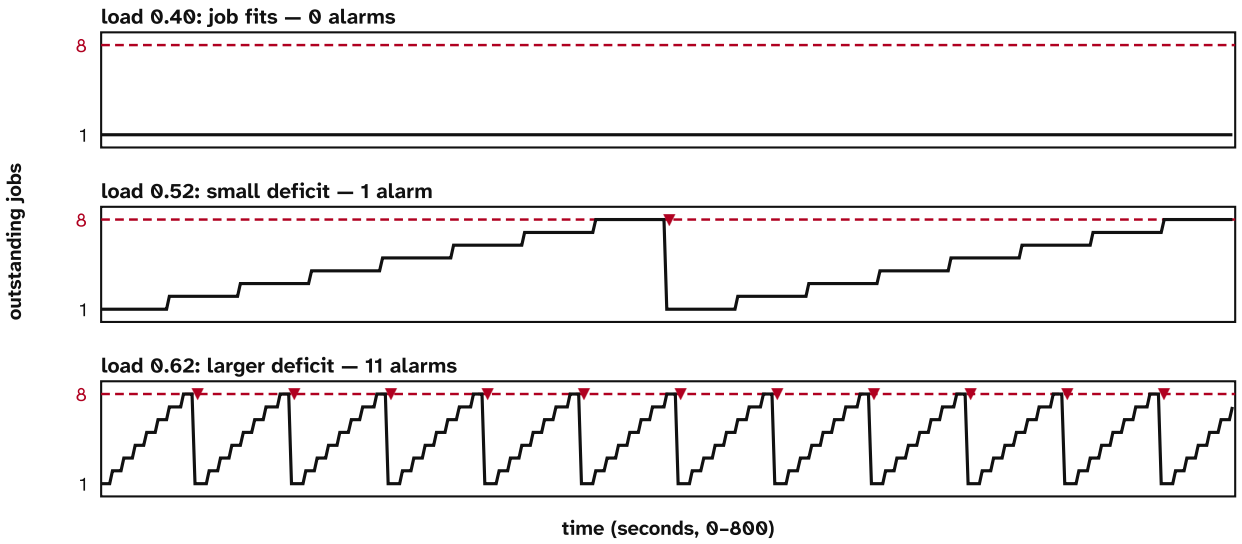


Figure 16. A toy model, not Apollo data: a fixed pool of 8 job slots and a job that does not fit its period. A small steady deficit does not fail at once: it fills the pool, then the system restarts, recovers and fails again. Lab H4.

RAN Lab H4 `apollo` toy model (period 2 s, job needs 1 s, pool 8): with the high-priority load at 0.40 the job fits and there are no alarms; at 0.52 (a small deficit) the pool fills once in 800 s; at 0.62 it fills and restarts **11 times**. The spacing of alarms is set by (pool size) ÷ (deficit per period).

Design choices when the budget is exceeded

Policy	What it does	Good for	Risk
Continue late	let the late job finish	work that is still useful	the next release arrives; backlog grows
Skip the next release(s)	drop work while behind	periodic estimates and logs	consumers see a gap; count it
Abort the job at its deadline	kill it	work that is worthless late	must leave state consistent; locks released
Degrade (shorter algorithm, lower rate)	switch to a cheaper mode	control with graceful loss of quality	the mode switch itself must be safe
Shed lower-criticality work	stop non-essential tasks	overload from many sources	needs a criticality ranking and a restart path
Restart / reset	return to a known state	when the state is suspect	loses work; restart loops (ch. 19)

Enforcement needs detection. Two real examples:

- **F´ rate groups** report “a warning event when a rate group cycle is started before previous is completed” (requirement ARG-004).

- **Pathfinder's** `bc_sched` checked each cycle that `bc_dist` had finished ("a hard deadline in the system") and, when not, "declared the error that initiated the reset": a deadline monitor with a defined, tested reaction.

> **RULE** **Rule.** Every periodic component needs three written answers: *how do I detect that a job overran, what do I do with the job, and what do I do about the next release?* If the framework gives a default, read it.

Count releases, not completions

A log that records only the response times of jobs that ran has *survivorship bias*: skipped releases never appear. **RAN** Lab H5 shows the numerical consequence for latency percentiles (chapter 20). In an RTOS, keep **three counters per periodic task**: releases, completions, deadline misses. Their differences tell the truth.

Check yourself

> **CHECK**
>
> 1. Your 10 ms task takes 14 ms once an hour. List what happens to the next release under each of: continue late, skip, abort. Which does your framework do by default?
> 2. In fixed-priority overload which of your components starve first? Is that the order you want?
> 3. What is the equivalent of Apollo's "restart protection" in your system: what survives a restart, and what is allowed to be lost?

Lab links. H3 (overload demos), H4 (`apollo` model), H5 (counting).

Shared state and races

The moment two flows of control touch the same data — two tasks, or a task and an interrupt — correctness depends on interleaving. The interleaving that breaks you will be rare, will not appear in a debugger, and will take the form of one of two small patterns.

What the evidence says concurrency bugs look like

Lu, Park, Seo and Zhou (ASPLOS 2008) examined **105 randomly selected real concurrency bugs** (74 non-deadlock, 31 deadlock) from four large programs (MySQL, Apache, Mozilla, OpenOffice). Their findings, as the paper states them:

Finding	Number	What it means for you
Non-deadlock bugs that are **atomicity violations** or **order violations**	**97%**	learn these two patterns and you have covered almost all of them
Non-deadlock bugs that are **order violations**	**32%**	"A must happen before B" bugs are common, and locks do not fix them
Bugs that need only a **partial order between 2 threads** to manifest	**96%**	tests with two tasks find most bugs
Non-deadlock bugs involving **one variable** / **several**	**66% / 34%**	a lock per variable misses a third
Bugs that manifest if a partial order among **≤ 4 memory accesses** is enforced	**92%**	the window is tiny: rare, timing-dependent
Non-deadlock bugs **fixed without adding or changing locks**	**73%**	the cure is usually a redesign: ownership, ordering, copying

EVIDENCE **Limits.** These are server and desktop programs in C, C++ and Java, not RTOS code. The authors caution that results "are associated with the four examined applications". Use the patterns, not the percentages, as a prior for what to look for.

The two patterns

Atomicity violation. A region of code assumed to execute without interruption is interleaved by another flow. The classic form is *check, then act*, or *read two related values at different times*.

Order violation. Two operations are assumed to occur in one order but nothing enforces it: a consumer reads data a producer has not yet initialised; a task starts using a queue before the task that creates it ran.

> **CASE FILE** **Therac-25, 1985–87.** The Leveson and Turner investigation of six radiation overdoses found, among other causes (process, interlocks, reporting), that the control software "allows concurrent access to shared data stored in shared variables, and that the 'test' and 'set' for such variables are not indivisible operations." The report describes race conditions on shared variables between the operator-interface task and the setup tasks, where a fast edit by an operator could be missed. The earlier machine's hardware interlocks had masked such defects; removing them exposed the software's. Two lessons carry over: **a shared flag's test-and-set must be one indivisible operation**, and **a safety net that hides a bug hides it from you too.**

A bug from the labs

RAN Lab R10: a filter task publishes an attitude estimate in a shared struct; a control task reads it at 100 Hz. The code reads one field, computes for 150 µs, then reads another assuming it describes the same sample:

```
int32_t  a  = att.angle;          /* read 1 */
uint32_t s1 = att.seq;
burn_us(150);                     /* compute with 'a' */
uint32_t ts = att.t_us;           /* read 2: assumes the SAME sample as 'a' */
uint32_t s2 = att.seq;
if (s1 != s2) torn++;             /* nothing prints this counter */
```

Of 309 control cycles, **47** read two different samples. The control loop worked, the mean latency looked fine (715 µs), and no log line mentioned it. The sensor ran from its own 997 µs clock, so the collision drifted instead of repeating, which is why a test that passed once told you nothing.

The fix is to take **one snapshot, atomically**, and then use only the copy:

```
att_t snap;
taskENTER_CRITICAL();             /* a few instructions: copy 16 bytes */
snap = att;                       /* (field-by-field in the lab) */
taskEXIT_CRITICAL();
/* use snap.* from here on */
```

Torn reads fall to 0. The same data could be passed by value in a queue, so the reader owns its copy and no one else touches it.

The tools, from least to most powerful

Tool	What it protects against	Cost and cautions
Don't share: one owner writes, others receive a copy (queue, message)	everything	a copy; the best default (chapter 15)
Single writer, atomic word: one task writes a 32-bit aligned word that others only read	torn reads of *one word*	atomic only for naturally aligned word-size accesses; **not** for structs, not for 64-bit values on 32-bit cores
Disable interrupts / critical section	interrupts and tasks	adds latency to *every* interrupt; keep to a few instructions (ch. 4)
Suspend the scheduler	other tasks (not interrupts)	blocks every task for its duration
Mutex	other tasks	may block; can invert priorities (ch. 13); never from an interrupt
Atomics / lock-free ring buffer	one producer + one consumer	subtle; for experts and well-tested library code
Double buffer / snapshot	readers needing a consistent set	memory, plus a swap that must itself be atomic

TRAP **Trap: `count++` on a `volatile`.** `volatile` forces the compiler to re-read and re-write the variable. It does **not** make read-modify-write atomic: an interrupt between the load and the store loses an increment. Eide and Regehr's EMSOFT 2008 study also found `volatile` accesses miscompiled in each of thirteen compilers tested (2008-era). Use the kernel's primitives or a language atomic.

RULE **Rule.** For every piece of shared data write down five facts: **owner** (who writes), **readers**, **protection** (copy, critical section, mutex, atomic word), **longest hold time**, and **what it blocks**. If any one is missing, there is a race you have not found yet.

Races between a task and an interrupt

An interrupt is a thread that can run between any two instructions of a task, and that cannot wait. So:

- The ISR **may not take a mutex**. It can write a word, post to a queue (FromISR), or set a flag.

- The task **must protect** multi-instruction sequences the ISR also touches (disable that interrupt or use a lock-free structure).
- A flag the ISR sets and the task clears needs thought about **both orders**: clear-then-check, or check-then-clear, can lose an event.

Why “it passed the test” means little

If a bug needs a specific ordering of a few accesses, then the probability of hitting it per run is the width of that window divided by the period of the thing that moves it. Phase-locked rates (a 1 kHz sensor and a 100 Hz controller from the same clock) can make the collision *always* or *never* happen; independent clocks make it intermittent. Review shared data by reading, and use invariants (a field and its complement; a sequence number read twice) that make the corruption *detectable*.

Check yourself

CHECK

1. A 32-bit counter is incremented in an ISR and read in a task on a 32-bit core. Is the read safe? What if the counter is 64 bits?
2. Find, in code you know, one place where two related variables are read at different times. What invariant could detect a mismatch?
3. A colleague proposes protecting a 12-byte struct with `volatile`. What do you answer?

Lab links. R10 (the torn read), R9 (watch a shared variable with gdb).

Priority inversion and deadlock

A lock makes data safe by making a task wait. If the one who waits is high priority and the one who holds the lock is low priority, the priorities have been turned upside down — and anything in between can keep the low one from finishing. This is the most famous bug in real-time systems, and it is cheap to prevent once you know its shape.

The shape

Three tasks: **H** (high), **M** (medium), **L** (low). L takes a lock. H becomes ready, preempts L, and then needs the lock: H blocks, L resumes — and then M becomes ready, preempts L, and runs as long as it likes. H, the most urgent task, waits behind M, which does not even use the lock. The wait is **unbounded**: it lasts as long as M's demand.

CASE FILE **Mars Pathfinder, July 1997.** (From the flight-software lead's own account, 15 December 1997.) The spacecraft's 1553 bus ran a cycle every 0.125 s. `bc_sched` (highest priority except the OS's own task) set up each cycle; `bc_dist` (third) collected the data; the science tasks, including the weather instrument task `ASI/MET`, were the lowest. `ASI/MET` called `select()`, whose internal bookkeeping took a mutex; it was preempted *while holding it*; `bc_dist` then called `pipeWrite()` and "blocked, taking the mutex"; "several of the medium priority tasks" ran; and when `bc_sched` woke it found `bc_dist` had not finished its cycle, "declared the error that initiated the reset." The spacecraft reset its computer, lost the rest of the day's activities, and recovered.

How it was found: "The software that flies on Mars Pathfinder has several debug features within it … not 'fortuitously' left enabled but remain in the software by design." A trace and log facility, instrumented around the pipe, queue, interrupt and `select` services, recorded into ring buffers. "In less that 18 hours we were able to cause the problem to occur [in the lab]. Once we were able to reproduce the failure the priority inversion problem was obvious." The fix "appeared obvious: change the creation flags for the semaphore so as to enable the priority inheritance" — which was not simple, because the flag was a vendor configuration variable that affected *every* `select` semaphore, so the team analysed and tested the side effects before changing the flight software.

Why it was missed: the problem "would only manifest itself when ASI/MET data was being collected and intermediate tasks were heavily loaded", and testing used "best case" data rates. The team had seen a reset before landing but could not repeat it, and ranked it below entry-and-landing work. Their lesson: "A good lesson when you fly COTS stuff — make sure you know how it works."

See it

RAN Lab R3 reproduces the structure on FreeRTOS. Task `met` (L) holds a lock for 10 ms; `bus_dist` (H) needs it; `comms` (M) is CPU-bound for 40 ms; `bc_sched` checks each 50 ms that `bus_dist` finished.

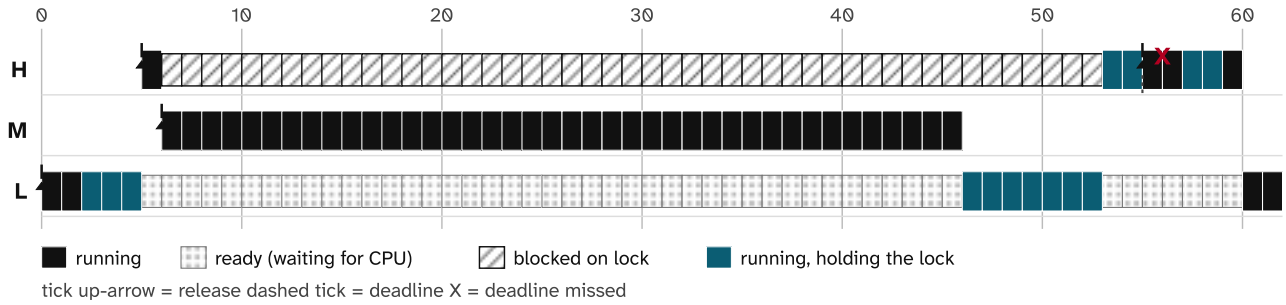


Figure 17. No protocol (simulator): H blocks at 6 ms, M runs from 6 to 46 ms, L finishes its critical section at 53 ms, H completes at 56 ms: a miss (X).

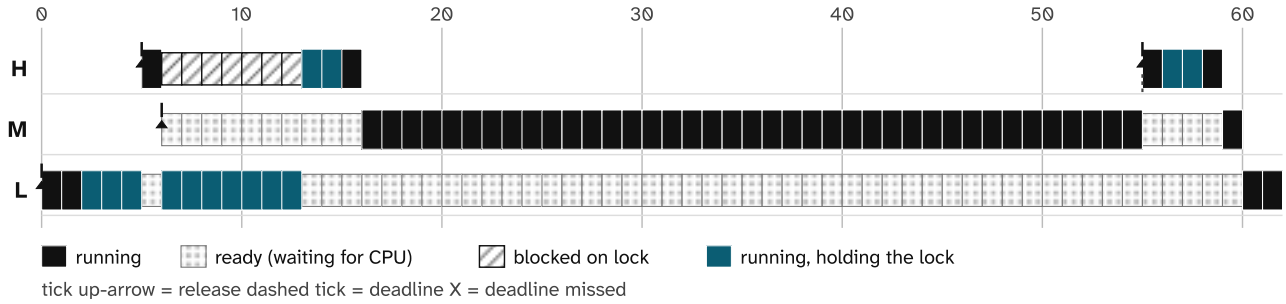


Figure 18. Priority inheritance: when H blocks, L runs at H's priority, leaves the critical section at 13 ms, H completes at 16 ms.

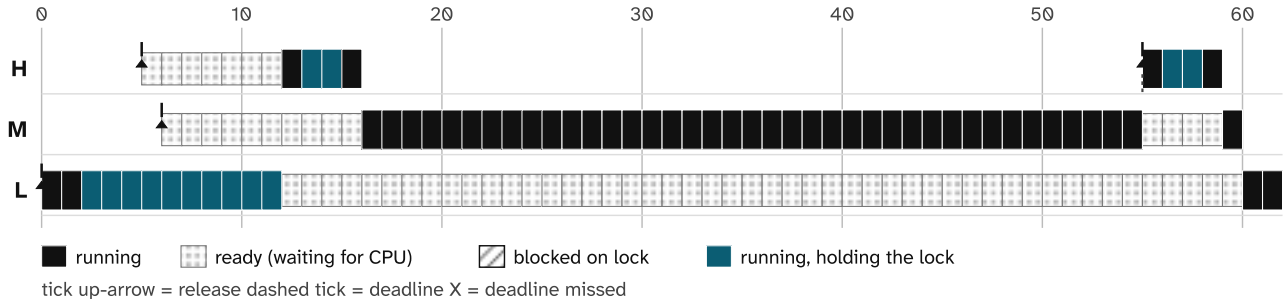


Figure 19. Immediate priority ceiling: L runs at the ceiling priority from the moment it takes the lock; H waits for the CPU, not for the lock.

The same experiment on the emulated kernel, from a context-switch trace:

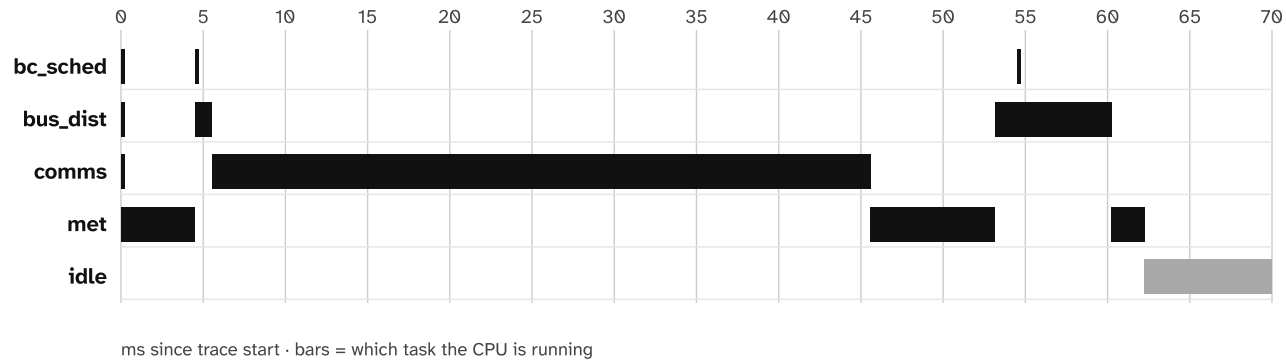


Figure 20. Real trace, binary semaphore: `comms` runs from 5 to 46 ms while `bus_dist` is blocked; `met` finishes at 53 ms and `bus_dist` at about 60 ms. `bc_sched` (tick at 55 ms) sees it unfinished.

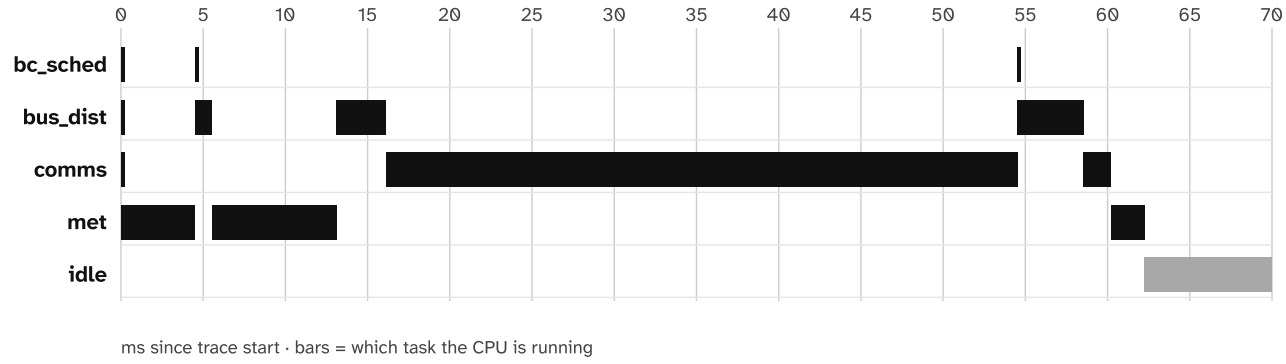


Figure 21. Real trace, mutex with inheritance: `met` finishes its critical section at about 13 ms and `bus_dist` runs at once.

	binary semaphore	**mutex**
`bus_dist` first response	**51 ms** (deadline 50): miss	**11 ms**
misses in 1 s	**5**, each caught by `bc_sched` as a "reset"	**0**
difference in code	`xSemaphoreCreateBinary()` + one `give`	`xSemaphoreCreateMutex()`

Protocols that bound the blocking

Protocol	**Rule**	**Bound on blocking**	**Costs**
None	—	unbounded	—
Priority inheritance	while a task holds a lock that a higher-priority task waits for, it runs at the waiter's priority	at most the length of the critical sections of the lower-priority tasks it can be blocked behind (possibly several in a chain)	can chain; does not prevent deadlock
Priority ceiling (immediate / original)	a lock has a ceiling = highest priority of any task that uses it; a holder runs at the ceiling	at most **one** critical section of a lower-priority task	needs all users known in advance; prevents deadlock (Sha, Rajkumar, Lehoczky, 1990)

Sha, Rajkumar and Lehoczky's paper begins: "A direct application of commonly used synchronization primitives such as semaphores, monitors, or the Ada rendezvous can lead to uncontrolled priority inversion, a situation in which a higher priority job is blocked by lower priority jobs for an indefinite period of time", and shows both protocols solve it; the ceiling protocol "reduces the worst case task blocking time to at most the duration of execution of a single critical section of a lower priority task" and "prevents the formation of deadlocks."

In FreeRTOS, "mutexes include a basic 'priority inheritance' mechanism, whereas binary semaphores do not." The same book cautions: inheritance "does not 'fix' priority inversion, but merely lessens its impact by ensuring that the inversion is always time bounded", it "complicates system timing analysis, and it is not good practice to rely on it for correct system operation." OSEK/AUTOSAR resources use the (immediate) ceiling protocol.

The blocking term. In the response-time equation (chapter 9), B for a task is the longest critical section of any *lower*-priority task on a lock whose users include a task of priority at or above it. Making critical sections short is therefore the most effective way to shrink every higher-priority task's response time.

Look at it with a debugger

RAN Lab R9 stops a gdb session the instant the checker declares the first reset and prints the kernel's lists with the helper `fr-tasks`:

```
task         state                       prio  notes
bc_sched     RUNNING                        6
bus_dist     suspended/blocked-forever      5  blocked on queue/semaphore @0x20000ff0
met          ready                          3
--- the lock (a binary semaphore): who owns it? ---
(no owner: counting/binary semaphores and queues are not owned by anyone)
```

A binary semaphore has **no owner**, so the kernel cannot know whom to boost. In phase 2 (a mutex), stopping at `xTaskPriorityInherit` and listing again shows `met ready 5 <-- priority 5 inherited (base 3)` and `mutex holder = met`.

Deadlock

Two or more tasks each hold something the other needs. The four conditions (Coffman et al. 1971, as presented in OSTEP): mutual exclusion, hold-and-wait, no preemption, circular wait. Lu et al. found that **97%** of the deadlocks in their sample involved **two threads** waiting on **at most two resources**, and **22%** were a single thread acquiring a lock it already held.

Defence	How
Lock order	one global order for taking locks; review pairs of locks
No blocking while holding	never call a function that can block (queue, delay, print) inside a critical section

Defence	How
Timeouts	`xSemaphoreTake(m, ticks)` with a finite timeout and a defined reaction
Recursive mutex	only if re-entry is intended; otherwise a self-deadlock is a bug you want to *find*
Ceiling protocol	prevents the circular wait by construction

TRAP **Trap: a library call that locks.** Pathfinder's lock lived inside the operating system's `select()`. Any call into a library, a driver, a logging framework or the standard I/O can take a lock. For every such call from a high-priority task ask: *what does it lock, and who else holds that lock at lower priority?*

RULE **Rule.** Use a **mutex** (with inheritance) for mutual exclusion, never a binary semaphore. Keep critical sections short and free of blocking calls. Never take a mutex in an interrupt. List every lock with its users' priorities.

Check yourself

CHECK

1. Draw a three-task inversion on paper using any numbers. How long is H blocked with no protocol? With inheritance?
2. In your system, find a lock used by tasks of different priorities. Is it a mutex? What is the longest time it is ever held?
3. Task A takes lock X then Y; task B takes Y then X. What happens, and what is the cheapest rule that prevents it?

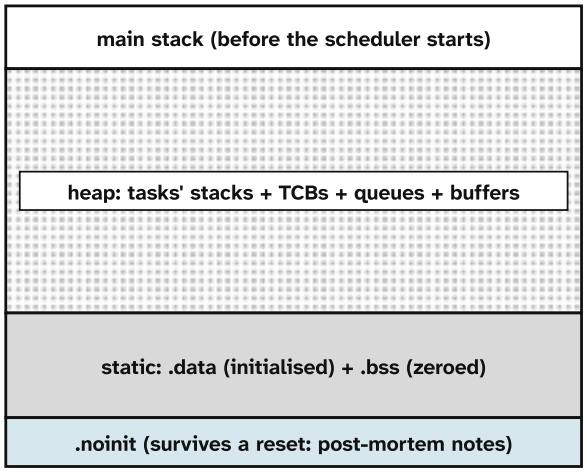
Lab links. H3 demo 4, R3, R9.

PART 3 · SHARE · CHAPTER 14

Memory: stacks, heaps and budgets

You will not manage memory addresses, but you will choose stack sizes, allocate buffers and decide whether a queue can grow. On a small system, memory is a fixed budget with no operating system to say “out of memory” politely, and the failure usually arrives after weeks, not on the first run.

Where RAM goes



Stack: per task, fixed size at creation.

Overflow corrupts the neighbour first (R6).

Heap: fragmentation can fail an allocation

with plenty of total free space (R6).

Static: sizes known at link time — preferred.

Rule (Holzmann): no allocation after initialization.

Measure: high-water marks, min-ever-free heap.

Figure 22. RAM of a small system. The task stacks, task control blocks, queues and buffers all live in the heap in FreeRTOS; static data is sized at link time.

Region	Holds	Lifetime	Sized by	Failure
Static (`.data`, `.bss`)	globals, statics	whole run	the linker	none at run time (it just does not link)
Stack (one per task)	locals, call frames, saved registers	while the task runs	**you, at creation**	overflow corrupts a neighbour
Heap	tasks, queues, buffers, `pvPortMalloc` blocks	until freed	`configTOTAL_HEAP_SIZE`	allocation fails, or fragments
`.noinit`	data that survives a reset	across resets	linker section	used for post-mortem notes (ch. 19)

The no-allocation-after-init policy

Holzmann's "Power of 10" (JPL, 2006), ten rules for safety-critical C, includes: simple control flow (no `goto`, no recursion), **every loop has a fixed upper bound**, assertions (at least two per function on average), and **no dynamic memory allocation after initialization**. The rules are meant to be checked mechanically. The reasoning is timing and failure: allocator time varies, fragmentation is path-dependent, and an allocation can fail where the code has no plan for failure. Many flight and automotive frameworks enforce this for you; **find out whether yours does** (chapter 22).

Option in FreeRTOS	Behaviour
static allocation (`xTaskCreateStatic`, …)	every object lives in memory you provide; no heap
`heap_1`	allocate only, never free: deterministic, cannot fragment
`heap_2`	best-fit, can free, **does not merge** neighbours; "not recommended for new designs"
`heap_4`	first-fit **with merging** (coalescing) of adjacent free blocks; the usual choice
`heap_5`	`heap_4` across several memory regions

Fragmentation, run

RAN Lab R6, `heap_4`, 48 KB heap. Allocate 40 blocks of 1,000 bytes; free every other one:

State	Free total	Largest single block	Free blocks
after allocating 40	5,728 B	5,728 B	1
after freeing every other	**25,888 B**	**5,728 B**	21
`pvPortMalloc(8000)`	—	—	**fails** (NULL) though 25,888 B are free
after freeing the rest	46,048 B	46,048 B	1
`pvPortMalloc(8000)`	OK	—	neighbours merged

"Free bytes" is a misleading number. Watch the **largest free block** and the **minimum ever free** (`xPortGetMinimumEverFreeHeapSize`): the latter is the margin you actually had.

Stacks: the quiet way to fail

Each task's stack is a fixed array. A call chain deeper than expected, a large local array, or recursion pushes the stack pointer past the end into **whatever is next in memory** — often another task's stack, a queue, or the kernel's own data — and nothing stops it.

The kernel can *look*, but only occasionally: `configCHECK_FOR_STACK_OVERFLOW` = 1 checks the stack pointer at **context switches**; = 2 additionally checks a fill pattern at the stack's end (the last 20 bytes). The FreeRTOS book: method 1 "can miss stack overflows that occur between context switches."

RAN Lab R6 part 3: a task with a 512-byte stack recurses to about 1.2 KB. It returns normally and prints "nothing has noticed". At its next context switch the hook fires. By then **720 of 2,048 bytes** of the neighbouring heap block (a canary placed just below) had been overwritten. The check is a *detector after the damage*, not a guard.

How to size a stack. Run the system through its worst case (slowest path, deepest call chain, all interrupts firing), read the **high-water mark** (`uxTaskGetStackHighWaterMark`: the minimum free words ever), and keep a written margin. RAN In Lab R6 the high-water report listed each task's minimum free stack: `manager` 145 of 256 words, `IDLE` 118, `Tmr Svc` 224. A task near zero is one code change from an overflow. Static stack-depth analysis also exists (Regehr et al.; not read here).

CASE FILE **Toyota throttle software, as testified (2013).** An expert for the plaintiffs reported that the stack was "94% full plus any recursion", that there was "no mitigation for stack overflow" and an "incorrect assumption that overflow always results in a system reset", and that "memory just past stack is OSEK RTOS area", so an overflow could damage the operating system's data. SEC Expert slides in contested litigation: take it as the *list of questions* a reviewer should be able to answer for any system: *how full is each stack at worst, is recursion banned, what is next to each stack, what detects overflow and what does it do?*

Resources that grow with mission time

CASE FILE **Spirit rover, January 2004.** After 18 sols on Mars the rover stopped responding to commands and rebooted repeatedly. The JPL paper by Reeves and Neilson: "The root cause of the anomaly was a design error in the software module that provides file system services. In addition, two significant configuration errors detrimentally affected the overall behavior of the system once the root cause error occurred." In outline: the file system library, at boot, allocated memory based on the number of files on flash (including deleted ones); files accumulated over seven months of cruise and a few days of operation; the memory the file system needed at boot became more than was available; the flight software entered a **delayed reset loop**. Recovery: use commands that avoid the file system ("crippled" mode) each sol, delete files, and later reformat. The lesson is a pattern: **a resource consumed at start-up, that grows with time in service, turns into an unrecoverable boot loop.**

Message and buffer ownership

A buffer passed between tasks needs an **owner at every moment**. The usual rules: a message is **copied** into the queue (the sender may reuse its buffer at once; the cost is the copy), or a **pointer** is passed and ownership moves with it (cheap; the sender must not touch it again, and *something* must free it). Pool allocators (fixed-size blocks from a static array) give constant-time, fragmentation-free allocation; make the pool depth part of the message's design (chapter 15).

> **RULE** **Rule.** For every allocation: who frees it, what happens if it fails, and what is the largest the pool can get. If an answer is "nobody", "we crash", or "unbounded", fix it before you ship.

Check yourself

CHECK

1. A heap shows 30 KB free and a 6 KB allocation fails. Name two measurements that would confirm fragmentation.
2. A task's stack high-water mark is 12 words free of 256. What do you do first, and what do you check afterwards?
3. Which resources in your system grow with uptime or number of files, messages or log entries? What bounds each?

Lab link. R6 (stack, heap and overflow); R5 uses `.noinit` for post-mortem data.

PART 4 · FLOW · CHAPTER 15

Queues and backpressure

A queue between two components lets each run at its own pace. It also hides a question that every queue must answer: what happens when the producer is faster than the consumer for long enough to fill it? “It won’t” is not an answer; it is the assumption behind most message-loss incidents.

What a queue decouples, and what it does not

A queue lets a producer (often an interrupt) hand data to a consumer (a task) without waiting for it. It removes the *timing coupling* of a direct call. It does **not** remove the *capacity coupling*: if the average arrival rate exceeds the average service rate the queue fills however large it is; if only the *peak* rate exceeds it, the queue size decides whether the burst is absorbed or lost.

FreeRTOS queues copy items in, block senders and receivers with a timeout, and have “FromISR” variants that never block. A queue of length one with `xQueueOverwrite` implements “latest value wins”, which is right for a sensor *state* and wrong for an *event*.

The four decisions every queue needs

Decision	Options	Question to ask
Depth	a number, from a burst calculation	how long can the consumer fall behind, at what rate?
Overflow policy	block the producer · drop newest · drop oldest · overwrite · error and escalate	which data is worth more: old or new? Can the producer block at all (an interrupt cannot)?
Visibility	a counter of drops, a high-water mark of depth	will anyone *notice* a loss?
Age limit	maximum useful age of an item	is a 200 ms-old sample better than none?

> **RULE** **Rule.** A queue without a written overflow policy and a drop counter is a bug waiting for load. The policy is a *requirement*; the framework’s default is a *choice somebody else made*.

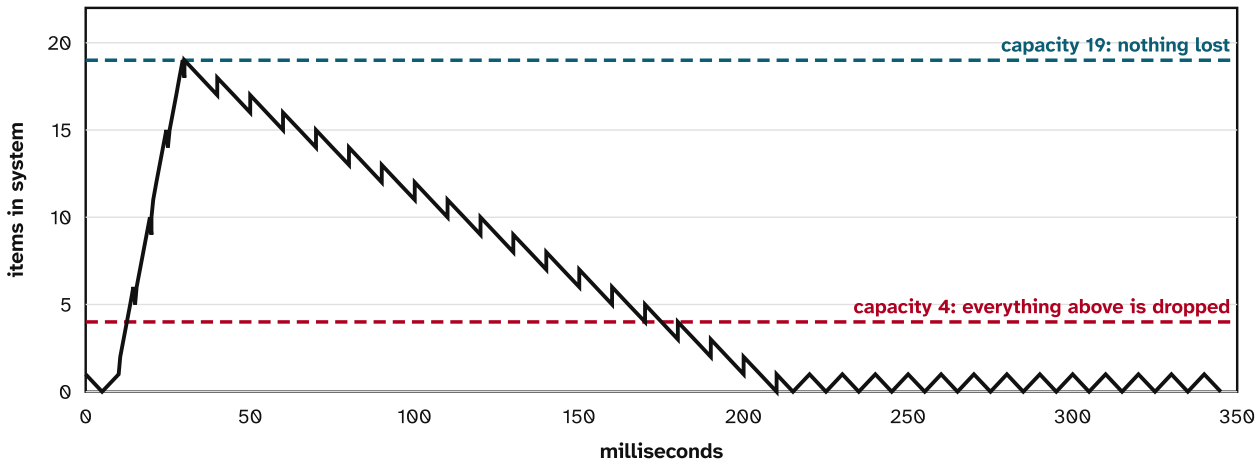
Sizing from a burst

Average load hides the burst. The formula is arithmetic, not statistics:

depth ≥ ⌈(peak arrival rate − service rate) × burst duration⌉ + 1

Little's law ($L = \lambda W$: the average number in a stable system equals the arrival rate times the average time each item spends in it) gives the *average* occupancy and is a good sanity check, but a queue must be sized for the peak.

RAN Scenario: 100 messages/s steady plus one burst of 20 messages in 20 ms each second (peak 1,100/s); the consumer needs 5 ms per message (200/s capacity, so the mean load is a comfortable 60%). Formula: $\lceil (1{,}100 - 200) \times 0.020 \rceil + 1 = \mathbf{19}$.



Host simulation (lab H4): 100 msg/s steady + a 20-message burst at 1000 msg/s; consumer 5 ms per message (200 msg/s).

Figure 23. Items in the system during one burst (host simulation). A queue of 19 holds the burst; a queue of 4 loses everything above the red line. The drain takes about 95 ms after the burst ends.

Queue depth	Host simulation, 10 s (lab H4)	FreeRTOS on the emulator, 5 s (lab R4)
4	134 dropped	75 dropped
8	98 dropped	55 dropped
18	8 dropped	5 dropped
19	**0 dropped**	**0 dropped**

Three independent routes — the formula, a simulation and a real kernel with an interrupt producer — agree on the same threshold: 19 slots, with 18 losing messages. Both also agree that with **no drops the worst latency is 95 ms** (a 5 ms service time × 19 queued): **a bigger queue buys loss-free delivery with latency**. If an item older than 50 ms is useless, a queue of 19 is the wrong design even though it never loses anything.

Choosing the policy

Policy	Latency of what you keep	What you lose	Use when
Block the producer	highest (producer is held)	nothing, if the producer can wait	the producer is a task and can be slowed

Policy	Latency of what you keep	What you lose	Use when
Drop newest (queue full → refuse)	old data stays	fresh data	**events** that must be processed in order, with bounded loss
Drop oldest (make room)	lowest	stale data	**state** where only the latest matters
Overwrite (length 1)	lowest	all but latest	sensor values, set-points
Escalate (error, shed load, restart)	—	—	loss is unacceptable and must be handled

RAN In Lab R4, at depth 4 the drop-oldest policy kept the worst latency at **20 ms** against **24 ms** for drop-newest, with the same number of drops. It keeps fresher data. Which is correct depends on whether the data is state or an event.

An interrupt cannot block. Its only options are to drop (and count), overwrite, or notify a task and move work out of the interrupt altogether (chapter 4).

Unfinished work is a queue too

CASE FILE **Apollo 11 (1969): the queue was the job table.** Each unfinished `SERVICER` job held one of eight "core sets". Work was never dropped by a producer; the *queue of unfinished jobs* grew until its fixed pool ran out and the executive restarted. Recognise the pattern wherever a periodic job can start before the previous instance finished: in thread pools, callback queues, rate groups with queued cycle messages, and retry loops. (Chapter 11.)

A consumer that stops silently

CASE FILE **Northeast blackout, 14 August 2003.** The US–Canada task force found that the alarm software at the utility "essentially 'stalled' while processing an alarm event", operators "were unaware" their alarms had stopped, and that at 14:41 "the primary server hosting the EMS alarm processing application failed, due either to the stalling of the alarm application, 'queuing' to the remote EMS terminals, or some combination of the two." Press coverage of the vendor's analysis describes "a couple of processes that were in contention for a common data structure" that "were both able to get write access … at the same time", a fault found only after weeks of searching about a million lines of code, in software the vendor said had run "in excess of three million online operational hours" without exercising it; once the alarm function stopped, "unprocessed events began to cue up" and the server hosting it failed SEC (the task force's own text says "stalled"; the race detail is the vendor's finding as reported). The pattern

> generalises: **a consumer that stops, with a queue in front of it, produces silence, then delay, then overflow.** What was missing was any signal from the alarm function that it was alive.

How to see a stalled consumer: expose **queue depth**, **age of the oldest item**, and a **heartbeat** from the consumer that a separate monitor checks (chapter 19). Depth rising with no consumer progress is a stall, not a burst.

Queues and the analysis

A queue breaks Liu and Layland's "independent tasks" assumption: the consumer's response time now depends on the producer's pattern. Two practical consequences:

1. **Treat the consumer as sporadic**, with a *minimum inter-arrival* equal to the fastest the queue can be refilled (often the producer's period), and a deadline equal to the age limit minus the queue wait.
2. **Add the queue wait to the end-to-end budget.** In the example above, the worst item waits 95 ms before it is even processed (chapter 17).

Check yourself

CHECK

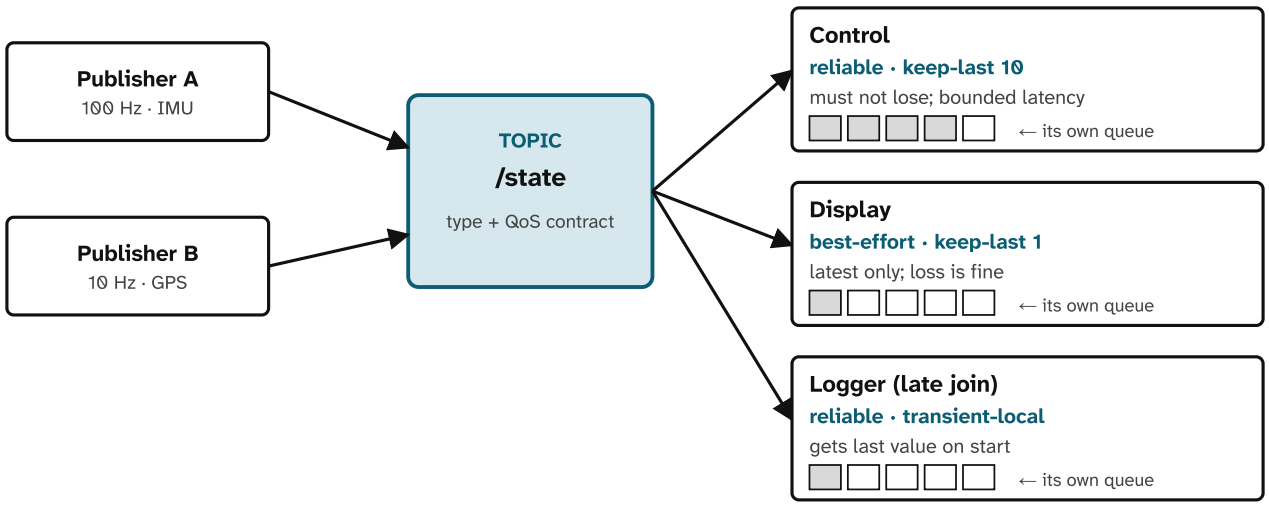
1. A sensor posts 500 samples/s, mean; during a 40 ms fault it posts 4,000/s. The consumer handles 800/s. Size the queue; state the policy; what extra latency does a full queue add?
2. Where in your system does the producer *not* know the consumer's rate? What does it do when the queue is full?
3. A consumer task blocks forever on a lock. Which numbers on a dashboard would show it within one second?

Lab links. H4 (host simulation and the `apollo` model), R4 (interrupt to queue).

Publish/subscribe and quality of service

In publish/subscribe a component sends data to a *topic* and does not know who receives it. That decoupling is the reason frameworks use it, and the reason timing problems are harder to see: the contract is no longer in a function signature. It is in the quality-of-service settings, or it is nowhere.

The model



Each arrow has a rate, a queue depth, a loss policy and an age limit — requirements, even when nobody wrote them down.

Figure 24. Publishers write to a topic; each subscriber has its own queue and its own QoS. The same data, three different guarantees.

Piece	What it is	Where it lives
Topic	a named channel with a message type	the framework's directory / middleware
Publisher	writes messages at its own rate	its own task or callback
Subscriber	receives into **its own queue**, then processes	the subscriber's task or executor
Message	typed data, ideally with a source timestamp and sequence number	allocated by the framework
QoS	the rules for delivery on each connection	configured per publisher and per subscriber

Three consequences follow from “each subscriber has its own queue”: a slow subscriber affects only itself; the publisher cannot know whether anyone is behind; and **every subscriber needs its own sizing and overflow analysis** (chapter 15).

The QoS vocabulary (ROS 2 / DDS)

ROS 2 documents these policies for every publisher and subscription:

Policy	Options	What it decides
History	*keep last* N · *keep all* (bounded by resource limits)	how many samples a slow reader can accumulate
Depth	the N for keep-last	queue size (“only honored if the history policy was set to keep last”)
Reliability	*best effort*: “may lose them if the network is not robust” · *reliable*: “may retry multiple times”	loss versus latency and retries
Durability	*transient local*: “the publisher becomes responsible for persisting samples for late-joining subscriptions” · *volatile*	what a **late joiner** sees
Deadline	“the expected maximum amount of time between subsequent messages being published to a topic”	detects a publisher slower than promised
Lifespan	“the maximum amount of time between the publishing and the reception of a message without the message being considered stale or expired (expired messages are silently dropped …)”	**age limit**
Liveliness + lease duration	automatic or manual; “the maximum period of time a publisher has to indicate that it is alive … (losing liveliness could be an indication of a failure)”	failure detection

A publisher and subscriber **match** only if the publisher offers at least what the subscriber requests: a best-effort publisher and a reliable subscriber do not connect, and the framework reports an incompatible-QoS event rather than failing loudly in your code. Read the compatibility table in your framework’s documentation for the full matrix.

EVIDENCE **Evidence and limits.** The list above is quoted from the ROS 2 documentation (rolling release). Other pub/sub systems (DDS itself, cFS's software bus, F′'s ports, MQTT) use different words for overlapping ideas. We do not claim identical semantics across them. The *questions* carry over: how many samples are kept, can they be lost, how old may they be, who hears the last one, who notices silence.

Translating the vocabulary into requirements

Question about a connection	QoS knob	Example answer
Can this data be lost?	reliability	control commands: no; display refresh: yes
How many samples may pile up?	history / depth	latest only (1) for state; N for a filter that needs a window
How old is too old?	lifespan, deadline	a position older than 100 ms is not used
Does a late joiner need history?	durability	a logger that starts late needs the last configuration
How do I know the publisher is alive?	liveliness + lease	missing for 3 periods → degrade
Who is slower than whom?	rate × depth	100 Hz producer, 10 Hz consumer, depth 10 → 1 s of backlog

RULE **Rule.** For every subscription write the sentence: *"This subscriber needs __ from this topic at least every __ and at most __ old; if it cannot keep up it __."* The blanks are your QoS.

What the middleware does not give you

- **Timing.** QoS says what to deliver, not when. Delivery latency depends on the transport, serialization, the executor and the priorities of the threads involved. Casini, Blaß, Lütkebohle and Brandenburg (ECRTS 2019) had to model the ROS 2 executor's scheduling to analyse response times of callback chains, reporting its timing behaviour as undocumented at the time; the practical rule is that **callbacks in an executor are scheduled by the executor's own order, not by your RTOS priorities**.
- **Ordering across topics.** Two topics have two queues; messages from different topics have no defined order unless you timestamp and merge.
- **A consistent snapshot.** A subscriber reading three topics sees them at three times (chapter 17).
- **Liveness.** A silent publisher looks the same as one with nothing to say, unless you set a deadline or liveliness.

The same ideas in other frameworks

Framework	Pub/sub construct	Where the contract lives
ROS 2	topics, QoS profiles	QoS settings, executor configuration
cFS	**software bus**: apps publish CCSDS messages and subscribe by message ID through **pipes**	pipe depth, message limits, scheduler table (check your build's values)
F´	typed **ports** wired point-to-point in a topology; async ports have a **queue per component**; guarded ports use a mutex	port kinds and queue depth per component instance
AUTOSAR	signals via the RTE; SOME/IP for service-oriented	signal/timing configuration
Your own	a queue per subscriber	whatever the code implies

F´'s "async port" is the clearest bridge from chapter 15: a message arrives at a port; the component's queue holds it; the component's own thread dequeues and handles it. cFS's pipe is the same object under another name.

TRAP **Trap: assuming "reliable" means "on time".** Reliable delivery retries; retries add latency, and a retransmission storm can load the network and the CPU exactly when the system is already struggling. For control data that must be fresh, best-effort with a short lifespan is often safer than reliable with a long queue.

Check yourself

CHECK

1. A 100 Hz IMU topic feeds a 100 Hz controller and a 1 Hz logger. Write the QoS for each subscriber and justify the difference.
2. A subscriber joins 10 s after the publisher started. What do you want it to receive first? Which setting controls it?
3. Your framework's default queue depth for a subscription is 10. At 200 Hz publishing and a consumer that stalls for 100 ms, what is lost, and does anything say so?

Rates and end-to-end latency

Each component has a rate. Why that rate? And when data crosses from a fast component to a slow one, or through a chain of three, how old is it when it arrives? These questions decide whether a system whose every component meets its deadline still misses its requirement.

Why a component runs at its rate

Driver	The question	Typical consequence
Responsiveness	how soon must the world see a reaction?	a lower bound on the rate (and an upper bound on latency)
Input data rate	how fast does the data arrive, and does an extra run add information?	running faster than the input only repeats work; running slower drops information
Dynamics of the controlled thing	how fast can it change?	control stability needs a minimum rate
CPU budget	what is the utilization (chapter 8)?	an upper bound on the rate
Jitter tolerance	how much variation in the period is acceptable?	a limit on the priority and priority-inversion budget
Compatibility	what rates do neighbours use?	harmonic rates simplify everything (below)

A rule from control theory. Åström and Wittenmark's rule of thumb, quoted in Cervin's thesis (2003): the sampling interval h should satisfy $\omega_c \cdot h = 0.2\text{–}0.6$, where ω_c is the closed-loop bandwidth. "A real-time system where the product $\omega_c \cdot h$ is small for all control tasks will be less sensitive to scheduling-induced latencies and jitter, but it will also consume more CPU resources." The same thesis states that variation of the sampling interval below **10%** of nominal "need not be compensated for", and that latency and jitter together "degrade the control performance". You will usually be *given* the rate; this tells you what you are trading when you are asked to move it.

Harmonic rates

Rates are **harmonic** if each is a whole multiple of the slower ones: 100, 50, 25, 10 Hz. Benefits: the hyperperiod is the longest period (ch. 5); RM can reach U = 1 (ch. 8); samples line up, so a slow task's input is exactly the n-th fast sample; end-to-end patterns repeat every long period. Non-

harmonic rates (7, 11 and 13 ms; hyperperiod 1,001 ms) move relative phases continuously, which finds rare collisions but makes tests slow to cover them (chapter 12).

Crossing rates

From → to	Problem	Common choices
fast → slow (100 Hz → 10 Hz)	which of ten samples does the slow task use?	take the latest · average · decimate · keep a window
slow → fast (10 Hz → 100 Hz)	what does the fast task use between updates?	hold · extrapolate · interpolate (adds latency)
unrelated rates	aliasing, phase drift, occasional double/zero delivery	resample with timestamps

Pick **one** rule per crossing and write it down. An unwritten crossing is an implicit decision made by whichever component reads last.

End-to-end latency is not the sum of response times

Consider a chain: a sampling task (T = 10 ms), a filter (T = 20 ms), a controller (T = 40 ms), each reading the latest value the previous one wrote. Each task's response time may be a few milliseconds; the *data* waits for the next period of every stage.

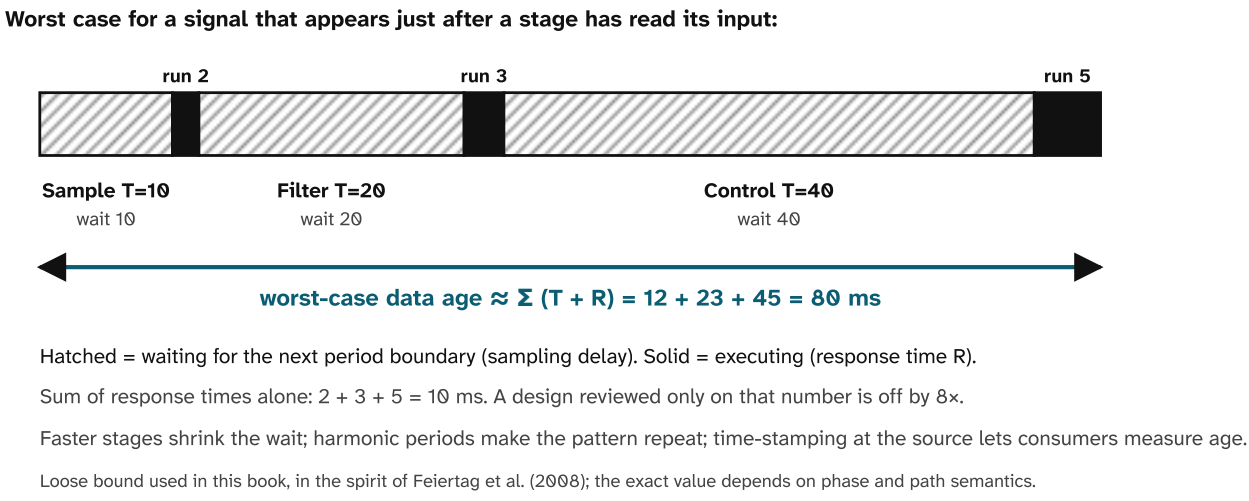


Figure 25. Worst case for a signal that appears just after a stage has read its input: it waits a full period (hatched) and then a response time (solid) at every stage: 80 ms in total against 10 ms of execution.

Feiertag, Richter, Nordlander and Jonsson (2008) formalised this for register-based systems with periodic sampling, and showed that "different path semantics" give different numbers: **data age** (how old is the newest value when it is used? — what control engineers need) and **reaction time**

(how long until a stimulus has some effect? — what body electronics needs). “Because the two can differ, a clear distinction is mandatory.” This book uses a loose bound, in the spirit of their work and not a quotation: for a register-based chain of tasks,

worst-case data age ≲ Σ (T_i + R_i)

and checks it by measurement rather than relying on it. A stricter analysis exists and depends on phase and on how data is passed. The lesson that survives all variants: **end-to-end latency is dominated by the sampling delays at the rate boundaries**. Faster stages, aligned phases, and **timestamping at the source** shrink or reveal it.

> RULE **Rule.** Stamp each datum with the time it was *measured*, at the place it was measured, and carry the stamp with it. A consumer that can compute the **age** of its input can check its requirement; one that cannot is guessing.

When the timestamp is wrong

> CASE FILE **Ingenuity Mars helicopter, Flight 6, 22 May 2021.** About 54 seconds into the flight, NASA/JPL reported, “a glitch in the pipeline of images delivered by the navigation camera” caused a single image to be lost, and “all subsequent navigation images” to be “delivered with an erroneous timestamp”. The navigation filter kept correcting for errors that were not there; “large oscillations ensued” (roll and pitch excursions above 20 degrees) and the helicopter landed safely, aided by a safeguard that ignored images during the descent. In a follow-up post, the team’s hypothesis was that “the large CPU load involved in capturing the RTE’s 13-megapixel color images, could result in rare instances of navigation camera images being dropped in the pipeline”, and the fix would “address the Flight 6 anomaly, where image timing delays manifested into aircraft estimation and control challenges.” **What it teaches.** (1) A **dropped message plus a position-based timestamp** corrupts every later message: the stamp must travel with the datum. (2) The proximate trigger was **CPU load** causing rare drops, so budget the load of an optional feature, not only the baseline. (3) A consumer should **check the consistency of timestamps** (a gap or a repeat means a loss) and degrade rather than act. The posts do not say which software layer assigned the timestamps.

Phase and the clocks that disagree

Two periodic activities derived from the same clock have a **fixed phase**; from independent clocks, a **drifting** one. RAN In Lab R10 the sensor runs from its own 997 µs clock while control runs at 10 ms ticks; a read-twice race collided with the filter in 47 of 309 cycles and drifted with time. Had

they been phase-locked, it would have collided *every* cycle or *never* — either of which passes or fails a short test for the wrong reason (chapter 12).

A rate review in five questions

1. What is the **source** rate, and is it exact or does the sensor have its own oscillator?
2. What is the **required reaction time** end to end, and what is its source (a person, a spec, a plant)?
3. Which rate crossings exist, and which rule applies at each?
4. What is the **worst-case data age** at the consumer, from the formula, and what did the *measurement* give?
5. What happens when a message is missing, repeated, or out of order?

Check yourself

CHECK

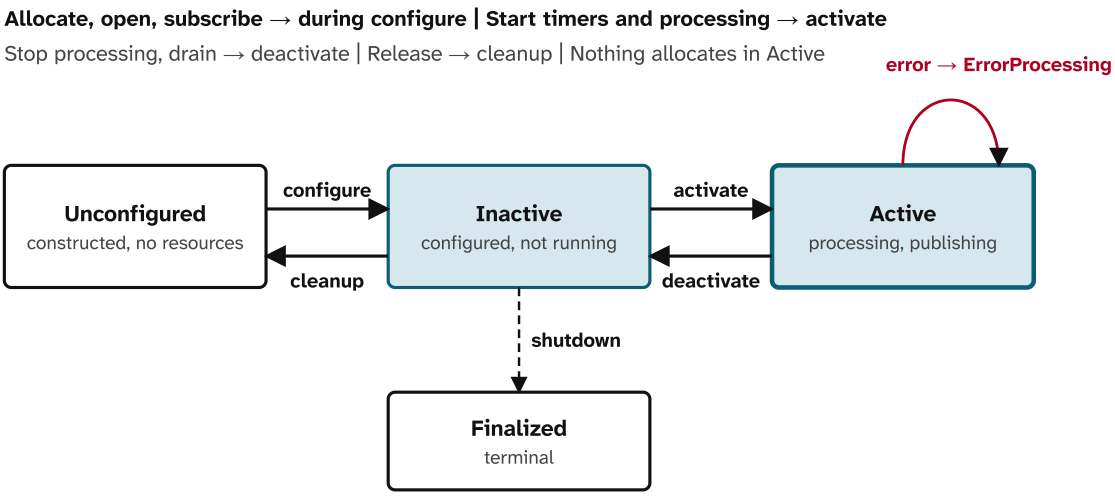
1. A chain samples at 5 ms, filters at 10 ms and commands at 20 ms with responses of 1, 2 and 3 ms. Compute the loose worst-case data age. Which stage change reduces it most?
2. A 50 Hz task uses the latest of a 400 Hz stream. What is the worst-case age of the sample it uses, ignoring jitter?
3. Where is the timestamp on a message in your system assigned? By what clock?

Lab links. R10 (timestamps and latency histograms), H5 (reporting latency honestly).

Lifecycles: components and messages

Every component is born, configured, run, stopped and destroyed, and every message is created, stamped, queued, processed and released. Bugs live in the transitions: the queue that does not exist yet when the first message arrives, the resource released while a message still points at it, the restart that forgets what it was doing.

A component's life



Shape follows the ROS 2 managed-node design (4 primary states, 7 transitions); other frameworks use other names for the same ideas.

Figure 26. A generic lifecycle, shaped like the ROS 2 managed-node design. Allocation belongs in configure; processing only in Active; nothing allocates while Active.

State	What may happen	What must not
Unconfigured	constructed; no resources	processing
Inactive	configured: memory allocated, queues created, subscriptions made, hardware opened	publishing live data; accepting service requests
Active	timers running, messages processed and published	allocation; reconfiguration
Finalized	terminal; resources released	anything

The ROS 2 design lists four primary states (**Unconfigured, Inactive, Active, Finalized**), six transition states, and seven transitions exposed to a supervisor (*create, configure, cleanup, activate, deactivate, shutdown, destroy*); "Finalized" is "always terminal". The vocabulary differs between frameworks (F´ topologies construct, configure and start components; cFS starts apps from a startup list; AUTOSAR has ECU-state management), but the structure repeats. **Read the lifecycle documentation of your framework before you read any component.**

The ordering problems

Problem	Example	Class (Lu et al.)
Consumer before queue	a task starts reading a queue that another task has not created	order violation
Producer before consumer is ready	an interrupt enabled before the task that drains it	order violation, then overflow
Subscriber before publisher	no data yet; a late joiner sees nothing (durability)	startup race
Release while in use	a buffer freed while a message still points at it	use-after-free
Two components start each other	A waits for B's topic; B waits for A's service	startup deadlock

Fixes are structural: **create everything in a defined order**; **start sinks before sources**; **gate activation on readiness** (a state, not a sleep); **release in the reverse order**. A `vTaskDelay(100)` "to let the other task start" is a race with a long fuse.

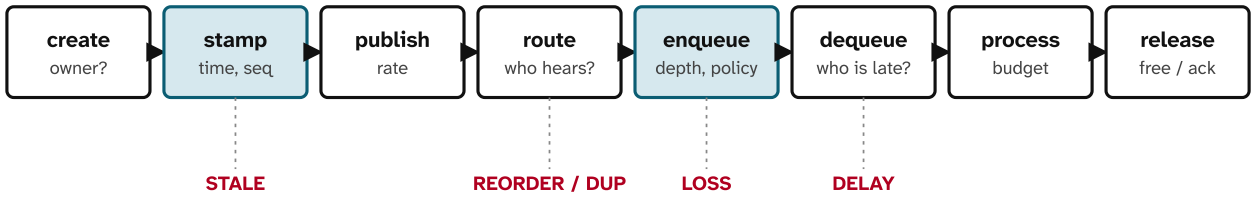
Restart is part of the lifecycle

> CASE FILE **Apollo 11: restart protection.** The guidance software was designed so the computer "could literally be turned off and back on without interrupting" the landing. The mechanism was **waypoints**: at suitable points a job registered where it could safely restart, so that "if processing happened to jump back to the last waypoint, no error would be introduced." After a restart, "processing would commence at the last registered waypoint", only the **most recent copy** of a job was restarted, and non-critical, non-protected functions were simply dropped. Designed against hardware glitches, it also turned out to shed overload (chapter 11).

For every component decide: **what is its state after a restart?** (nothing; reloaded from storage; reconstructed from neighbours); **how does it know it restarted** (a counter, a reset-reason register); **what do its neighbours do meanwhile**. The Spirit rover's boot loop (chapter

14) is the cautionary tale: start-up code that fails *every* time cannot be fixed by restarting it.

A message's life



LOSS at enqueue: queue full → dropped. Is the drop counted and visible?

STALE at stamp: a timestamp assigned late (or by stream position) corrupts every later consumer — Ingenuity Flight 6.

DELAY at dequeue: the consumer is starved by higher-priority work, or blocked on a lock.

REORDER / DUPLICATE at route: several paths or retries (reliable delivery) can deliver twice or out of order.

Rule: every arrow needs an owner, an age limit and a named behaviour when it cannot keep up.

Figure 27. A message's life and where it goes wrong. Every arrow needs an owner, an age limit and a named behaviour when it cannot keep up.

Stage	Who owns the message	What can go wrong	What to check
Create	the producer	allocation fails; fields uninitialised	pool depth; checked allocation
Stamp	the producer	timestamp taken late, from the wrong clock, or by position	stamp where measured, with a named clock (ch. 17)
Publish	hand-off	rate faster than promised	rate monitoring; deadline QoS
Route	the framework	wrong subscribers; duplicates	subscription table
Enqueue	the queue	full → drop or block	policy, counter (ch. 15)
Dequeue	the consumer	consumer late; item aged out	age check on dequeue
Process	the consumer	exceeds budget; blocks	budget, timeout
Release	the last owner	freed twice, or never	single owner; pool accounting

If you can fill in this table for one message in your system, you understand that data path. RAN Lab R10's pipeline has four of these arrows (ISR → raw queue → filter → shared attitude → control → command queue → actuator); the defects planted in it are a missing *Enqueue* counter, a read at two *Process* times, and a *Process* stage with the wrong priority.

Observing a lifecycle

Technique	Use
State variable + log on every transition	the first thing to add; transitions are rare, so logging is cheap
Reset reason + boot counter in `.noinit`	tells the next boot why the last one ended (chapter 19)
Readiness flag per component	gates neighbours instead of sleeps
Queue depth and age at start-up	catches “producer ran ahead”

Check yourself

CHECK

1. List the start-up order of three components that exchange messages. What breaks if you reverse it? What enforces the order today: code, a configuration file, a delay?
2. A component restarts. What does it publish first? What do its subscribers do with a gap?
3. Trace one message end to end in your system with the table above. Where is the first arrow you cannot fill in?

Reliability: watchdogs, containment and recovery

Every component will eventually stop, hang, corrupt something, or run forever. Reliability is the decision, made in advance, about who notices, what is done, how far the damage spreads, and how the system gets back.

Five decisions per failure

Decision	Question	Typical answers
Detect	who notices, and how fast?	watchdog, deadline monitor, heartbeat, assertion, fault handler
React	what happens first?	log, degrade, restart the component, reset the system, enter a safe state
Contain	how far can the damage reach?	separate memory, separate processes, bounded queues, rate limits
Recover	how does it come back?	warm restart with saved state, cold restart, redundant unit takes over
Evidence	what is left for the engineer?	reset reason, saved registers and task states, trace tail, counters

A system that has answered “detect” but not “react” has a good alarm and no fire brigade.

What can go wrong, and what notices it

Failure	Looks like	Noticed by
Crash (fault, assert)	processor halts or resets	fault handler, hardware reset
Hang (infinite loop, deadlock, task blocked forever)	the system runs, one function is dead	**per-task** supervision; queue age; heartbeat
Starvation (higher priority never yields)	low-priority work never runs	a low-priority heartbeat that stops
Slow (overrun)	deadline missed	deadline monitor (chapter 11)
Corruption (stack overflow, race, wild pointer)	wrong values, later crash	invariants, canaries, MPU, checksums
Babbling (floods a bus or queue)	everyone else is slowed	rate limits, bounded queues
Silence (a monitor that fails to monitor)	nothing alarms	a monitor of the monitor

The watchdog, three designs

A **watchdog timer** is a counter that resets the processor if software does not reload (“feed”, “kick”, “tickle”) it in time. It exists to turn any software hang into a reset. Its quality depends entirely on **what must be true before software feeds it**.

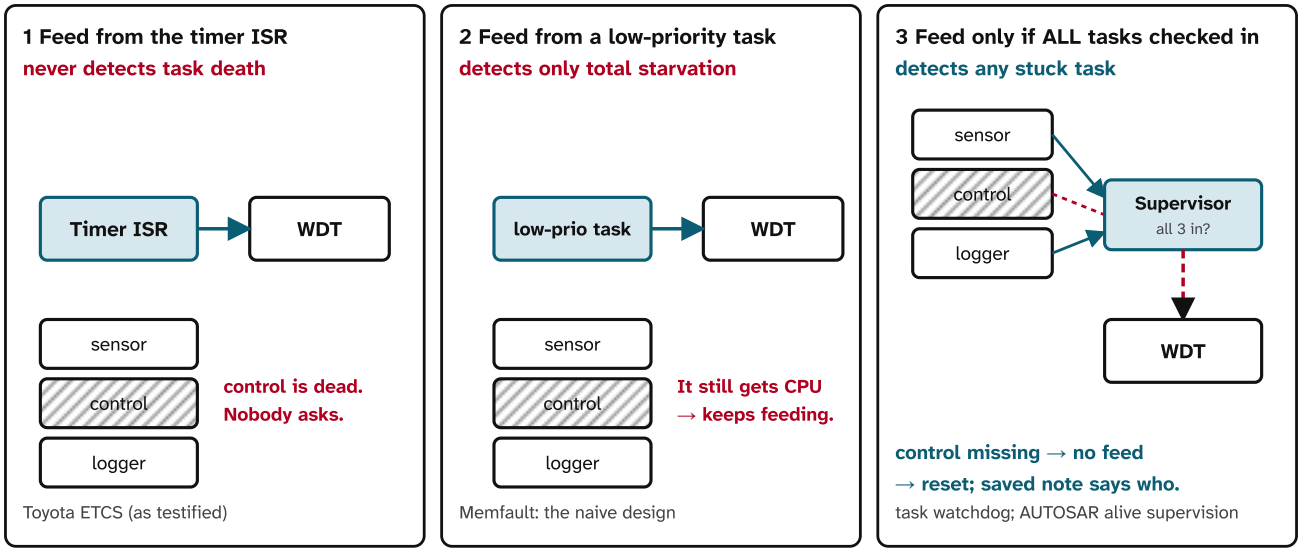


Figure 28. Three ways to feed a watchdog and what each can detect. Only the third notices a single dead task.

RAN Lab R5 injects one fault into a three-task system (the `control` task blocks forever on a lock after 15 cycles) and runs each design:

Design	Watchdog fed by	Outcome in 1.5 s
1	the timer interrupt	150 feeds; `control` stopped at cycle 15 (~300 ms); **no reset, nothing noticed**
2	a lowest-priority task	9 feeds; `control` dead; **no reset, nothing noticed** (the feeder still gets CPU)
3	a supervisor, only if *all* tasks checked in since the last feed	**reset at tick 700**; the next boot read `.noinit` and printed *“tasks that had NOT checked in: control”*

This matches what the sources warn about.

- Memfault’s article demonstrates design 2 failing: “a high priority thread is suspended waiting for a mutex” while “our low priority thread will be scheduled to run by the RTOS and can keep feeding the watchdog”; the fix is “an API tasks can register with” so the hardware watchdog is fed “only if all tasks have recently checked in”.
- Ganssle’s long-standing advice: “Build a watchdog that monitors the entire system’s operation. Don’t assume that things are fine just because some loop or ISR runs often enough to tickle the watchdog timer.” and “It’s fun to watch a multitasking product crash — the entire system might be

hung, but one task still responds to interrupts." The watchdog should "leave breadcrumbs behind, generating debug information for the developers", and software "should [not] be able to reprogram the WDT".

- Koopman's slides (for the plaintiffs in the Toyota litigation) list the same checks: kicking from a timer interrupt "usually a bad sign"; "any single task death must inhibit watchdog kick"; and, as testified for the throttle controller, "watchdog kicked by a hardware timer service routine" with the watchdog detecting the death of the 1 ms task but "not others". SEC
- AUTOSAR's Watchdog Manager names three kinds of supervision: **alive** (checkpoints arrive at the right frequency), **deadline** (the time between two checkpoints is in range) and **logical** (checkpoints occur in the right order).

RULE **Rule.** A watchdog is fed only by code that has *verified* progress of every function you care about. Feed from one place, in a task, never from an interrupt. Use a windowed watchdog if you have one (too early is also an error). Before the reset happens, save *who* was missing.

TRAP **Trap: the debugger and the watchdog.** Many microcontrollers freeze the watchdog while the CPU is halted at a breakpoint; some do not. A watchdog that resets mid-debugging, or one that quietly stops, changes what you observe (Memfault; Ganssle suggests a jumper that disables only the *reset*, not the watchdog logic, so the circuit is still exercised).

Containment: how far does a failure spread?

CASE FILE **Ariane 5, flight 501, 4 June 1996.** The Inquiry Board found the immediate cause in the inertial reference system software, reused from Ariane 4: a conversion of a 64-bit floating-point value to a 16-bit signed integer overflowed. The conversion "was not protected", and "the system specification stated that: the failure should be indicated on the databus, the failure context should be stored in an EEPROM memory ..., and finally, the SRI processor should be shut down." "It was the decision to cease the processor operation which finally proved fatal." The board adds: "the reason behind this drastic action lies in the culture within the Ariane programme of only addressing random hardware failures." Both units ran **identical hardware and software**, so both failed the same way: redundancy protected against random hardware faults and not against a *systematic* software fault. The function that failed was an alignment routine, "operative for 50 seconds after starting of the Flight Mode", not required for Ariane 5. **Lessons.** (1) *What the code does on an exception is a design decision with system-level consequences* — halt, reset, saturate, or carry on? (2) Replicas of the same software share the same bugs. (3) Reused code carries the assumptions of its original envelope.

Containment tools, from cheapest to strongest:

Tool	Contains	Cost
bounded queues and rate limits	floods	a drop policy (chapter 15)
ownership + copy at boundaries	corruption through shared data	a copy
memory protection (MPU) between tasks	wild writes	configuration effort, regions
separate processes or cores	crashes	communication overhead
separate processors or power domains	everything but the interface	hardware

Recovery by design: Pathfinder

CASE FILE **Pathfinder (1997), the recovery that worked.** "We knew that if this problem occurred we would reset. We built in mechanisms to recover the current activity so that there would be no interruptions in the science data … We built in the ability (and tested it) to go through multiple resets while we were going through the Martian atmosphere." The detector was a deadline check between two tasks; the reaction a reset; the recovery state-in-RAM and a designed re-entry. "We had our priorities right" — the team could rank an unreproduced inversion below landing software *because* the failure had a safe, rehearsed outcome. Compare Clementine (1994), described in Ganssle's article: a software hang fired thrusters, and a software timeout on the thrusters "failed when the firmware hung".

CASE FILE **Ingenuity, April 2021: a protection that blocked flight.** The helicopter's first high-speed spin attempt did not reach the flight state; the team traced it to "a computer protection and reliability feature" (the watchdog) and fixed it with a flight-controller software update. A watchdog or monitor that is too strict is also a failure mode: it must be part of the test plan.

Evidence for the engineer on call

Item	How	Lab
Reset reason	read the reset-cause register at boot (Memfault: `RESETREAS` on nRF52)	R5 (`.noinit` marker)
Who was missing	save the check-in mask before resetting	R5
Trace tail	ring buffer survives in RAM; dump after reset	R3, R8
Fault registers	stacked PC, LR, status registers in the fault handler	`startup.c` prints them
Counters	drops, overruns, restarts, uptime	R10

Pathfinder's team could reproduce the bug in under 18 hours *because the trace was already in the flight software*. The debugging-project book (*State First*) is the companion for what to do with such evidence.

A reliability review in one table

Failure to survive	Detector	Reaction	Blast radius	Recovery	Evidence kept
task hangs					
queue overflows					
stack overflows					
sensor goes silent					
watchdog fires repeatedly					
restart fails every time					

If a cell is blank, that is an unmade decision, and it will be made by accident.

Check yourself

CHECK

1. In your system, who feeds the watchdog, from where, and under what condition? What failure would not stop it being fed?
2. A component fails at start-up every time. What does the system do after the third reset?
3. Which of your replicated components share code? What failure would take out both?

Lab links. R5 (watchdog designs), R3 (a deadline monitor), R9 (reading state after the fact).

PART 5 · TRUST · CHAPTER 20

Observability: proving it meets its requirements

“It works on my bench” is a statement about the average. Requirements are statements about the worst case, or about a high percentile. This chapter is about gathering evidence that speaks to a requirement: what to record, how to report it, and what not to trust.

From requirement to evidence

Requirement	Evidence that speaks to it	Evidence that does not
“responds within 5 ms”	latency histogram over a long run with the worst-case load; p99.9 and max	mean latency
“never drops a message”	a drop counter that stays zero over N hours at peak rate	“we haven’t seen any”
“period 10 ms ± 10%”	histogram of intervals between releases	average period
“recovers from a hang”	a test that injects the hang and records the reset and recovery time	a code review of the watchdog
“stack never overflows”	high-water marks after the worst-case test; margin stated	a feeling about call depth

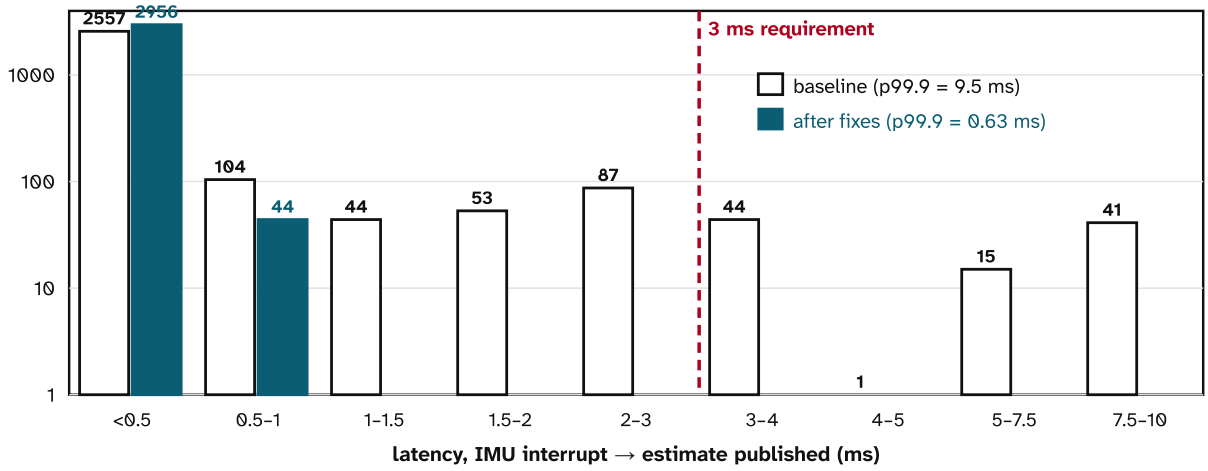
RULE **Rule.** Every requirement gets **one number**, a **method** and a **margin**: *“p99.9 estimate latency ≤ 3 ms (measured at the filter output over 3,000 samples under the worst-case load; margin 4×).”* If you cannot write that sentence, you cannot say it passes.

Means hide tails

RAN Lab R10, the capstone pipeline, baseline run, 2,946 latency samples (IMU interrupt to published estimate):

Statistic	Value
mean	**715 µs** (a dashboard would show this)
median	416 µs
p99	8,320 µs

Statistic	Value
p99.9	**9,496 µs**
above the 3 ms requirement	101 samples (3.4%)



Real data: labs/transcripts/r10_dump*.raw (3000 samples each, lab R10 on QEMU).

Figure 29. Baseline versus the fixed pipeline on a log scale. The baseline's mean is a healthy 715 µs; its tail runs to 10 ms. After four fixes every sample is under 1 ms (p99.9 = 631 µs).

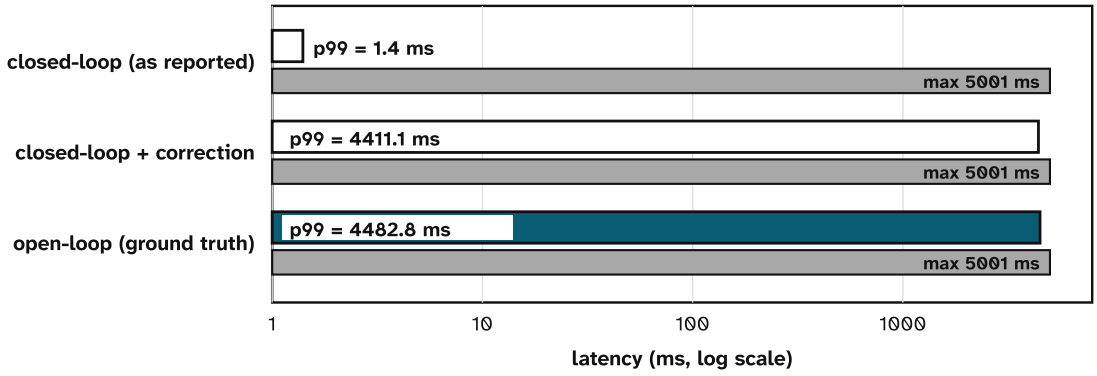
An average of 715 µs is compatible with a requirement of 3 ms at p99.9 being violated **three percent of the time**. Report percentiles and the maximum, from a histogram, and state the sample count and the load.

The measurement trap: coordinated omission

If a tester sends request *n+1* only after the response to request *n*, a stall *suppresses the very samples that would have been slow*. Gil Tene's name for it: "when omission is coordinated with observed events, it can dramatically skew the statistical analysis of the remaining results."

RAN Lab H5: a service that normally answers in about 1 ms stalls for 5 s in a 60 s test at an intended 100 requests/s.

Method	p99	max
closed loop, as reported	**1.4 ms**	5,001 ms
closed loop, corrected (fill in the missed samples)	4,411 ms	5,001 ms
open loop, measured from the *intended* send time (ground truth)	**4,483 ms**	5,001 ms



Real run of lab H5: one 5-second stall in a 60-second test at 100 requests/s.

Figure 30. The same system, three reported tails. The closed-loop tester recorded one slow request (of 5,501); the truth is 558 of 6,000 above 100 ms.

The RTOS version: if an overrunning task *skips* releases and your log only records the response times of jobs that ran, the missing releases never appear. **Count releases, completions and misses separately** (chapter 11). Do not average percentiles; merge histograms (a published property of percentile data, noted in Tene's talk).

What to record, from cheapest to richest

Tool	What it gives	Cost	In the labs
Counters (drops, overruns, restarts)	rates and totals	a few bytes, near zero time	R3, R10
High-water marks (stack, heap, queue depth)	margins	free from the kernel	R6
Timestamps at source (hardware timer)	latency and age	a register read	R2, R10
Latency histogram	the tail	a small array	R4, R10
Run-time statistics	CPU share per task	needs a fast counter; interrupt time is charged to the task interrupted	R8
Context-switch trace	who ran when	a ring buffer and a hook	R3, R8
GPIO toggle + logic analyser	period, jitter, latency with ~zero overhead	a pin and an instrument	(described)
Hardware trace (ETM/SWO) or commercial tracers	everything, with low overhead	tooling	Tracealyzer, SystemView, Perfetto/LTTng named; not run here

The Pathfinder trace. The team's own words: a "trace/log facility … instrumented the pipe services, msgQ services, interrupt handling, select services, and the tExec task. The facility initializes at startup and continues to collect data (in ring buffers) until told to stop", and "we could not have

figured this out ever if we did not have the tools to give us the insight." Leave a ring-buffer trace in the product; make the *scheduler's own error handler* dump it.

RAN The labs' tracer records `(time, event, task id)` at every context switch through one kernel hook (`traceTASK_SWITCHED_IN`) into a 1,024-entry ring. Converted to a timeline, it makes a priority inversion a picture (chapter 13) and an interrupt's interference visible (chapter 4).

Is the maximum I saw the maximum there is?

No. Wilhelm et al. (2008) separate **measurement-based** methods ("a subset of all possible executions") from **static analysis**, and insist a bound be "safe" (never underestimate) and "tight". A measured maximum is a **lower bound** on the worst case. The more complex the processor (caches, pipelines, speculation), the larger the gap. Practical consequences:

1. Call it the **observed maximum (high-water mark)** and say what it was observed under.
2. Drive tests toward the worst case on purpose: slowest paths, cold caches, all interrupts firing, maximum message size, lowest battery.
3. Add margin to budgets, and **validate in the field** with the same counters and histograms.
4. If an upper bound is a hard requirement, use a static timing analysis tool or a conservative design (no caches, simple code, bounded loops, the "Power of Ten").

If a test shows **zero** failures in *N* independent trials, the failure probability per trial could still be as high as roughly 3/N at 95% confidence (the "rule of three", standard statistics). Ten thousand clean runs do not show a 1-in-a-million property.

Observing without disturbing

A debugger stops the CPU; the hardware around it may not stop. RAN In Lab R9 two silent breakpoint stops moved the program's virtual clock ahead by about 1,982 µs (about 1 ms per stop), and in Lab R3 a breakpoint on a function called in the first cycle made the first deadline miss *vanish* (first reset at tick 257 instead of 55). **Rule of thumb: use breakpoints for logic, not for timing.**

For timing questions:

- **Tracepoints / logpoints** that record and continue (gdb `dprintf`, or the lab's conditional `break … if` with `silent … continue`), **only on rarely-called code**;
- **Hardware watchpoints** on the one variable that matters (R9: `watch resets`);
- **Ring-buffer traces** in the program itself; read them afterwards;
- **State inspection after the event**, with a helper that understands the RTOS (`fr-tasks`).

`fr-tasks` is 60 lines of gdb Python that reads the kernel's own lists. It shows what a thread view cannot: which task waits on what, and any inherited priority. The technique transfers to any framework with in-memory task tables.

A requirement-to-evidence table for your system

ID	Requirement	Metric	Method	Margin	Result
R-1	no sample lost	`dropped_raw` over run	counter	zero	
R-2	estimate ≤ 3 ms	p99.9 latency	histogram, 3,000 samples, worst-case load	4×	
R-3	consistent snapshot	`torn` over run	invariant check	zero	
R-4	every task meets its period	`ctrl_late`	period monitor	zero	

Chapter 24 fills this table in for a real defect-ridden pipeline.

Check yourself

CHECK

1. A test reports “average latency 1.2 ms, max 4.8 ms over 1,000 runs”. Which three questions do you ask before believing a 5 ms requirement is met?
2. You cannot afford a trace buffer on a tiny target. Name two cheaper pieces of evidence you can keep.
3. Your test harness waits for each reply before sending the next request. What can it not show?

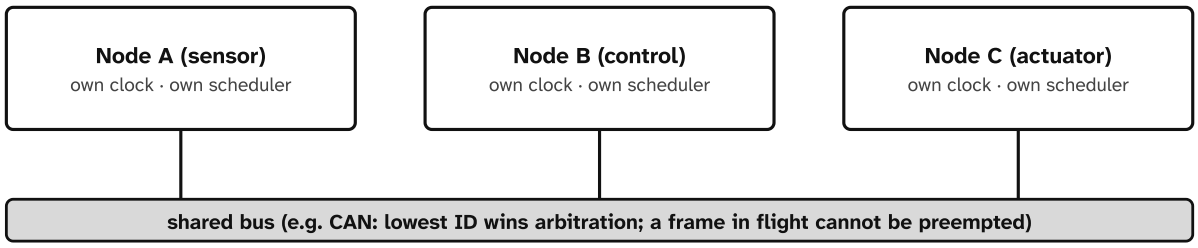
Lab links. H5 (coordinated omission), R8 (trace + statistics), R9 (gdb without breakpoints), R10 (histograms).

PART 5 · TRUST · CHAPTER 21

Beyond one CPU: buses, clocks and other computers

Everything so far assumed one processor with one clock. Real systems add a second core, a bus to another controller, a network to a computer in another room. Each addition brings the same six questions back with new answers.

What changes



New questions that a single CPU never asked:

- Whose clock is 'now'? Drift, offset, sync protocol (PTP / gPTP), leap seconds.
- What is the worst-case delay on the bus? Priority + blocking by the frame in flight (CAN).
- What if a node is silent? Heartbeats / liveliness with a lease; who decides it is dead?
- What order did things happen in? Timestamps from different clocks are not comparable without sync (Lamport 1978).

Figure 31. Three nodes on a shared bus. Each has its own clock and scheduler; the bus has its own arbitration and delays.

Single CPU	With more than one
one clock	many clocks that drift; “now” must be agreed or carried in the data
a call either runs or doesn’t	a message may be lost, delayed, duplicated or reordered
a crash stops everything	one node fails while others carry on (**partial failure**)
a lock protects data	there is no shared lock across a bus; use ownership and protocols
response time = sum of local pieces	add transmission, arbitration, and the other node’s scheduling

Many cores in one chip

On a multicore microcontroller or application processor the new problems are **which core runs what** (pinned or migrating), **shared resources** (caches, buses, memory controllers) that make one core's execution time depend on another's, and **true parallelism** for races (chapter 12): two tasks can now truly run at the same instant, so the "it cannot happen because the other has lower priority" arguments of single-core code no longer hold. FreeRTOS supports symmetric multiprocessing in recent versions; its own book notes the examples assume a single core. Treat a multicore RTOS as a different course: this book's analyses (response-time analysis in particular) are for **one** processor.

A bus is a scheduler too: CAN

The Controller Area Network (CAN) is the dominant in-vehicle bus. Every message has an **identifier**, and the identifier is its **priority**: when several nodes start at once, the lowest identifier wins arbitration and the others wait. A frame already on the bus cannot be preempted. A CAN bus is therefore a **fixed-priority, non-preemptive scheduler**, and a message's worst-case response time has the same shape as a task's (own transmission time, plus blocking by one lower-priority frame already in flight, plus interference from higher-priority messages).

> CASE FILE **CAN schedulability analysis, 1994 → 2007.** Davis, Burns, Bril and Lukkien (2007) report that more than "400 million CAN enabled microcontrollers" were manufactured each year, and that the 1994 worst-case response-time analysis for CAN, "cited in over 200 subsequent papers and transferred to industry in the form of commercial CAN schedulability analysis tools", used by "a large number of major automotive manufacturers" for cars of which "millions" were built, "is flawed. It may provide guarantees for messages that will in fact miss their deadlines in the worst-case." They give revised analysis, and show that the priority-assignment policy "previously claimed to be optimal for CAN, is not in fact optimal." **Lesson:** a published, widely used analysis can be wrong; use tools *and* measurement, and know the model's assumptions.

Time across nodes

Two clocks that start equal drift apart at the rate of their oscillator error: at 50 ppm each, up to 100 ppm relative, or 8.6 seconds per day (chapter 3). Systems that need a shared notion of time use a **synchronisation protocol** (PTP / IEEE 1588 and, for automotive and industrial

Ethernet, gPTP) or put timestamps from a trusted source into the messages. Lamport's 1978 paper makes the underlying point precisely: in a distributed system there is no global clock, only a **"happened before"** relation, and logical clocks give an order consistent with it.

Need	Technique
order of events across nodes	logical clocks, sequence numbers per sender
compare *times* across nodes	synchronised clocks, with a known error bound
measure latency across nodes	timestamp at the source, subtract at the sink **using synchronised clocks**
detect a dead node	heartbeat with a lease; the decision "dead" is a timeout, hence a guess

> TRAP **Trap: subtracting timestamps from two clocks.** The result includes the clocks' offset. A negative latency, or one that jumps when synchronisation steps the clock, is a symptom (chapter 3's Cloudflare case is the same bug on one machine).

Messages between nodes

Property	What can happen	Defence
Loss	frame corrupted or dropped	acknowledgement + retry (adds latency); or tolerate (best effort)
Duplication	a retry after a lost ack	make the receiver **idempotent**; sequence numbers
Reordering	multiple paths	sequence numbers; reorder window
Delay	arbitration, queues, other traffic	deadlines and lifespans (chapter 16)
Partition	a link is down	heartbeats; degraded mode; reconcile on reconnect

"Reliable" delivery does not mean "on time": retries consume bandwidth and time. For control data that must be fresh, a stale retry is worse than a loss (chapter 16).

The same six questions, across nodes

Question	Across a bus or network
Timing	end-to-end budget = source rate + queueing + transmission + remote scheduling + remote processing
Scheduling	bus priority (CAN IDs), network QoS, time-triggered schedules
Messaging	per-link queues, loss policy, ordering
Synchronization	ownership, versioned data, leader election (not locks)
Reliability	heartbeat/lease, redundant paths, safe state on silence
Observability	synchronised timestamps, traces merged by a common clock

Check yourself

CHECK

1. A CAN message with a low priority is 130 µs long. A higher-priority message is released just as it starts. What does the high-priority message's analysis need to include that a preemptive task's does not?
2. Two nodes each timestamp a sample with their own 25 MHz clock at 50 ppm. After one hour, how far apart can the clocks be?
3. A command message is retried after a lost acknowledgement. What must the receiver be able to tolerate?

Reading an unfamiliar framework

You will rarely build the scheduler. You will be dropped into a framework that has already chosen the policy, the queues, the lifecycle and the message model, often automating them away. This chapter is the fast route to finding out what it chose, in the first hour, with a running system and a few files.

The first hour



Gates between steps:

- After 2: can you state the scheduler policy, tick rate and number of priority levels?
- After 3: does every task have a trigger or period, a priority, a stack size and an owner?
- After 4: does every periodic component have a deadline and an estimated or measured budget?
- After 5: do you have one number per requirement (p99.9, max, drops, misses) — not an average?

Then branch: BUILD → component card · DEBUG → symptom triage · UNDERSTAND → trace one message end to end

Figure 32. The first hour in a new RTOS system. Each step has a gate: a question you must be able to answer before moving on.

Start the way the debugging book does: **run it first.** Build with debug symbols, run it (on hardware or an emulator), read the log. Then answer, in this order, the questions below. A blank is not a failure: it is the list of things to ask.

Ten questions to put to any framework

#	Question	Where to look	Why it matters
1	**What is the scheduler policy?** fixed priority, time slicing, EDF, cooperative, table-driven	kernel configuration file; framework docs	chapters 5–10
2	**What is the tick, and what are the rates?** tick frequency; every periodic rate	kernel config; schedule/rate-group tables	chapters 3, 17

#	Question	Where to look	Why it matters
3	**How many threads, at what priorities, with what stacks?**	task list at run time; framework topology/launch files	chapters 6–7, 14
4	**Which code runs in interrupts and in timer callbacks?**	vector table; ISR list; timer service config	chapter 4
5	**What are the queues: depth, policy, counters?**	queue creation, QoS, pipe depth	chapter 15
6	**What is shared, and how is it protected?** mutexes, guarded ports, critical sections	grep locks; port/message definitions	chapters 12–13
7	**How is memory managed?** static or heap; which heap scheme; stack margins	heap config; linker map; high-water marks	chapter 14
8	**What starts, in what order, and what restarts?** lifecycle manager; start-up table	startup code; lifecycle docs	chapter 18
9	**Who supervises whom?** watchdogs; heartbeat; health monitor; reset behaviour	watchdog code; framework health service	chapter 19
10	**How can I see it?** trace hooks, run-time stats, logs, counters, debug interface	config flags; framework telemetry	chapter 20

The kernel configuration file is the first document

In FreeRTOS the file is `FreeRTOSConfig.h`. Ten lines tell you more about the system than any diagram. These are from the lab build (`labs/rtos/common/FreeRTOSConfig.h`), with the question each answers:

Setting	Lab value	Question it answers
`configUSE_PREEMPTION`	1	is it preemptive? (1)
`configUSE_TIME_SLICING`	1	do equal priorities take turns? (1, 7)
`configTICK_RATE_HZ`	1000	what is a tick? (2, 3)
`configMAX_PRIORITIES`	8	how many priority levels? (1, 7)
`configTICK_TYPE_WIDTH_IN_BITS`	32-bit	when does the tick wrap? (3)
`configTOTAL_HEAP_SIZE` + heap scheme	48 KB, `heap_4`	how much memory, which allocator? (7)
`configCHECK_FOR_STACK_OVERFLOW`	2	what detects overflow, and when? (7, 14)

Setting	Lab value	Question it answers
`configUSE_MUTEXES`	1	do mutexes (with inheritance) exist? (6, 13)
`configASSERT`	defined	what happens on a violated assumption? (9)
`configUSE_TRACE_FACILITY`, `configGENERATE_RUN_TIME_STATS`	1	can I measure? (10)
`configKERNEL_INTERRUPT_PRIORITY`, `configMAX_SYSCALL_INTERRUPT_PRIORITY`	`0xE0`, `0xA0`	which interrupts may call the kernel? (4)
`configTIMER_TASK_PRIORITY`	highest	at what priority do timer callbacks run? (7)

Other kernels have the same file under other names: Zephyr's `prj.conf`/Kconfig, AUTOSAR's OS configuration, a cFS build's `*_platform_cfg.h`. Learn to read configuration as a requirements document the previous team left you.

How frameworks package the six questions

	FreeRTOS	Zephyr	F´	cFS	ROS 2
Timing	tick, `xTaskDelayUntil`, timers	`k_timer`, kernel clock	rate groups driven by a cycle port	scheduler table slots	timers in the executor
Scheduling	fixed priority + slicing	fixed priority (+ optional EDF within a level), cooperative threads	thread priority per active component	task priority per app	the executor's order; OS thread priorities
Messaging	queues, notifications	message queues, FIFOs	typed ports; **async port = queue**	**software bus**, pipes	topics + QoS
Synchronization	mutex (inheritance), semaphore	mutex, semaphore	**guarded** ports, queued messages	OS mutex calls	callback groups, locks
Reliability	your design (chapter 19)	your design	health/watchdog components	health & safety app, watchdog	liveliness QoS, lifecycle nodes
Observability	trace hooks, run-time stats	tracing subsystem	telemetry and events from every component	event and housekeeping messages	`ros2 topic`, tracing tools

The cFS row and parts of the Zephyr row were not verified against primary sources in this project; confirm in your own build's documentation. The F´, ROS 2 and FreeRTOS rows follow the pages cited in `research/05` and `07`.

Worked reading: an F´ rate group

JPL's F´ documentation lists three component kinds — **passive** ("no queue and no thread"), **queued** ("a queue but no thread") and **active** ("both") — and three port kinds: **synchronous** (an immediate call), **asynchronous** (a message placed in the receiving component's queue and handled on its thread) and **guarded** (a call protected by the component's mutex). A rate-group component (`Svc::ActiveRateGroup`) "invokes each output port in order" when its cycle port fires and "tracks execution time and detects overruns".

Translate with the book:

1. Each **active component** is a task: it has a priority, a stack and a queue depth (chapters 6, 14, 15).
2. A **rate group** is a cyclic-executive frame running on a thread: the order is the table; the sum of its members' budgets is the frame's load (chapter 5).
3. An **async port** is a queue with a drop or block policy; the depth is in the topology (chapter 15).
4. A **guarded port** is a mutex (chapter 13): find who else calls it, at what priority.
5. **Overrun detection** gives you the "miss policy" for free; find out what it *does* (chapter 11).

Ten minutes of reading the topology file yields the thread list, the queue depths, the rates and the lock structure: the same answers as the first-hour table above.

Evidence over assumption

When documentation and behaviour disagree, believe behaviour. Pathfinder's flight-software lead concluded: "A good lesson when you fly COTS stuff — make sure you know how it works." The executor timing of ROS 2 had to be reverse-engineered before it could be analysed (Casini et al. 2019). Verify the answers to the ten questions by *measuring*, with the tools from chapter 20.

Check yourself

CHECK

1. Open the configuration of a system you work on. Fill in the table of twelve settings. Which did you not know?
2. Pick one component. Draw its threads, queues and locks from the code, then from the running system (tasks and queue depths). Where do they differ?
3. What is the default behaviour of your framework when a rate-group or periodic job overruns?

Handbook link. The *Framework Recon Sheet* and *First-Hour Checklist* in the Field Handbook are these tables as forms.

The implicit requirements

The requirements you are given say what the component does. The ones that decide whether it works on the device are never stated: they are consequences of running on a scheduler, with queues, interrupts and a finite memory, on a machine with a clock. This chapter is the list. Use it as a review checklist and as a prompt for questions to the people who wrote the spec.

How to use the list

Each row is a requirement that is **implicit** in almost every RTOS component. For each, ask the question and look for the evidence. If nobody can answer, you have found a gap before the device did. The chapter column points to the explanation; the handbook has the same list as a checklist you can tick.

1 · Timing

#	Implicit requirement	Ask	Evidence	Ch.
T1	Every periodic component has a **period** and a **deadline**	What are they, and where is each written?	spec or code comment; default D = T is a choice	1
T2	Every component has a **budget** (WCET) and the budget is an **upper bound**	Who set it, how, under what load?	analysis or high-water mark + margin	9, 20
T3	The **miss policy** is defined	What happens to the next release when a job overruns?	framework docs; test	11
T4	**Jitter** is bounded	How much can the release time vary, and who cares?	period histogram	3, 17
T5	**Interrupts and the tick** are in every budget	What fraction of the CPU do they use?	trace, run-time stats	4
T6	**Counters wrap** and clocks have a type	When does each one wrap? Which comparisons break?	width, rate, uptime arithmetic	3

#	Implicit requirement	Ask	Evidence	Ch.
T7	**CPU load** has headroom	What is U, and what is the exact worst-case response?	`rtcalc`, measurement	8, 9
T8	**Rates** are justified and compatible	Why this rate? Harmonic with its neighbours?	control or input argument	17
T9	Delays use **absolute** times for periods	`vTaskDelay` or `DelayUntil`? Initial `last` valid?	code review	2, 3

2 · Scheduling

#	Implicit requirement	Ask	Evidence	Ch.
S1	**Priorities** follow timing, not importance	Who chose them, by what rule?	rate/deadline order	7
S2	No task **spins** or never blocks	Does every task wait on something?	code review; trace	6, 7
S3	**Equal-priority** tasks are fair enough	Who shares a level, and is time slicing on?	config	7
S4	**Timer callbacks** are short	At what priority do they run? Do they block?	timer task priority	7
S5	The **idle task** can run	Can everything block? Are deleted tasks reclaimed?	idle share in stats	6
S6	Overload degrades **predictably**	Which components starve first?	overload test	11

3 · Messaging

#	Implicit requirement	Ask	Evidence	Ch.
M1	Every queue has a **depth** that is justified	From what burst?	formula, test	15
M2	Every queue has an **overflow policy**	Block, drop newest, drop oldest, overwrite?	code, QoS	15, 16

#	Implicit requirement	Ask	Evidence	Ch.
M3	**Drops are counted and visible**	Where is the counter? Who reads it?	counter, telemetry	15
M4	Data has an **age limit** and a **source timestamp**	How old is too old? Stamped where, by what clock?	timestamp check	17
M5	Each **rate crossing** has a rule	Latest, average, hold, interpolate?	design note	17
M6	Each subscription has **QoS** that matches its need	Reliability, depth, durability, deadline, lifespan?	QoS table	16
M7	**Startup order** and readiness are defined	Who must exist before whom? What gates activation?	lifecycle	18
M8	**Message ownership** is clear at every stage	Who frees it? Copy or move?	pool accounting	14, 18

4 · Synchronization

#	Implicit requirement	Ask	Evidence	Ch.
Y1	Every shared datum has an **owner**, a **protection rule**, and a **longest hold time**	Fill the five-fact line	table	12
Y2	Multi-field data is read as a **snapshot**	Are two related values ever read at different times?	invariant check	12
Y3	**Mutexes** (with inheritance) guard shared resources; binary semaphores signal	Which is used where?	code review	13
Y4	**Blocking time** is bounded and counted in response times	What is the longest lower-priority critical section?	B term	9, 13
Y5	Locks are taken in a **fixed order** and never held across blocking calls	Lock-order table?	review	13

#	Implicit requirement	Ask	Evidence	Ch.
Y6	**Library and driver calls** are known to lock or not	What does this call lock, at whose priority?	docs, trace	13
Y7	ISRs use only **ISR-safe** calls and short critical sections	Which calls? How long with interrupts off?	review, latency	4, 12

5 · Reliability

#	Implicit requirement	Ask	Evidence	Ch.
R1	A **watchdog** exists, is fed only after progress of **every** task, and is not fed from an interrupt	Who feeds it, under what condition?	code, injection test	19
R2	The **reaction** to a detected failure is defined	Reset, restart one component, degrade?	design, test	19
R3	**Containment**: a failing component cannot silently take down another	Stack, queue, memory boundaries?	review	14, 19
R4	**Restart** state is defined and tested, including **repeated** restarts	What survives? What if start-up always fails?	restart test	18, 19
R5	**Evidence** survives a reset	Reset reason, check-in mask, trace tail?	post-mortem	19
R6	**Stacks and heaps** have measured margins, overflow is detected	High-water marks? Hook?	margins	14
R7	**Resources that grow** with time are bounded	Files, logs, caches, queues, uptime?	soak test	14
R8	**Silent consumers and sensors** are detected	Heartbeat, lease, age check?	injection test	15, 19

6 · Observability

#	Implicit requirement	Ask	Evidence	Ch.
O1	Each requirement has **one number, a method and a margin**	p99.9? max? over how many samples, under what load?	requirement table	20
O2	Latency is reported as a **histogram**, from **open-loop** measurement	Mean or tail? Does the tester wait for replies?	histogram	20
O3	**Releases, completions and misses** are counted separately	Where?	counters	11, 20
O4	There is a **trace** and it is **left in the product**	Ring buffer? Who dumps it, on what event?	trace	20
O5	The **observation itself** does not change the result	Printing in a task? Breakpoints on hot paths?	cost measured	3, 20
O6	The **observed maximum is not called WCET**	Which words does the report use?	review	9, 20
O7	**Tests drive the worst case**	Slow path, burst, all interrupts, cold cache?	test plan	20

Scoring yourself

CHECK Pick a component you own. For each of the 45 rows above, mark: **known** (you can point to the evidence), **assumed** (you believe it but have no evidence) or **unknown**. A good first target is no *unknown* in Timing and Reliability. The number of *assumed* rows is the size of your risk.

EVIDENCE **Where this list comes from.** Each row corresponds to a mechanism that failed in a cited case: T6 (Patriot, 787, tick wrap lab), T2/T7 (Toyota check, R2 measurements), S2 (the lab's spinning-task trap), M3 (the capstone's silent drops, Northeast blackout), M4 (Ingenuity Flight 6), Y3/Y4/Y6 (Pathfinder), Y2 (Therac-25, the capstone's torn reads), R1 (Toyota testimony, Memfault, lab R5), R3/R4 (Ariane, Spirit), O2 (coordinated-omission lab), O4 (Pathfinder's trace). It is a *checklist derived from incidents and labs*, not a complete theory; extend it with your own.

A worked system: taking over a pipeline

You have inherited a small sample-to-actuator pipeline. It runs. The dashboard says latency is fine. You have been asked whether it meets its four requirements. This chapter does it with the method of the book, using the capstone lab (R10) so every number is a real, reproducible result.

The system and its contract

Component	Trigger	Priority	Work	Writes / reads
IMU interrupt	hardware timer, **997 µs** (the sensor's own clock)	above all tasks	stamp a sample, post to `q_raw` (depth 4)	`q_raw`
filter	each raw sample	4	400 µs; every 250th sample a 6 ms **recalibration**	publishes `att` (struct)
control	periodic, 10 ms	3	150 µs	reads `att`, posts to `q_cmd` (depth 1, overwrite)
actuator	each command	5	200 µs	reads `q_cmd`
logger	periodic, 50 ms	**6**	3 ms of formatting	reads counters

ID	Requirement
R-1	no IMU sample is lost between the interrupt and the filter
R-2	the attitude estimate is published ≤ 3 ms after its IMU interrupt, for 99.9% of samples
R-3	control reads one consistent attitude estimate per cycle
R-4	every task meets its period

Step 1: run it, and read what it says

RAN

```
samples produced   : 3000
estimates published: 2946
commands applied   : 309
mean latency       : 715 us
control cycles     : 309 (expected ~309)
```

Already one line disagrees with R-1: **54 samples produced and never published.** The mean latency looks excellent. (Chapter 20: means hide tails.)

Step 2: write the budget and predict

Ask the analysis before touching the device. List every task with a budget, rank by priority, and compute the worst-case response time of the filter (chapter 9):

Case	Filter response (µs)	Reading
logger at priority 6 (as shipped)	**3,600**	already above R-2's 3,000 µs, **before** counting the recalibration
logger moved to priority 1	**600**	within R-2 with 5× margin
filter's budget includes the 6 ms recalibration	**> 6,000**	no priority arrangement can satisfy R-2

Three things have been predicted without running anything: **the logger is the wrong priority**, **the recalibration cannot live inside the filter**, and (from chapter 15) a queue of 4 cannot absorb the backlog a 3 ms stall creates: 3 ms ÷ 0.997 ms ≈ 3 samples arrive while the filter is stopped, plus the recalibration's ~6 more.

Step 3: see the system

Counters not shown by the dashboard; evidence from a latency dump and a trace.

RAN Histogram of 2,946 estimates: p50 416 µs, p99 8,320 µs, **p99.9 9,496 µs**; 101 samples above 3 ms. The trace shows the cause of the first stretch:

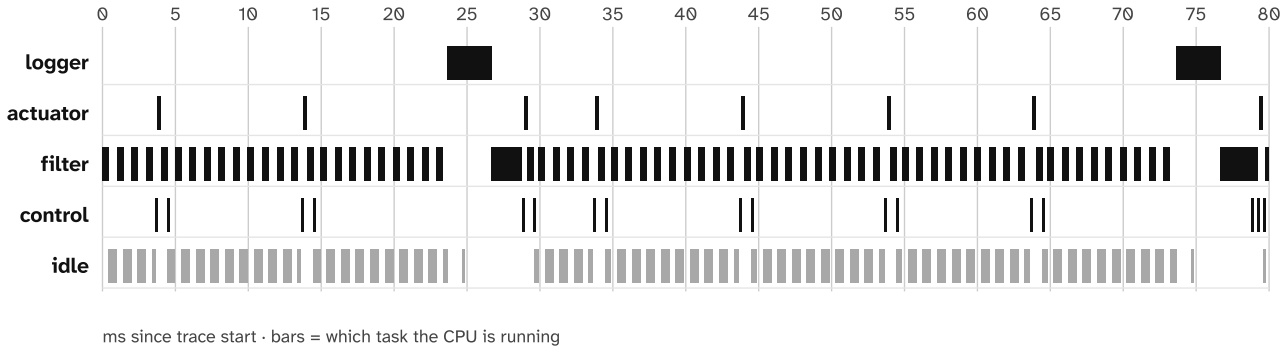


Figure 33. Baseline trace, 80 ms. The filter runs continuously except for a 3 ms gap where the logger (highest priority) runs; during that gap raw samples queue up. Control and actuator run every 10 ms.

A debugger session with a watchpoint or a counter gives the rest. The hidden counters, revealed (`make REVEAL=1`):

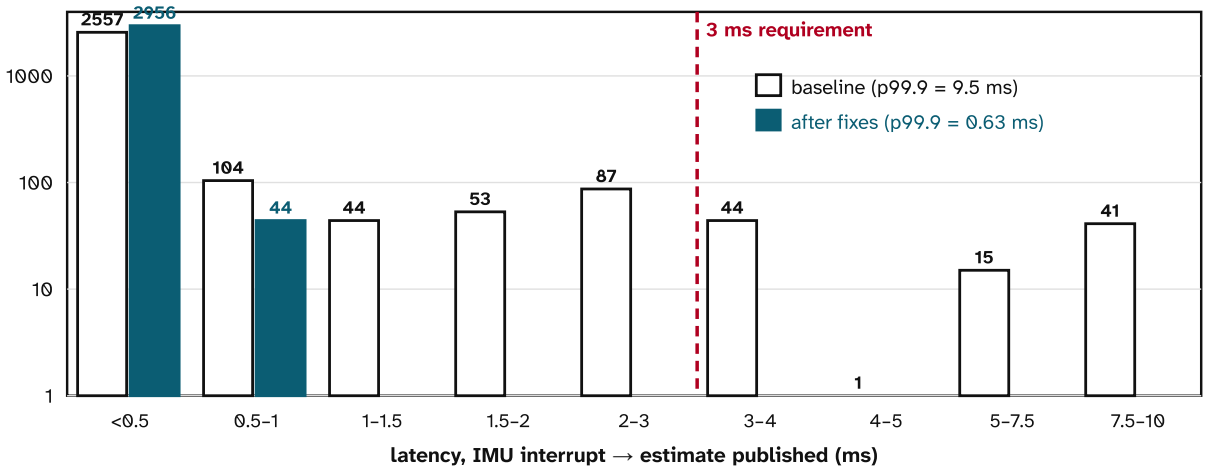
Counter	Baseline	Meaning
`dropped_raw`	**54**	R-1 violated, silently
`torn`	**47** (of 309 cycles)	R-3 violated: control read two different samples
`ctrl_late`	**61**	R-4 violated: control cycle > 11 ms
`isr_late`	0	the interrupt itself is never delayed

Step 4: fix one thing at a time, and re-measure

Each fix is a small patch in `labs/rtos/r10_pipeline/solutions/`. After each, the same run.

Patch	Change	`dropped_raw`	`torn`	`ctrl_late`	p99.9 (µs)
baseline	—	54	47	61	9,496
1	logger priority 6 → 1	22	49	10	6,471
2	queue depth 4 → 8, drop counter printed	**0**	39	10	6,476
3	snapshot under a critical section	0	**0**	10	6,477
4	move the recalibration to a low-priority task	0	0	**0**	**631**

The order teaches as much as the numbers. Patches 1–3 each fix a counted failure but leave R-2 untouched: **p99.9 stays at 6.5 ms because a 6 ms job still sits inside the high-priority filter.** That was never a bug in the code; it is a requirement the *design* cannot meet. Only moving the slow path out (patch 4) changes the tail.



Real data: labs/transcripts/r10_dump*.raw (3000 samples each, lab R10 on QEMU).

Figure 34. Latency of the estimate, baseline versus all four fixes. Every sample is under 1 ms afterwards.

Step 5: fill in the evidence table

ID	Requirement	Metric	Method	Margin	Result
R-1	no sample lost	`dropped_raw`	counter, 3,000 samples	zero	**0** — met

ID	Requirement	Metric	Method	Margin	Result
R-2	≤ 3 ms, p99.9	p99.9 estimate latency	histogram, 3,000 samples, worst-case load	4.8× (631 µs vs 3,000)	**met**
R-3	one consistent estimate	`torn`	invariant check (`angle_inv == ~angle`)	zero	**0** — met
R-4	every task meets its period	`ctrl_late`	period monitor	zero	**0** — met

What is not claimed. This is 3,000 samples (3 s) on an emulator with a deterministic clock. It shows the *method* works and the fixes do what they claim; it does not show the property holds over hours, on a real board, with real jitter, or at the worst-case alignment of the recalibration with the logger. A 95%-confidence statement from 3,000 clean samples is "failure rate below roughly 1 in 1,000" (the rule of three, chapter 20). Soak and fault-injection tests extend it.

What the example demonstrates about the discipline

1. **The implicit requirements were all in the table of chapter 23**: M3 (drops visible), Y2 (snapshot), S1 (priorities follow timing), O1/O2 (percentiles), T2 (the budget included the rare path).
2. **Analysis predicted the main defects before the run**; measurement then quantified them.
3. **Four defects, four kinds of evidence**: a counter, an invariant, a trace, a histogram. No one tool found them all.
4. **Fixing was cheap once found**; finding them without the discipline would have taken weeks of "mean latency looks fine".

> **CASE FILE** **The same pattern at the other end of the scale.** Pathfinder's team had the trace facility, the deadline monitor and a safe reset. They could reproduce the inversion in under 18 hours and fix it by a configuration flag. They had also *seen* it before landing and not reproduced it, and ranked it below other work. The difference between a lab pipeline and a spacecraft is the cost of being wrong; the method is the same.

Your turn

- **TRY IT** Lab R10: run it, **do not** read `main.c` first. Find the four problems using only: the first-look output, `make DUMP=1` and `tools/lathist.py`, `make TRACE=1` and `tools/trace2gantt.py`, and gdb (`labs/rtos/gdb/freertos.py`). Write your hypotheses before each probe (the debugging book's rule). Only then run `make REVEAL=1`.

APPENDICES · APPENDIX A

C survival kit for application engineers

Just enough C and memory to read embedded application and middleware code. Not an encyclopedia; the list of things that surprise people who write in higher-level languages.

Types, sizes and wrap-around

Idiom	Why
Use `<stdint.h>`: `uint8_t`, `int16_t`, `uint32_t`	`int` and `long` have different sizes on different targets
`unsigned` arithmetic **wraps** silently; `signed` overflow is **undefined behaviour**	compare tick differences in unsigned, then cast to signed (chapter 3)
`size_t` for sizes; `TickType_t` for ticks	FreeRTOS: 16 or 32 bits by configuration
`float` may be software-emulated (slow, variable time)	check the target: a “float” in a 1 kHz loop may dominate its budget
a 64-bit read or write on a 32-bit core is **two** accesses	not atomic (chapter 12)

Pointers, in one page

A pointer is a number: an address. `&x` is the address of *x*; `*p` is the value at address *p*. A struct passed **by value** is copied; by pointer, shared. In embedded code you will see:

```
typedef struct { uint32_t seq, t_us; int32_t angle; } att_t;
att_t a;          /* a block of 12 bytes */
att_t *p = &a;    /* p holds the address of a */
p->angle = 5;     /* same as (*p).angle = 5 */
```

- A pointer into a **stack variable** is invalid after the function returns.
- A pointer to a message in a queue slot is **not yours** after the next send or receive unless the API says so.
- `const` on a pointer or array lets the linker place data in flash: use it for tables.

Memory-mapped hardware

A peripheral register is a fixed address; reading or writing it does something. The compiler must not remove, merge or reorder those accesses, so register pointers are `volatile`:

```
#define UART0_DATA (*(volatile uint32_t *)0x40004000)
UART0_DATA = 'x';           /* sends a character */
```

`volatile` means “every access in the source is a real access”; it is *not* atomicity and *not* ordering against other variables (chapters 4, 12). You will seldom write these lines: a vendor library or the framework does. You will read them, and you need to know a statement like `TIMER1_CTRL = 1 | 8` is

“set bits `0` and 3”.

Bit operations

Operation	Meaning
`x | (1u << n)`	set bit *n*
`x & ~(1u << n)`	clear bit *n*
`(x >> n) & 1u`	read bit *n*
`x ^ (1u << n)`	toggle bit *n*

Read-modify-write of a register or shared flag is **not atomic**; if an interrupt can touch the same word, protect it.

Where each variable lives

Declaration	Lives in	Initial value	Note
global / `static` with initialiser	`.data` (RAM, copied from flash at boot)	the initialiser	
global / `static` without initialiser	`.bss` (RAM, zeroed at boot)	`0`	
local variable	the task’s **stack**	**undefined**	stack size matters (chapter 14)
`pvPortMalloc` block	heap	undefined	can fail; can fragment
`const` global	flash	the initialiser	no RAM
`__attribute__((section(".noinit")))`	RAM, not zeroed	**survives a software reset**	post-mortem notes (lab R5)

Things that look right and are not

Code	Problem
`if (count++ > 0)` on a shared variable	read-modify-write race
`memcpy(dst, src, n)` of a struct that another task is writing	torn copy
`char buf[512];` inside a task with a 256-byte stack	overflow, detected late
`printf` in an ISR or a high-priority task	long, may lock, uses a lot of stack
comparing `now >= deadline`	wraps (chapter 3)
`for (;;) { poll(); }` with no delay	starves lower priorities
`static` local in a function called by two tasks	shared state, silently
a large struct returned by value from a function	stack and copy cost

APPENDICES · APPENDIX B

Formulas and numbers to keep

Time and clocks

Quantity	Formula	Example
Counter wrap time	2^bits ÷ clock	2^{32} ÷ 25 MHz = 171.8 s; 2^{32} ÷ 1 MHz = 71.6 min
Tick wrap time	2^bits ÷ tick rate	16-bit @1 kHz: 65.5 s; 32-bit @1 kHz: 49.7 days
Drift from oscillator error	error (ppm) × 10^{-6} × elapsed	50 ppm × 86,400 s = 4.3 s/day
Two clocks	sum of both errors	2 × 50 ppm = 100 ppm = 8.6 s/day
Safe time comparison	`(int32_t)(now - then) >= 0`	valid for intervals < 2^{31} ticks
Sleep error per iteration	work time, if relative	accumulates linearly (lab H1)

Utilization and schedulability

Quantity	Formula
Utilization of a task / set	U = C/T; U = Σ C_i/T_i (include interrupts)
RM guarantee (Liu–Layland)	U ≤ n(2^(1/n) – 1) → 1.000, 0.828, 0.780, 0.757, 0.743, 0.718, … 0.693
Harmonic periods	RM schedulable up to U = 1
EDF (deadline = period)	schedulable iff U ≤ 1
Hyperperiod	LCM of periods
Response time (fixed priority)	R = C + B + Σ_hp ⌈(R + J_j)/T_j⌉ · C_j, iterate from R = C + B
Schedulable	R ≤ D for every task
Slack	D – R
Overload rescaling under EDF	average period T_i × U (when U > 1)

Queues

Quantity	Formula
Queue depth for a burst	⌈(peak rate – service rate) × burst duration⌉ + 1
Time to drain	backlog ÷ (service rate – arrival rate)
Worst latency in a full queue	depth × service time

Quantity	Formula
Little's law (averages)	$L = \lambda \cdot W$

End-to-end

Quantity	Formula
Loose data-age bound, register-based chain	$\Sigma (T_i + R_i)$ over stages
Control rate rule of thumb	$\omega_c \cdot h = 0.2 – 0.6$ (Åström–Wittenmark)
Tolerable jitter (control)	variation < 10% of nominal need not be compensated (Cervin)

Statistics for test evidence

Statement	Rule
Zero failures in N independent trials	failure probability < ~3/N at 95% confidence (rule of three)
Percentiles	do not average; merge histograms
Max over a test	a lower bound on the worst case, never the WCET

Numbers from this project's labs (for orientation, not for your board)

Lab	Result
R1	work 3 ms + `vTaskDelay(10)`: period 12,999 µs, drift +297 ms/99 iterations; `vTaskDelayUntil`: 9,998 µs, −0.15 ms
R2	RTA 1,000 / 3,000 / 10,000 µs → measured 1,006 / 3,024 / 10,087 µs
R2 (U = 0.971)	simulator predicts 20% misses; kernel measured 80%
R4 / H4	19 slots hold a 20-message burst against a 5 ms consumer; 18 lose messages
R6	25,888 B free total, 5,728 B largest block: an 8,000 B request fails
R8	with a 12.5% interrupt: RTA without it 10.0 ms, with it 11.5 ms, measured 11.6 ms
R10	mean 715 µs, p99.9 9,496 µs → after four fixes p99.9 631 µs

APPENDICES · APPENDIX C

FreeRTOS quick map

Names and behaviour follow the FreeRTOS kernel book (v10.6.2-era text) and the kernel V11.3.2 used by the labs. Check your port's documentation for anything not listed.

Which call from where

Purpose	From a task	From an interrupt	Blocks?
create a task	`xTaskCreate` / `xTaskCreateStatic`	—	no
delay	`vTaskDelay(n)`	—	yes
periodic delay	`xTaskDelayUntil(&last, n)` (`vTaskDelayUntil`)	—	yes
queue send / receive	`xQueueSend`, `xQueueReceive` (with timeout)	`xQueueSendFromISR`, `xQueueReceiveFromISR`	task: yes; ISR: never
latest-value mailbox	`xQueueOverwrite` (queue length 1)	`xQueueOverwriteFromISR`	no
mutex	`xSemaphoreCreateMutex`, `xSemaphoreTake`, `xSemaphoreGive`	**not allowed**	yes (inheritance)
binary semaphore (signal)	`xSemaphoreCreateBinary`, `Take`, `Give`	`xSemaphoreGiveFromISR`	yes; **no inheritance**
task notification	`xTaskNotifyGive`, `ulTaskNotifyTake`	`vTaskNotifyGiveFromISR`	take: yes
event group	`xEventGroupSetBits`, `WaitBits`	`xEventGroupSetBitsFromISR` (deferred)	wait: yes
critical section	`taskENTER_CRITICAL` / `taskEXIT_CRITICAL`	`taskENTER_CRITICAL_FROM_ISR`	no (masks interrupts)
suspend scheduler	`vTaskSuspendAll` / `xTaskResumeAll`	—	no
software timer	`xTimerCreate`, `xTimerStart`	`…FromISR`	callbacks run in the **timer task**

Purpose	From a task	From an interrupt	Blocks?
heap	`pvPortMalloc`, `vPortFree`; `xPortGetMinimumEverFreeHeapSize`; `vPortGetHeapStats`	—	—
stack margin	`uxTaskGetStackHighWaterMark`	—	—
run-time stats	`uxTaskGetSystemState` with `configGENERATE_RUN_TIME_STATS`	—	—

Task states

Running · Ready · Blocked (delay or event) · Suspended. The scheduler always picks the highest-priority Ready task; equal priorities rotate each tick if `configUSE_TIME_SLICING` is 1.

Settings that change behaviour (see `FreeRTOSConfig.h`)

Setting	Effect
`configUSE_PREEMPTION`	`0` = cooperative
`configUSE_TIME_SLICING`	`0` = equal priorities run until they block
`configTICK_RATE_HZ`	the tick; every delay is a multiple
`configTICK_TYPE_WIDTH_IN_BITS`	16 or 32 (or 64 on some ports); wrap time
`configMAX_PRIORITIES`	number of levels; `0` is idle
`configKERNEL_INTERRUPT_PRIORITY`, `configMAX_SYSCALL_INTERRUPT_PRIORITY`	which interrupts may call kernel `FromISR` functions
`configCHECK_FOR_STACK_OVERFLOW`	1 or 2 plus `vApplicationStackOverflowHook`
`configUSE_MALLOC_FAILED_HOOK`	`vApplicationMallocFailedHook`
`configASSERT`	what happens when the kernel detects misuse
`configUSE_TIMERS`, `configTIMER_TASK_PRIORITY`	the timer service task and its priority
`configSUPPORT_STATIC_ALLOCATION`	allow no-heap objects
trace macros (`traceTASK_SWITCHED_IN`, …)	hooks for tracing (lab R3, R8)

Gotchas found in this project

Gotcha	Where
`xTaskDelayUntil` with a `last` in the future does not block	ch. 3, lab R3
`xTaskCatchUpTicks` must **not** be called with the scheduler suspended	lab R7
a binary semaphore has **no owner**, so no inheritance	ch. 13, lab R3, R9
stack depth is in **words**	ch. 6, 14
stack overflow is detected at the **next context switch**, after the damage	ch. 14, lab R6
the kernel's "free heap" can be large while the largest block is small	ch. 14, lab R6
`vTaskDelete(NULL)` needs the idle task to run to free memory	ch. 6
run-time-stats clock should be 10–100× faster than the tick (the lab uses 1 MHz ≈ 1000×)	ch. 20, lab R8

Glossary

Term	Meaning
Active object	an object with its own thread and queue that handles one event to completion at a time
Alive / deadline / logical supervision	three watchdog checks (AUTOSAR): checkpoints at the right rate / right spacing / right order
Atomic	indivisible: no other flow of control can observe it half-done
Atomicity violation	code assumed atomic that is interleaved (Lu et al.)
Background / foreground	the main loop and tasks / interrupt handlers
Backpressure	a signal or effect that slows a producer because a consumer cannot keep up
Blocking (term B)	time a task waits for a lower-priority task holding a resource
Budget	the execution time a component is allowed; a *claim* until measured and defended
Cooperative scheduling	tasks switch only when they block or yield
Coordinated omission	measurement that waits for replies and thereby hides the slow ones
Critical instant	the worst-case release alignment: all higher-priority tasks released together with the task
Critical section	code that must not be interleaved; often implemented by masking interrupts or a lock
Cyclic executive	a fixed timetable of calls repeated every major cycle
Data age	how old a value is when it is used
Deadline	latest acceptable completion time of a job, relative to its release
Deadlock	tasks each holding something another needs; none can proceed
EDF	earliest deadline first: run the ready job with the nearest absolute deadline
Frame (minor cycle)	the fixed time slot of a cyclic executive
Harmonic periods	each period is an integer multiple of the shorter ones
High-water mark	the largest use (or smallest free amount) ever observed
Hyperperiod (major cycle)	LCM of the periods
Idempotent	repeating it gives the same result as doing it once

Term	Meaning
Inheritance (priority)	a lock holder temporarily takes the priority of the highest-priority waiter
Interrupt latency	event to first instruction of the handler
ISR	interrupt service routine
Jitter	job-to-job variation in release, start or completion time
Job	one release (instance) of a task
Lifespan (QoS)	maximum age of a message before it is dropped
Liveliness	evidence that a publisher is still alive
Minimum inter-arrival	the shortest time between releases of a sporadic task
Monotonic clock	a clock that never goes backwards
Mutex	a lock with an owner; in FreeRTOS, with priority inheritance
Order violation	two operations assumed in one order with nothing enforcing it
Overrun	a job exceeding its budget (or period)
Preemption	a higher-priority task taking the CPU from a lower-priority one at once
Priority ceiling	a lock's ceiling is the highest priority of its users; holders run at it
Priority inversion	a high-priority task blocked by a lower-priority one (unbounded if medium tasks intervene)
QoS	quality of service: delivery guarantees per connection
Rate group	a set of components run together at one rate (F´)
Rate-monotonic (RM)	shorter period → higher fixed priority
Release	the instant a job becomes ready
Response time	completion minus release, including waiting
Run-to-completion	a handler finishes before another begins; it never blocks
Sporadic	released by events, with a minimum inter-arrival time
Stack high-water mark	minimum free stack ever
Tick	the kernel's periodic timing interrupt and its unit of time
Time slicing	equal-priority tasks take turns each tick
Utilization	C/T for a task; the CPU fraction a set needs
Watchdog	a timer that resets the system unless software feeds it
WCET	worst-case execution time: a **safe upper bound** (an observed maximum is not WCET)

Evidence ledger and sources

The full registry, with URLs, is `research/00-source-registry.md`. Everything in this book that sounds like a fact about a paper, incident or tool is traceable to a row below. **Status: P** = source text read; **S** = reputable secondary read; **Sn** = search summary only; **RAN** = executed in this project.

Papers and standards

Source	Used for	Status
Liu & Layland (1973), *JACM*	assumptions A1–A5, bound, EDF optimality	P
Stankovic (1988), *IEEE Computer*	definition of real-time	Sn
Buttazzo (2005), *Real-Time Systems*	cyclic executive pros/cons; RM vs EDF; overload	P
Audsley et al. (1993), *Software Eng. J.*	response-time analysis	P
Sha, Rajkumar, Lehoczky (1990), *IEEE Trans. Computers*	inheritance and ceiling protocols	P
Lehoczky, Sha, Ding (1989)	88% average breakdown	Sn
Locke (1992)	cyclic executive vs fixed priority	Sn
Wilhelm et al. (2008), *ACM TECS*	WCET: safe/tight, measurement vs analysis	P (key passages)
Cervin (2003), PhD thesis	control timing rules of thumb	P (passages)
Lu, Park, Seo, Zhou (2008), ASPLOS	105 real concurrency bugs	P
Lee (2006), *IEEE Computer*	threads and nondeterminism	P
Eide & Regehr (2008), EMSOFT	`volatile` miscompilation	S
Davis, Burns, Bril, Lukkien (2007), *Real-Time Systems*	CAN analysis flaw	P (abstract)
Casini et al. (2019), ECRTS	ROS 2 executor response-time analysis	P (abstract-level)
Feiertag et al. (2008)	data age vs reaction time	Sn
Dunkels et al. (2006), SenSys	protothreads	P
Holzmann (2006), *IEEE Computer*	Power of Ten	P
Lamport (1978), *CACM*	ordering, logical clocks	P (downloaded; used for the idea)
Tene (2012–15) talk & post	coordinated omission	P
Smotherman, “History of Interrupts”	interrupt history	P
OSTEP (Arpaci-Dusseau)	deadlock conditions, scheduling basics	P
ROS 2 docs (QoS; lifecycle design)	QoS policies; lifecycle states	P

Source	Used for	Status
F´ docs (ActiveRateGroup SDD; component/port selection; cheatsheet)	component kinds, rate groups	P
FreeRTOS kernel book; Zephyr scheduling docs; Linux `SCHED_DEADLINE` docs	kernel behaviour	P
AUTOSAR WdgM; OSEK OS	watchdog supervision types; ceiling resources	Sn

Incident reports and accounts

Case	Source	Status
Apollo 11 alarms	Eyles (2004), AAS	P
Mars Pathfinder	Reeves (1997) email; Jones (1997) summary	P
Patriot, Dhahran	GAO/IMTEC-92-26	P
Boeing 787 counter	FAA/EASA AD 2015-09-07	P
Therac-25	Leveson & Turner (1993)	P
Ariane 501	Inquiry Board report (1996)	P
Spirit FLASH anomaly	Reeves & Neilson (2005)	P
Northeast blackout 2003	Task Force final report; The Register	P (alarm sections) / P (tier C)
Ingenuity Flights 6 & 8	NASA/JPL blog posts	P
Starliner OFT-1	news reports (SpacePolicyOnline)	P (tier C) — used only in `research/`
Cloudflare leap second	Cloudflare post-mortem	P
Toyota ETCS	Koopman (2014) slides, **plaintiff's expert**	P — **[SEC]**, disputed

Things we could not confirm (and how this book treats them)

See `research/09-claims-we-could-not-verify.md`. The important ones: the 787 counter's width and unit (inference only); Toyota material is expert testimony; product release years for RTOSes (omitted); cFS pipe/scheduler defaults (not stated); which software layer assigned Ingenuity's timestamps (not stated by NASA); real-board timing (all labs ran on an emulator with a deterministic clock).

What was executed

RAN Host labs H1–H5 (macOS, Apple clang 17, Python 3.12); RTOS labs R0–R10 on FreeRTOS-Kernel V11.3.2, QEMU 11.1.2 (`mps2-an385`), `arm-none-eabi-gcc` 16.2.0, gdb 18.1. Transcripts: `labs/transcripts/`. **[MANUAL]**: the VS Code `launch.json` example.

Where to go next: ranked reading

Ranked by: written by the people who did the work; carries evidence (data, counterexamples, incident reports); free; fits the six questions. “Read” = opened for this project; “listed” = a standard work we did not open (check before relying on it).

Start here (free, short, high yield)

#	Resource	Why	Status
1	**Eyles (2004), “Tales from the Lunar Module Guidance Computer”**	the best account of why a priority executive beats a time-sliced one, with a real overload	Read
2	**Reeves (1997), Pathfinder email**	five pages on inversion, including how it was found and why it was missed	Read
3	***Mastering the FreeRTOS Real Time Kernel*** (free, GitHub)	ch. 4 tasks, 5 queues, 7 interrupts, 8 resources, 12–13 debugging	Read
4	**Liu & Layland (1973)**, first 10 pages	the source of 69.3% and the assumptions	Read
5	**OSTEP**: Limited Direct Execution, Scheduling, Concurrency bugs, Semaphores	free textbook chapters; the OS background	Read
6	**Memfault Interrupt** (watchdog; context switching; fault analysis)	runnable firmware articles	Read
7	**Koopman’s Toyota slides** (CC BY 4.0)	a worked review applying these topics to a product, **with the expert-witness caveat**	Read

Then, for depth

Resource	Why	Status
Buttazzo (2005), “Rate Monotonic vs. EDF: Judgment Day”	corrects folklore with counterexamples	Read
Audsley et al. (1993)	response-time analysis in nine pages	Read

Resource	Why	Status
Sha, Rajkumar, Lehoczky (1990)	inheritance and ceiling	Read (abstract, §II–III)
Lu et al. (2008)	data on what concurrency bugs are	Read
Wilhelm et al. (2008)	WCET overview	Read (passages)
Cervin (2003) thesis, ch. 2	timing and control	Read (passages)
Holzmann (2006), Power of 10	two pages of coding discipline	Read
Dunkels et al. (2006), protothreads	stackless concurrency	Read
Davis et al. (2007)	a cautionary tale about published analysis	Read (abstract)
Gil Tene, “How NOT to Measure Latency”	reporting latency honestly	Read
Ganssle, “A Guide to Watchdog Timers”	practitioner watchdog design	Read
Valvano (UT Austin), *Embedded Systems* and the RTOS volume; edX course	build a tiny RTOS from a Cortex-M	Syllabus read

Standard textbooks (listed, not opened here)

Jane W. S. Liu, *Real-Time Systems* · Burns & Wellings, *Real-Time Systems and Programming Languages* · Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications* · Buttazzo, *Hard Real-Time Computing Systems* · Klein et al., *A Practitioner’s Handbook for Real-Time Analysis* · Koopman, *Better Embedded System Software* · Yiu, *The Definitive Guide to ARM Cortex-M*.

Framework documentation

FreeRTOS · Zephyr (Kernel → Scheduling) · F´ (component and port selection; rate-group SDDs) · cFS (cFE documentation) · ROS 2 (QoS; managed nodes) · AUTOSAR (Watchdog Manager) · Linux (`SCHED_DEADLINE`, real-time docs).

A path

If you have…	Read	Do
one afternoon	Eyles, Reeves, this book’s ch. 1, 7, 9, 13, 15, 19, 20, 23	R3 and R10
one week	the FreeRTOS book ch. 4–8; OSTEP scheduling and concurrency; this book Parts 1–3	H3, R1–R3, R6
one month	all of this book; Buttazzo; Audsley; Lu; Wilhelm	every lab; then port R10 to your framework